The chemsim Manual

An emergent chemistry engine, explained from the beginning

A self-contained guide for a reader who knows physics and has never done any chemistry.

Molecules are graphs. Reactions are graph-rewrite rules. Everything else — yields, side products, boiling points, which product wins at which temperature — is computed by integrating the system forward in time, rather than looked up.

Layer 7	the window: a worker thread over the engine
Layer 6	the headless stepper: events, waits, save and replay
Layer 5	the vessel: four phases, pressure, energy balance
Layer 4	the numeric core: stiff ODEs, activity, the Rust seam
Layer 3	the network: discover reactions, derive reverse rates
Layer 2	templates: SMARTS graph rewrites plus kinetics
Layer 1	properties: thermochemistry, volatility, condensed phase
Layer 0	matter: molecular graphs, canonical identity

Generated from the `chemsim` repository • 29 August 2026

Contents

Preface: what this document is

You have a simulator on your disk with roughly 46,000 lines of Python in it, a corpus of 1,583 chemical compounds, and about a million words of design notes. This manual is an attempt to explain all of it, from the beginning, to somebody who knows physics and does not know chemistry.

It is written on three assumptions.

First, that you know physics. Not that you remember it, but that the shapes are familiar: partition functions, free energy, Boltzmann factors, coupled first-order ODEs, stiff systems, conserved quantities, reference levels that can be chosen freely as long as you choose them consistently. Almost every chemical idea in this project is one of those wearing different clothes, and the fastest route to understanding it is usually to find out which one. Wherever that is true, there is a green box saying so.

Second, that you do not know chemistry. So a *mole* gets defined, and so does an *ester*, a *pKa*, and what it means for a template to be "SMARTS on a mechanism". Nothing is assumed except the physics.

Third, that the interesting part of this project is not the chemistry and not the code, but the seam between them — the repeated decision about *what to parameterise and what to compute*. That decision is made about thirty times in this codebase, and it is made differently each time for a stated reason. Most of this manual is about those reasons.

0.1 The one-sentence version

The project stores *reactions as transformation rules* and *molecules as graphs*, computes every number it can from molecular structure, and then integrates the resulting system of differential equations forward in time — so that yields, side products, boiling points, temperature sensitivity and failure modes come out as *consequences* rather than as things somebody typed in.

That is the whole thesis. Everything else is the price of making it true.

0.2 How to read it

The manual has six parts and a set of appendices. They are meant to be read in order, but they are also meant to be abandonable.

Part I — Chemistry from a standing start. Eleven short chapters covering the chemistry the rest of the manual needs: what a molecule is, what a mole is, what free energy has to do with which way a reaction goes, what a rate constant is, what happens when a liquid boils, what an ion is, and what an electrode does. If you have never done chemistry, this is the part that matters. Nothing in it is specific to this project.

Part II — Where the numbers come from. Every equation in Part I has constants in it, and somebody has to supply them. This part is about estimating chemical properties from molecular structure, which is a curve-fitting problem with unusually sharp teeth.

Part III — The machine. The architecture, layer by layer, bottom to top. This is the part that maps onto files.

Part IV — Numerics. Stiffness, the ODE solver, the Jacobian, and what is and is not conserved.

Part V — What comes out. The behaviour nobody wrote down, and how the project measures how much chemistry it actually covers.

Part VI — Limits. What it cannot do, and why each of those is a specific missing model rather than a general shortfall.

Then the appendices: a glossary, a symbol table, a map from concepts to files, and the commands that run things.

0.3 The four kinds of box

Boxes are used sparingly and each colour means one thing.

> **THE POINT**
>
> The idea you should carry out of the surrounding pages. If you read only these, you will have the shape of the thing and none of the detail.

> **IF YOU KNOW PHYSICS**
>
> A translation into something you already know. These are the shortcuts.

> **A TRAP THIS PROJECT ACTUALLY FELL INTO**
>
> A specific mistake that was made in this repository, found by measurement, and recorded. There are a lot of these and they are the most valuable content in the project's own notes — a wrong answer that *looked right* is worth more than a right answer, because it tells you where the model's edges are.

> **ASIDE**
>
> A digression that is interesting but skippable.

0.4 About the numbers and figures in this manual

Wherever a figure could be computed by the actual simulator, it was. The vapour-pressure curves, the ethanol–water azeotrope, the boiling plateau, the benzoic-acid solubility curve and the lime-kiln equilibrium in this document are all outputs of the code in `src/chemsim`,

generated by `docs/manual/make_figures.py`. They are not illustrations of what it is supposed to do.

As a spot check of that claim: the boiling plateau in Chapter 7 comes out at **351.46 K** against a measured 351.4 K for ethanol, and the benzoic-acid curve in Chapter 9 passes through **3.24 g/L** at 298 K against a measured 3.44 — while the same equation with one term deleted gives **1347 g/L**. Neither number is stored anywhere in the code.

Quoted numbers that are not from a figure are taken from the project's own `README.md`, `MILESTONES.md` and `data/catalog/COVERAGE_REPORT.md`, which are themselves generated by running things.

0.5 A note on tone

The source files in this project are unusually argumentative. They do not just say what the code does; they say what was tried, what was measured, what was refused, and why. That style is worth preserving here, because in a simulator the difference between a good model and a bad one is very rarely visible in the output — both produce plausible-looking numbers. The only defence is to write down the reasoning and the measurement beside every choice, so that a wrong one can be found later by reading rather than by accident.

This manual therefore spends as much time on refusals as on features.

PART I

Chemistry from a standing start

CHAPTER 1

Matter, and why a molecule is a graph

1.1 Atoms

An atom is a nucleus — protons and neutrons, carrying essentially all the mass — surrounded by electrons. The number of protons, Z, is the *element*: $Z = 1$ is hydrogen, $Z = 6$ carbon, $Z = 8$ oxygen, $Z = 26$ iron. Chemistry is almost entirely about the electrons; the nucleus participates only by setting Z and by being heavy enough that we may treat it as stationary while electrons move (the Born–Oppenheimer approximation, which is why molecular *structure* is a meaningful idea at all).

Electrons occupy orbitals of increasing energy. The ones in the outermost occupied shell — the **valence** electrons — are the ones close enough to another atom's valence electrons to matter. The energetics work out such that an atom is unusually stable when its valence shell is full: two electrons for hydrogen, eight for most of the second row. This is the "octet rule", and while it is a rule of thumb rather than a law, it is a very good one for the light elements that make up most of organic chemistry.

1.2 Bonds

Two atoms that each need an electron can share a pair, and both then count it towards a full shell. That shared pair is a **covalent bond**. It is a genuine energy minimum: the potential energy of the two-atom system, plotted against their separation, has a well of depth typically 300–500 kJ/mol and a minimum at around 10^{-10} m.

> **IF YOU KNOW PHYSICS**
>
> A bond is a bound state. Breaking one costs its well depth; making one releases it. Almost all of chemical energetics is bookkeeping over which bonds were broken and which were made, and this is exactly why the *group contribution* methods of Part II work at all: if a molecule's energy is roughly a sum over its bonds, then it is roughly a sum over its parts.

An atom's **valence** is how many bonds it wants: hydrogen 1, oxygen 2, nitrogen 3, carbon 4. Two atoms can share two pairs (a *double bond*) or three (a *triple bond*). Bond *order* is that count, and higher order means shorter and stronger.

A special case that matters constantly: in a ring of six carbons with alternating single and double bonds — benzene — the electrons are not localised into three double bonds at all. They are spread over the whole ring, and the ring is about 150 kJ/mol more stable than the alternating

structure predicts. This is **aromaticity**. It is why benzene rings survive conditions that destroy almost anything else, and why the whole of Chapter 18's Hammett machinery exists.

1.3 So: a molecule is a graph

Put those together. A molecule is:

- a set of atoms, each labelled with its element, its charge, and how many hydrogens hang off it;
- a set of bonds between them, each labelled with an order.

That is a **labelled undirected graph**. Not "can be modelled as" — the correspondence is exact for the purposes of everything in this project. Two molecules are the same substance if and only if their graphs are isomorphic with labels preserved.

> **THE POINT**
>
> The single most consequential representational decision in this project is that a molecule is a graph and a reaction is a graph rewrite. Everything downstream — that reactions generalise across substrates, that products can be *discovered* rather than enumerated, that properties can be estimated from structure — follows from it.

1.4 SMILES: writing a graph on one line

Graphs are awkward to type. **SMILES** (Simplified Molecular Input Line Entry System) is a compact linear notation for exactly this graph.

The rules you need to read this manual:

SMILES	means
`C`	a carbon, with hydrogens filled in to satisfy valence: methane, CH_4
`CC`	two bonded carbons, each filled with hydrogens: ethane, C_2H_6
`CCO`	carbon–carbon–oxygen: ethanol
`O`	one oxygen with two implicit hydrogens: water
`C=C`	a double bond: ethene
`C#N`	a triple bond
`c1ccccc1`	six aromatic carbons in a ring: benzene (lower case = aromatic)
`CC(=O)O`	acetic acid: parentheses are branches
`CC(=O)OCC`	ethyl acetate
`[Na+]`	an explicit atom with a charge

SMILES	means
`[O-2]`	oxide ion
`OC(=O)c1ccccc1`	benzoic acid

Hydrogens are usually implicit, which is a convenience that will cause exactly one serious bug later (Chapter 18).

The important property is that SMILES is *canonicalisable*: a given graph has many valid SMILES strings, but a canonicalisation algorithm maps all of them to one. This project uses the canonical form as a species' identity, which is why `chemsim.matter.Molecule` compares equal iff canonical SMILES match, and why a `Molecule` can be a dictionary key.

1.5 SMARTS: writing a *pattern* over graphs

SMILES describes one molecule. **SMARTS** describes a *class* of substructures — it is to SMILES roughly what a regular expression is to a string.

`[CX4;!H0:1][OX2H1:2]` is a SMARTS pattern from this project's oxidation template, and it reads:

- `[CX4...]` — a carbon with four connections (i.e. saturated),
- `;!H0` — which does *not* have zero hydrogens (so it has at least one),
- `:1` — call this atom number 1,
- `[OX2H1:2]` — bonded to an oxygen with two connections and one hydrogen (i.e. an alcohol's `-OH`), call it atom 2.

That single pattern is a complete selectivity model for a family of reactions, and the project's notes say so explicitly: it matches a primary alcohol and a secondary one, it *refuses* a tertiary alcohol (no hydrogen on the carbon to lose), and on glycerol it matches at two different sites and therefore produces two different products from one rule. None of that was enumerated. It follows from the pattern.

> **THE POINT**
>
> Reaction *selectivity* — which product forms, and which substrates a rule applies to — is encoded as SMARTS specificity. That is where the curation effort in this project goes, and writing a pattern carelessly is the main way to get a confidently wrong answer.

1.6 Isomers, and three kinds of sameness

Two molecules can have identical formulas and be different substances.

Structural isomers differ in connectivity: `CCO` (ethanol, a drinkable liquid boiling at 351 K) and `COC` (dimethyl ether, a gas) are both C_2H_6O. The graphs are not isomorphic, so they are different species and this project handles them correctly by construction.

Stereoisomers have the same connectivity but a different arrangement in space: a carbon with four different groups on it comes in two mirror-image forms, and a double bond can have its

substituents on the same side (*cis*/Z) or opposite sides (*trans*/E). This project *does* distinguish them — RDKit's canonical SMILES is isomeric by default, so `C/C=C/C` and `C/C=C\C` are different state-vector entries.

> **A TRAP THIS PROJECT ACTUALLY FELL INTO**
>
> The identity model is *ahead of* the reaction model here, and the project says so in `matter/molecule.py`: templates do not yet control stereochemistry, so a graph rewrite can silently scramble a stereocentre it was not thinking about. Distinguishing things you cannot control is the safer of the two failure modes, but it is still a mismatch.
>
> There is a measured consequence in the corpus. A sugar can be written as a six-membered ring (*pyranose*) or a five-membered one (*furanose*), and the catalog spelled glucose one way and fructose the other. The two are genuinely interconvertible in reality; to the graph model they are unrelated species, and the isomerisation between them was priced at $K = 4.8 \times 10^{-8}$ — a confident number describing the wrong question.

Tautomers are the third kind, and this project does not handle them: they interconvert by moving a hydrogen and a double bond, fast enough that in practice you have a mixture, but they are distinct graphs. Keto and enol forms are separate species here. This is recorded as a known limitation.

1.7 Where this lives in the code

`src/chemsim/matter/molecule.py`, all 263 lines of it, and nothing above it in the stack imports RDKit. That is the project's **first inversion boundary**: parsing, canonicalisation, substructure matching and template application are all RDKit calls, and they all happen inside this one module, so the cheminformatics backend is replaceable without touching anything else.

CHAPTER 2

Counting matter: moles, concentration, and balance

This chapter is short and entirely mechanical, but every equation in the rest of the manual is dimensionally anchored to it.

2.1 The mole

Molecules are small and numerous, so chemistry does not count them individually. The **mole** is a fixed count:

$$1 \text{ mol} = N_A = 6.02214076 \times 10^{23} \text{ entities.}$$

It is a unit in the sense that "dozen" is a unit. Its usefulness is that the mass of one mole of a substance in grams is numerically its molecular mass in atomic mass units — one mole of water (H_2O, mass 18) is 18 g. So a bench chemist weighing out 18 g of water knows they have N_A molecules without ever thinking about N_A.

> **IF YOU KNOW PHYSICS**
>
> The mole exists so that the two scales chemists work at — the molecular and the weighable — differ by a constant rather than by an act of imagination. $R = N_A k_B = 8.314$ J/(mol K) is Boltzmann's constant per mole, and every RT in this manual is a $k_B T$ that has been rescaled. When you see $\exp(-E_a/RT)$, read $\exp(-\epsilon/k_B T)$.

2.2 Concentration, and why this project mostly avoids it

Concentration is amount per volume. The chemist's unit is *molarity*:

$$[\text{A}] = \frac{n_A}{V}, \qquad \text{mol/L, written M.}$$

Rate laws are written in concentrations, because a reaction rate depends on how often two molecules meet, which depends on how crowded the liquid is.

> **BUT THE STATE VECTOR IS IN MOLES**

This project's vessel integrator stores **moles**, not concentrations, and the reason is worth stating early because it recurs. Concentration needs a volume in the denominator, and the liquid volume is *itself a state variable*: it shrinks as things boil off or crystallise out, and it goes to zero when the flask boils dry. Moles stay meaningful at that limit; concentrations do not. So the state is moles and the RHS divides by a volume computed on the fly, guarded against zero.

Mole fraction is the other common measure — $x_i = n_i / \sum_j n_j$, so $\sum_i x_i = 1$. Phase equilibrium is naturally written in mole fractions (Chapters 7–9); rates are naturally written in molarity. Both appear.

2.3 A reaction, written down

A chemical equation is a statement about counts:

$$\mathrm{CH_3COOH + C_2H_5OH \rightleftharpoons CH_3COOC_2H_5 + H_2O}$$

(acetic acid plus ethanol gives ethyl acetate plus water — this is *Fischer esterification*, the project's running example). The numbers in front, when they are not 1, are **stoichiometric coefficients**. Written as a vector ν, negative for reactants and positive for products, a reaction is exactly a column of an integer matrix, and the composition changes along it:

$$\frac{\mathrm{d}\mathbf{n}}{\mathrm{d}t} = \nu\, r$$

for a single reaction proceeding at rate r. For a network of m reactions over n species this becomes $\mathrm{d}\mathbf{n}/\mathrm{d}t = \Delta^{\mathsf{T}}\mathbf{r}$ with Δ the $m \times n$ stoichiometric matrix, and *that matrix product is essentially the whole of the numeric core.* It is what `chemsim/numerics/integrator.py` evaluates.

2.4 Balance is a conservation law, and it is enforced

A chemical equation must balance: the same number of atoms of each element on both sides, and the same total charge. This is not a convention, it is conservation of nucleons and of charge, and it means Δ has a null space.

Let E be the $n \times k$ matrix whose (i, e) entry is the number of atoms of element e in species i, with an extra column for charge. Then balance is

$$\Delta E = 0.$$

THE POINT

This project *checks* that identity at network-build time and rejects a template that violates it, rather than integrating it and producing matter. From the project's own notes: "Element and charge conservation are enforced — a malformed template that doesn't balance is rejected, not

> silently integrated." It is the oldest guardrail in the codebase and it has caught real bugs at every level, including in the hand-authored catalog, where a check added late found that **75 of 367 rows do not balance**.

Because $\Delta E = 0$ holds exactly in integers, $\mathbf{n}^{\mathsf{T}} E$ is conserved by the reaction terms *to machine precision*, not merely to solver tolerance. Transport between phases moves moles between blocks of the state vector without changing their identity, so it conserves the same quantity. This is why the project can assert exact element conservation across all four phase blocks and mean it. (Energy is a different story; see Chapter 27.)

2.5 Units, once, so it never has to be said again

Internally the project is SI-ish and unit objects never enter the hot loop:

quantity	unit
amount	mol
concentration	mol/L (M)
temperature	K
energy	J/mol
pressure	bar
volume	L
time	s
rate constant A	whatever makes rate come out in mol/(L s)

That last row is not a joke. The units of a rate constant depend on the overall order of the reaction — s^{-1} for first order, $\mathrm{L\ mol^{-1}\ s^{-1}}$ for second, and $(\mathrm{L/mol})^{8}\ \mathrm{s}^{-1}$ for the ninth-order sulfur-burning declaration in Chapter 18. This is a genuine nuisance and the project handles it by not representing units at all below Layer 4 and being disciplined instead.

CHAPTER 3

Energy: enthalpy, entropy, and which way a reaction goes

This is the chapter that does the most work in the rest of the manual. If you retain one thing from Part I, retain this.

3.1 The question

Mix two things. Does anything happen? Chemistry answers this with a single scalar, and the physics behind it is one you know: a system at fixed temperature and pressure, exchanging heat with a reservoir, minimises its **Gibbs free energy**.

3.2 Enthalpy

At constant pressure, a system that expands does work $p\,\mathrm{d}V$ on its surroundings, so the heat it absorbs is not $\mathrm{d}U$ but $\mathrm{d}U + p\,\mathrm{d}V$. Define

$$H = U + pV$$

and the heat absorbed at constant pressure is exactly ΔH. That is all enthalpy is: internal energy with the expansion bookkeeping folded in, so that "heat released" is a state function.

A reaction with $\Delta H < 0$ releases heat — **exothermic**; a flask gets warm. $\Delta H > 0$ absorbs it — **endothermic**; a flask gets cold, or needs a burner. Typical magnitudes: 50–200 kJ/mol for making or breaking a bond, and this project's lime kiln runs at $\Delta H = +179.2$ kJ/mol.

3.3 Entropy

$S = k_B \ln \Omega$, and per mole, $S = R \ln \Omega$. Nothing surprising. What matters chemically is that entropy has two reliable sources:

- **more particles.** A reaction that turns one molecule into two increases the entropy substantially — roughly +150 J/(mol K) per extra gas molecule. This is why decompositions are entropy-driven and why heating helps them.
- **more disorder in position.** Gas ≫ liquid > solid. Vaporising a mole of a liquid costs about +90 J/(mol K) (Trouton's rule, and it is surprisingly universal).

3.4 Gibbs free energy

$$G = H - TS, \qquad \Delta G = \Delta H - T\Delta S.$$

A process at constant T and p runs spontaneously if $\Delta G < 0$.

IF YOU KNOW PHYSICS

This is the Legendre transform you would write down anyway. Maximising the *total* entropy of system-plus-reservoir, with the reservoir's entropy change being $-\Delta H/T$, is the same statement as minimising $H - TS$ for the system alone. Equivalently, G is $-k_B T \ln Z$ for the isothermal–isobaric ensemble, per mole. Chemists write G because a flask is at fixed T and p by default; nothing deeper is going on.

The two terms compete, and T sets the exchange rate between them. An endothermic reaction that increases entropy ($\Delta H > 0, \Delta S > 0$) is forbidden when cold and allowed when hot, with the crossover at $T = \Delta H/\Delta S$. That single sentence explains kilns, cracking, distillation, and why a lime kiln has to be hot: 179,190 J/mol divided by 160.3 J/(mol K) is 1118 K, and Figure 9.2 says the same thing with a curve.

3.5 Standard states, which is the subtle part

G is only defined up to a reference, exactly like a potential. Chemistry fixes the reference with two conventions.

First: elements in their standard state have zero. Define the *standard enthalpy of formation* ΔH_f° of a compound as the enthalpy change making one mole of it from its elements in their most stable form at 298.15 K and 1 bar. Then $\Delta H_f^\circ = 0$ for O_2 gas, for graphite, for liquid bromine, for rhombic sulfur — by definition, not by measurement. A reaction's enthalpy is then a difference of tabulated formation values:

$$\Delta H^\circ_{\text{rxn}} = \sum_i \nu_i \, \Delta H^\circ_{f,i},$$

and the element terms cancel automatically because the reaction balances. This is the chemical equivalent of choosing a zero of potential energy: entirely arbitrary, entirely consistent, and it collapses an N^2 table of reaction energies into an N table of formation energies.

AN ESTIMATOR THAT RETURNS A NON-ZERO VALUE FOR A REFERENCE STATE IS PROVABLY WRONG

Because $\Delta H_f^\circ = \Delta G_f^\circ = 0$ for an element in its standard state is a *definition*, an estimator that returns anything else has been caught red-handed. This project has caught three:

species	estimator said	error, as a factor in K
Cl_2	$\Delta H_f = -74.81$ kJ/mol	$\sim 10^{13}$

F_2	$\Delta G_f = -440.5$ kJ/mol	astronomical
S_8	$\Delta G_f = +275.96$ kJ/mol	$\sim e^{91}$

The first was fixed species by species, which is why the lesson did not generalise and the other two survived. The class fix is that `properties/thermochemistry.py` now **refuses to let any estimator price an elemental species at all** — it comes from a curated table or it is refused by name. See `properties/element_data.py`.

Second: the standard state has to say what phase, and this is where it gets sharp. A reference state that is a *gas* has an ideal-gas formation value of exactly zero. A reference state that is *condensed* does not — its ideal-gas record is its vaporisation or sublimation energy, which is a real measured number:

element	reference state	ΔH_f (ideal gas)	ΔG_f (ideal gas)
H_2, N_2, O_2, F_2, Cl_2	gas	0	0
Br_2	liquid	+30.90	+3.08
Hg	liquid	+61.40	+31.85
I_2	solid	+62.40	+19.29
S_8 (rhombic)	solid	+100.42	+48.68

A TRAP THIS PROJECT ACTUALLY FELL INTO

Bromine and iodine were pinned to 0.0 in this repository before `element_data.py` existed. The species-by-species fix for the chlorine bug put a 62 kJ/mol error *into* iodine while taking a 75 kJ/mol error *out of* chlorine — the same bug one level up. This is the clearest example in the project of why a fix has to close a class rather than a member.

3.6 Moving between standard states

Group-contribution thermochemistry (Chapter 12) is **ideal-gas** data: it describes an isolated molecule at 1 bar. Almost every reaction in this simulator happens in a *liquid*. Using the gas numbers unmodified is not a small approximation; it is the claim that a molecule costs the same to make whether or not it is surrounded by solvent.

The fix is exact and cheap. A pure liquid is in equilibrium with its own vapour at its saturation pressure P^{sat}, so their molar Gibbs energies are equal:

$$\mu(\text{liquid}) = \mu(\text{gas at } P^{\mathrm{sat}}) = \mu(\text{gas at 1 bar}) + RT \ln \frac{P^{\mathrm{sat}}}{P^{\circ}}.$$

Hence per species

$$\Delta G_f(\ell) = \Delta G_f(g) + RT\ln\frac{P^{\mathrm{sat}}}{P^\circ}, \qquad \Delta H_f(\ell) = \Delta H_f(g) - \Delta H_{\mathrm{vap}}.$$

The Gibbs shift is always negative, since $P^{\mathrm{sat}} < 1$ bar below the boiling point. And ΔH_{vap} is taken from the *same* Antoine curve by Clausius–Clapeyron rather than from a second correlation, so both halves come from one source and the entropy you derive from them is real: water 44.1 against a measured 44.0, ethanol 42.7 against 42.3, acetone 31.6 against 31.0 kJ/mol.

For Fischer esterification the effect is a factor of 2.4 — $K(298\ \mathrm{K})$ moves from 19.4 to 8.1, against a measured value near 4 (Figure 3.1). What remains is group-contribution error in the formation data itself, which is Chapter 12's problem.

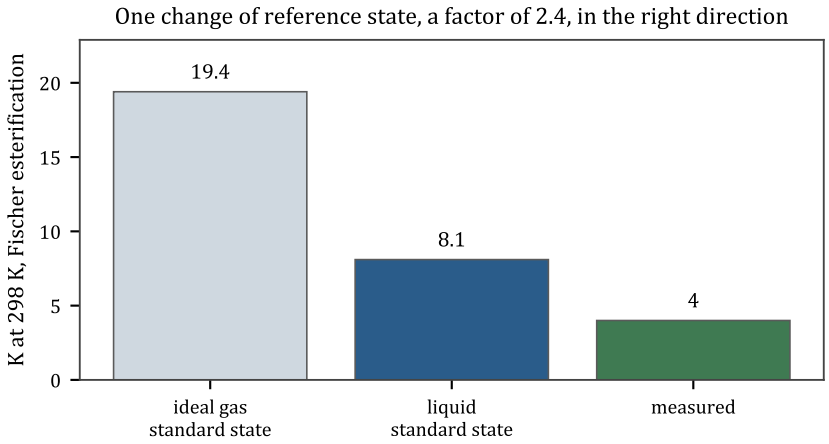

Figure 3.1: One change of reference state, in the right direction.

THE POINT

The same expression serves a dissolved gas, with its Henry constant in place of P^{sat} — one formula, two standard states. Species too involatile to trust (ions, and anything whose extrapolated P^{sat} falls below 10^{-12} bar) keep the basis their data was derived on, *and say so on the record*.

A STANDARD STATE CAN BE WRONG IN A COMMENT

In milestone S12 the project found that a source-code comment had priced its own reaction on the wrong standard state, flipping the sign of ΔS and landing 163 kJ/mol from the right answer. The code was right and the hand-computed numbers written beside it were wrong. This is exactly why the project prefers derived quantities to transcribed ones: a derivation can be re-run, a comment cannot.

3.7 Heat capacity, and what this project does not do

$C_p = (\partial H/\partial T)_p$. It matters twice: a vessel's temperature response is $\mathrm{d}T/\mathrm{d}t = \dot{q}/C_p$, and strictly ΔH itself depends on temperature via Kirchhoff's law, $\mathrm{d}(\Delta H)/\mathrm{d}T = \Delta C_p$.

This project uses temperature-dependent C_p for the *energy balance* (a cubic in T per species, Chapter 13) but assumes ΔH and ΔS of reaction are temperature-independent when computing $K(T)$ — that is, van 't Hoff with a constant enthalpy. The correction was built and rejected in milestone M6 as not worth its cost at the temperature spans involved; the decision is recorded rather than assumed.

CHAPTER 4

Equilibrium: how far, rather than whether

Chapter 3 gave a yes/no answer. Real reactions do not go to completion; they stop somewhere, with reactants and products both present, and the reverse reaction running exactly as fast as the forward one. Where they stop is the subject of this chapter.

4.1 The equilibrium constant

For a reaction $\sum_i \nu_i A_i = 0$ with ν negative for reactants, define the **reaction quotient**

$$Q = \prod_i a_i^{\nu_i}$$

where a_i is the *activity* of species i — for now, read it as concentration divided by a reference concentration, so that Q is dimensionless. Chapter 8 makes it more honest.

The Gibbs energy of the mixture, as a function of how far the reaction has run, is

$$\Delta G = \Delta G^\circ + RT \ln Q,$$

and equilibrium is where $\Delta G = 0$, i.e. where Q takes the particular value

$$\boxed{K = \exp\left(-\frac{\Delta G^\circ}{RT}\right).}$$

$K \gg 1$ means the reaction runs essentially to completion; $K \ll 1$ means it barely starts; $K \approx 1$ means you get a mixture. Because ΔG° is inside an exponential divided by $RT \approx 2.5$ kJ/mol at room temperature, an error of 6 kJ/mol in ΔG° is a factor of ten in K.

> **WHY FORMATION-DATA ACCURACY DOMINATES EVERYTHING**
>
> RT at 298 K is 2.48 kJ/mol. Group-contribution estimates of ΔG_f carry several kJ/mol of uncertainty *per species*, and a reaction sums four of them. So a 5 kJ/mol estimator error is a factor of $\sim$7 in K, which is the difference between a 90% yield and a 50% one. This is why Part II is as long as it is, and why the project's own list of limitations puts "group-contribution formation data is the dominant error in K" first.

4.2 Le Chatelier, as a derivative

Differentiate $\ln K = -\Delta G^\circ/RT = -\Delta H^\circ/RT + \Delta S^\circ/R$ with respect to temperature:

$$\frac{\mathrm{d}\ln K}{\mathrm{d}T} = \frac{\Delta H^\circ}{RT^2}, \qquad \text{or} \qquad \frac{\mathrm{d}\ln K}{\mathrm{d}(1/T)} = -\frac{\Delta H^\circ}{R}.$$

This is the **van 't Hoff equation**. Heating increases K for an endothermic reaction and decreases it for an exothermic one — which is the quantitative form of Le Chatelier's qualitative principle, and it is a straight line if you plot $\ln K$ against $1/T$ (Figure 4.1).

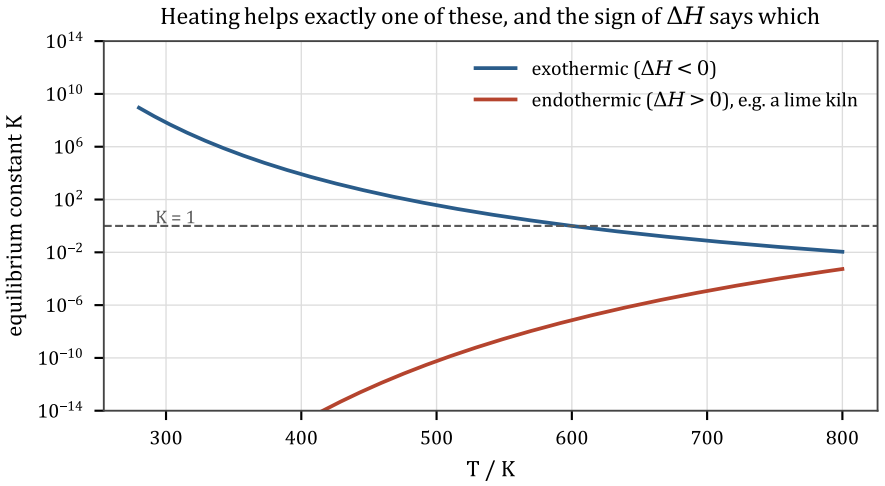

Figure 4.1: Heating helps exactly one of these.

> **THIS PRODUCES A MECHANIC NOBODY WROTE**
>
> In this simulator, an exothermic reaction in an insulated flask *heats itself*. The higher temperature raises K's denominator and, being exothermic, lowers K. So the flask gets less product than a well-cooled one would. Nobody coded that; it is van 't Hoff plus an energy balance, and it is the reason a real preparative chemist uses an ice bath.

4.3 The pressure and concentration versions

Chemists write K three ways and it is worth keeping them straight, because the project's `reactions/thermo.py` has to convert between them.

- K_a, on activities, is the thermodynamic one and is dimensionless.
- K_p, on partial pressures in bar, for gas reactions.
- K_c, on molarities, for solution reactions.

They differ by factors of RT raised to $\Delta n = \sum_i \nu_i$, the change in the number of moles. For $\Delta n = 0$ they coincide; otherwise they do not.

THE CONVERSION FACTOR IS NOT ARRHENIUS, AND HIDING IT IN THE PRE-EXPONENTIAL MADE K DRIFT

Converting activities to molarities carries a factor $T^{\Delta n}$. This project originally folded that into the reverse pre-exponential A at one reference temperature, which is exact at T_{ref} and wrong everywhere else — K drifted as $(T/T_{\mathrm{ref}})^{\Delta n}$. The fix was to give the rate law a temperature *exponent*, a modified Arrhenius form:

$$k = A\,T^{\,n}\exp(-E_a/RT), \qquad n_{\mathrm{rev}} = n_{\mathrm{fwd}} + \Delta n.$$

n is zero for every *declared* rate in the project — only detailed balance ever sets it — so the common case stays pure Arrhenius and the kernel skips the exponent entirely when no reaction needs one. That is a good example of how these fixes are shaped: make the general case expressible, keep the common case free.

There is a related trick that appears repeatedly: **write the reaction so that $\Delta n = 0$ and the conversion cancels**. Acid dissociation is written $\mathrm{HA} + \mathrm{H_2O} \rightleftharpoons \mathrm{A^-} + \mathrm{H_3O^+}$ rather than the more familiar $\mathrm{HA} \rightleftharpoons \mathrm{A^-} + \mathrm{H^+}$ for exactly this reason (Chapter 10).

4.4 What equilibrium is not

Equilibrium says where the system ends up if you wait long enough. It says nothing about *how long*, and it says nothing about which of several possible products you get on the way. Diamond is thermodynamically unstable relative to graphite at room temperature; the conversion takes longer than the age of the universe. That gap between “allowed” and “happens” is kinetics, and it is the next chapter.

THE POINT

Thermodynamics is a statement about the *destination*; kinetics is a statement about the *journey*. A simulator that models only the first can tell you what a flask contains after infinite time. This one models both, which is why it can tell you that a flask makes ester at 320 K and ether at 480 K — both are allowed at both temperatures, and only the rates differ.

CHAPTER 5

Rates: how fast, and which product

5.1 Mass action

The simplest useful model of a reaction rate says: a reaction between A and B happens when an A meets a B, so its rate is proportional to how often that happens, so it is proportional to the product of their concentrations.

$$\text{for A} + \text{B} \to \text{P}: \qquad r = k\,[\text{A}][\text{B}].$$

In general, for a reaction with reactant multiset $\{A_i\}$,

$$r_j = k_j \prod_i C_i^{\alpha_{ji}}$$

where α_{ji} is the **rate-law exponent**, or *order*, of species i in reaction j. The composition then evolves as

$$\frac{\mathrm{d}\mathbf{C}}{\mathrm{d}t} = \Delta^{\mathsf{T}}\mathbf{r}(T, \mathbf{C}).$$

That pair of equations is the entire content of `chemsim/numerics/integrator.py`. It is a system of coupled, non-linear, first-order ODEs, and it is what the whole architecture exists to feed.

5.2 Order is not stoichiometry, except when it is

For an **elementary** step — one that really is a single collision — the order equals the stoichiometric coefficient, because that is how many of each molecule has to be in the collision. This project assumes elementary steps by default, so the multiset of reactants supplies the exponents. `2 EtOH -> ...` records ethanol twice and gets exponent 2.

> **IT STOPS BEING TRUE THE MOMENT A TEMPLATE WRITES A GLOBAL STOICHIOMETRY**
>
> Sulfur burning is
>
> $$\text{S}_8 + 8\,\text{O}_2 \to 8\,\text{SO}_2$$

and it is *not* an elementary step — nine molecules do not meet. Taken as mass action it is ninth order, eighth in O_2, and the project measured what that costs: it needs $A = 7 \times 10^{24}$ $(\text{L/mol})^8/\text{s}$ to run at all; it is *forgiven* where O_2 is in excess, because the attractor does the work; and it is **not** forgiven where O_2 is limiting, because $[O_2]^8$ stalls asymptotically and the reported yield becomes a reading of the author's pre-exponential rather than of the chemistry.

So `ReactionTemplate` grew an `orders` field: one exponent per reactant slot. The burner declares $(1, 1, 0, \dots)$ — first order in each — and the eight oxygens still leave in the stoichiometry.

And a declared order may never be reversible. That invariant is enforced, and Chapter 21 shows it arriving as a module boundary between two different solid-phase rate laws.

There is one more case where order and stoichiometry part company, and it is the most important one in practice: a **catalyst** appears on both sides of the arrow, so its stoichiometric coefficient is zero, but its exponent is 1. It multiplies the rate and changes nothing else. See Chapter 18.

5.3 The rate constant, and why it is exponential in $1/T$

Empirically, over an enormous range of reactions,

$$\boxed{k(T) = A \exp\left(-\frac{E_a}{RT}\right)}$$

the **Arrhenius equation**. E_a is the *activation energy* — an energy barrier the reactants must climb before they can become products — and A, the *pre-exponential factor*, is roughly the rate at which the attempt is made.

The exponential is a Boltzmann factor, and it is the fraction of collisions energetic enough to clear the barrier (Figure 5.1). A rule of thumb worth memorising: at room temperature, $RT = 2.5$ kJ/mol, so **a 10 kJ/mol change in barrier is a factor of 55 in rate**, and a rate typically doubles for every 10 K.

IF YOU KNOW PHYSICS

Transition state theory makes this sharper. Treat the barrier top as a species in equilibrium with the reactants, and you get the Eyring equation

$$k = \frac{k_B T}{h} \exp\left(-\frac{\Delta G^{\ddagger}}{RT}\right) = \frac{k_B T}{h}\, e^{\Delta S^{\ddagger}/R}\, e^{-\Delta H^{\ddagger}/RT},$$

which identifies A with $(k_B T/h)\, e^{\Delta S^{\ddagger}/R}$ — an attempt frequency ($k_B T/h \approx 6 \times 10^{12}$ s^{-1} at 300 K) times an entropic factor for how ordered the transition state has to be. This is where the project's "10^{11}–10^{14} s^{-1} for a unimolecular thermal step" band comes from: it is not folklore, it is $k_B T/h$ with an entropy correction of a couple of orders either way.

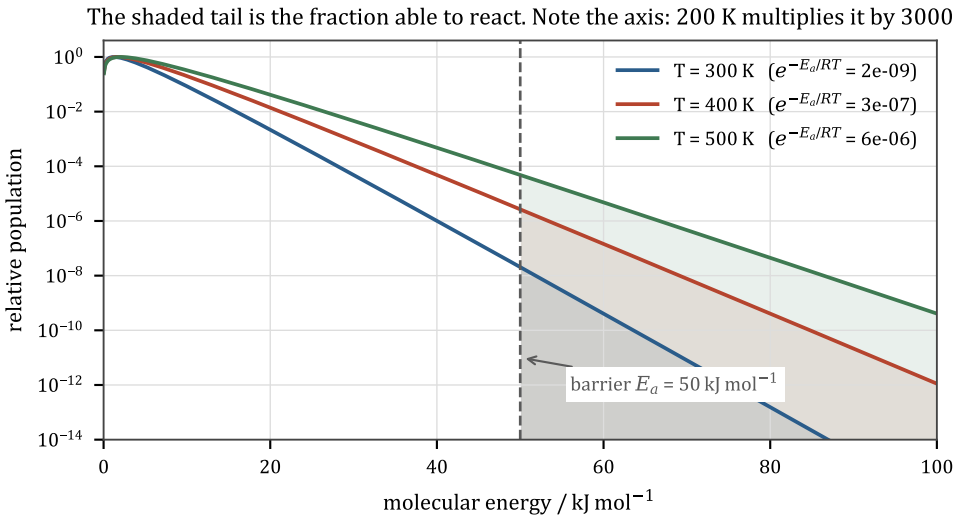

Figure 5.1: The exponential is counting the shaded tail.

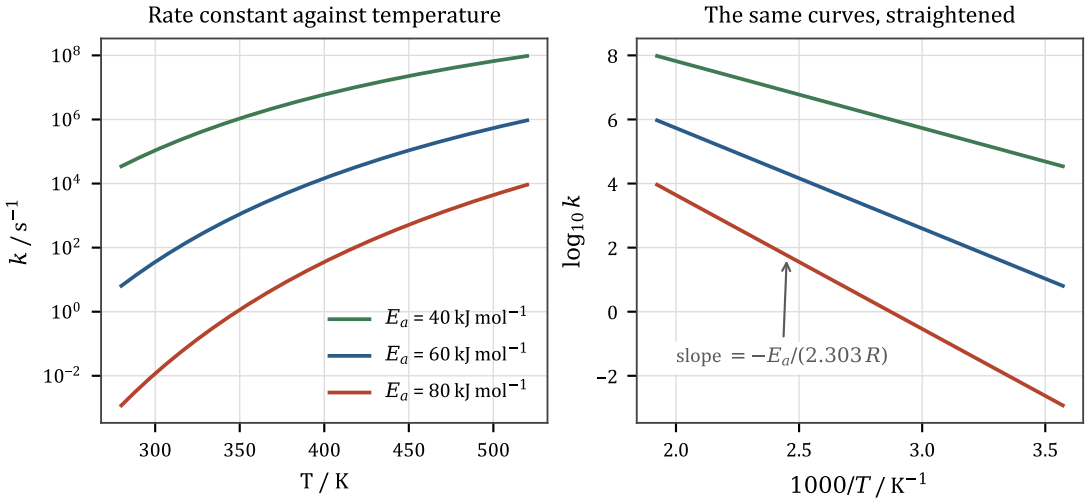

Figure 5.2: Three barriers, and the log plot that measures them.

5.4 The pre-exponential is this project's honest weak point

The project is explicit about this and it deserves repeating here, because it bounds what any number out of the simulator means.

- **Barriers (E_a) are sourced.** Each template in `reactions/library.py` and `reactions/synthesis.py` carries the literature band its barrier came from, and *the ordering between templates is the load-bearing part*: ethanol over sulfuric acid gives diethyl ether at 140 °C and ethylene at 180 °C because the elimination has the higher barrier. Get that ordering wrong and the temperature response is backwards, however good the individual numbers look.
- **Pre-exponentials (A) are order-of-magnitude choices** inside the range physically sensible for the molecularity. They set the absolute timescale.

THE POINT

So: read a simulated *branching ratio* or *temperature response* as a prediction. Do **not** read a simulated reaction *time* as one. The project says this in its own library docstring and it is the correct calibration for anything you get out of it.

5.5 A ceiling nobody can exceed

Because A is hand-authored, it can be wrong, and a rate constant that exceeds the physical collision limit is a detectable error rather than a matter of taste. Two molecules in solution cannot meet faster than diffusion allows ($\sim 10^{10}$ L mol^{-1} s^{-1}); a bond cannot vibrate faster than $\sim 10^{13}$ s^{-1}.

AN ENERGY LEAK TURNED OUT TO BE A RATE CONSTANT 94 MILLION TIMES THE COLLISION LIMIT

Milestone M12 chased an insulated flask that destroyed 495 J of energy after a precipitation event, and the project's conservation report could not see it because the report checks matter. Four candidate causes were measured and refuted. The actual cause was a **derived** rate constant — one that came out of detailed balance, not out of anybody's hand — sitting at 9.4×10^7 times the collision limit. At that speed the solver cannot resolve the transient, and the energy balance leaks.

The fix is a cap, and the interesting part is its shape: it must scale **both** pre-exponentials, forward and reverse, by the same factor. Scaling one would change K, which would be a thermodynamic falsehood introduced to fix a numerical problem. `validation/rate_ceiling.py` now audits every network in the project, and it audits the *reverse* a reversible template implies, not only the forward somebody typed.

5.6 Competing reactions: the actual point

Nothing above is interesting for a single reaction. It becomes interesting when several reactions compete for the same starting material, because then the *ratio* of their rates decides what you get, and that ratio is

$$\frac{r_1}{r_2} = \frac{A_1}{A_2}\exp\!\left(-\frac{E_{a,1} - E_{a,2}}{RT}\right).$$

Two consequences:

1. **The branching depends on temperature**, and it does so exponentially in the *difference* of barriers. A 20 kJ/mol gap that gives 3000:1 selectivity at 300 K gives only 20:1 at 700 K.
2. **A reaction with a higher barrier can still win if it has a higher prefactor**, at high enough temperature. The crossover is a real, computable temperature (Figure 5.3), and in chemistry this is the distinction between *kinetic* and *thermodynamic* control.

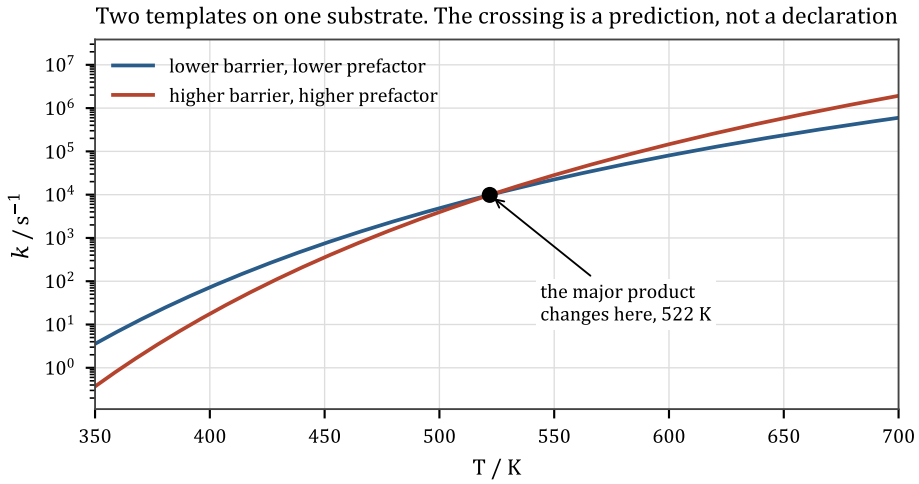

Figure 5.3: Two templates racing. The crossing is a prediction.

The simulator demonstrates this on the same flask with nothing declared. One charge of ethanol and acetic acid, `alcohol_chemistry()` loaded, one hour:

T / K	ethyl acetate	diethyl ether	ethene
320	1.476	0.000	0.000
400	1.408	0.023	0.00003
480	0.421	0.751	0.017

THE POINT

This table is the project's founding claim in miniature. There is no rule anywhere that says what happens at 480 K. There are three templates with three barriers, and an integrator.

Nobody wrote "if hot, make ether". The barriers differ, so the branching does

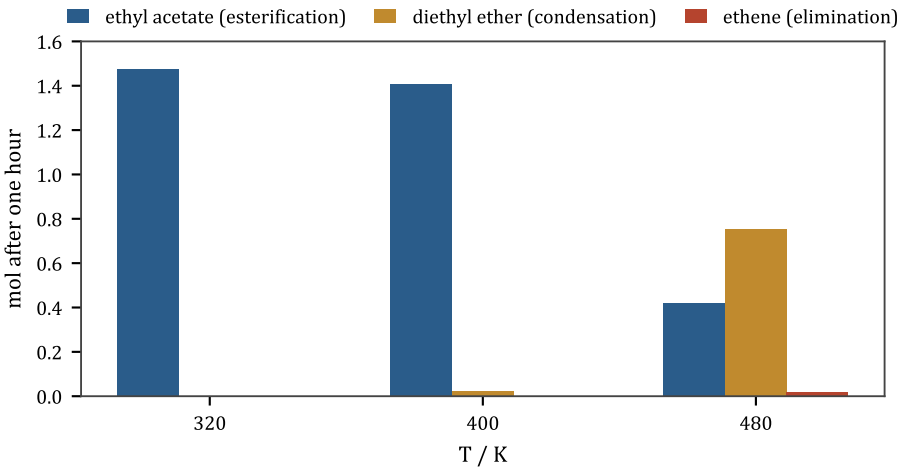

Figure 5.4: Nobody wrote “if hot, make ether”.

CHAPTER 6

Detailed balance: the hinge of the whole design

Chapters 4 and 5 look independent. They are not, and the constraint that links them is the single most important structural idea in this project.

6.1 The constraint

A reversible reaction runs both ways. At equilibrium the two rates are equal — not just the net flux zero, but *this particular forward process* balanced by *this particular reverse process*. That is **detailed balance**, and for a reaction $\mathrm{A} + \mathrm{B} \rightleftharpoons \mathrm{C} + \mathrm{D}$ it says

$$k_f[\mathrm{A}][\mathrm{B}] = k_r[\mathrm{C}][\mathrm{D}] \quad \Longrightarrow \quad \frac{k_f}{k_r} = \left.\frac{[\mathrm{C}][\mathrm{D}]}{[\mathrm{A}][\mathrm{B}]}\right|_{\mathrm{eq}} = K.$$

THE POINT

$$\frac{k_f(T)}{k_r(T)} = K(T) \quad \text{at every temperature.}$$

The forward rate, the reverse rate, and the equilibrium constant are three quantities with two degrees of freedom. You may choose any two; the third is determined.

IF YOU KNOW PHYSICS

This is microscopic reversibility: the underlying equations of motion are time-reversal symmetric, so the transition rate from state i to state j and back are related by the ratio of the states' Boltzmann weights. Detailed balance is that statement coarse-grained to whole species. It is the same argument that gives you the fluctuation–dissipation theorem, and it is equally non-negotiable: a model that violates it has a perpetual motion machine in it.

6.2 What that buys, structurally

Substitute Arrhenius on both sides:

$$\frac{A_f e^{-E_{a,f}/RT}}{A_r e^{-E_{a,r}/RT}} = \exp\!\left(\frac{-\Delta H + T\Delta S}{RT}\right)$$

and match the temperature-dependent and temperature-independent parts separately. The result is two lines of arithmetic:

$$\boxed{A_{\text{rev}} = A_{\text{fwd}}\, e^{-\Delta S/R}, \qquad E_{a,\text{rev}} = E_{a,\text{fwd}} - \Delta H.}$$

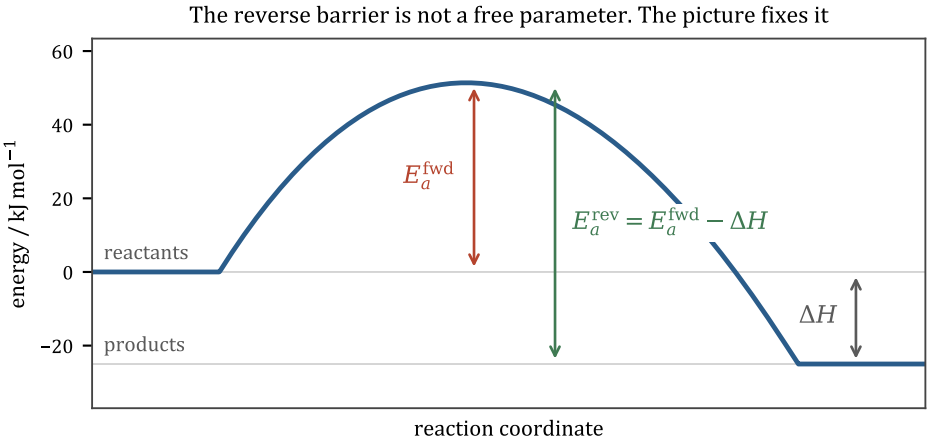

Figure 6.1: The reverse barrier is not a free parameter — the picture fixes it.

Figure 6.1 is the same statement drawn. The barrier from the product side is the barrier from the reactant side minus the drop between them. There is nothing to choose.

6.3 Why this is the design's hinge

Three consequences, and each one shapes a layer of the codebase.

1. A template declares forward kinetics only. There is no hand-typed reverse rate anywhere in this project, by design. The `reversible=True` flag on a `ReactionTemplate` means "generate the reverse from thermochemistry", and it therefore requires a thermochemistry provider to be present at network-build time. A hand-typed reverse would be a free parameter that silently contradicts the thermodynamics — you would have a model whose equilibrium constant disagreed with its own formation data, and nothing would tell you.

2. The equilibrium is *derived*, never encoded. The project's esterification equilibrium is not a number anybody typed. It is ΔG_f of four species, put through $K = e^{-\Delta G^\circ/RT}$, put through detailed balance, put through an integrator. A closed reactor started from pure reactants and one started from pure products land on the same quotient, and that is asserted in the test suite.

3. Layer 4 never learns what "reversible" means. This is the elegant part. Because the derived reverse is itself of Arrhenius form, it enters the network as *an ordinary reaction* — another row in Δ, another entry in A and E_a. The numeric core stays a pure mass-action integrator with no concept of reversibility, no pairing of reactions, and no equilibrium logic. All the thermodynamics happens once, at setup.

THIS IS THE PROJECT'S CHARACTERISTIC MOVE, AND YOU WILL SEE IT EIGHT MORE TIMES

Do the model-specific reasoning **once, at assembly time**, and hand the hot loop a uniform numeric array. Raoult's law and Henry's law become one array of Antoine coefficients. Lee–Kesler, Rackett and Rowlinson–Bondi become polynomials in T. A forward and a reverse reaction become two rows of the same matrix. The kernel evaluates one functional form and has never heard of any of the models that produced it.

That discipline is what makes the `numerics` boundary a clean seam for a future Rust kernel, and it is what keeps cheminformatics out of the inner loop entirely.

6.4 The floor, and the notice it prints

$E_{a,\mathrm{rev}} = E_{a,\mathrm{fwd}} - \Delta H$ can come out negative if a declared forward barrier is smaller than the reaction's endothermicity. A negative barrier is unphysical — it would mean a rate that *decreases* with temperature. The project raises it to zero and **prints a notice**:

> A declared barrier below the reaction's endothermicity is raised to the thermodynamic floor ($E_{a,\mathrm{rev}} \geq 0$) with a printed notice.

That is a small thing done in the project's characteristic style: the guard does not silently repair the input, it says which input it repaired, because the real problem is a bad declared barrier and the notice is the only thing that will lead you to it.

EA = MAX(DH, 0) IS ZERO ON AN EXOTHERMIC ROW

A related derivation went wrong in milestone S9. For a *decomposition*, a reasonable default barrier is the reaction's own endothermicity: you cannot break the bond for less than the bond is worth, so $E_a = \max(\Delta H, 0)$. That reasoning is sound *for a decomposition* and was applied to a table that had grown to include exothermic rows — where it silently returns a barrier of **zero**, i.e. a reaction with no temperature dependence at all that runs at its pre-exponential from absolute zero upward. The lesson recorded is that a default derived from one mechanism must be re-justified when the table grows a second.

6.5 Catalysis and detailed balance

A catalyst speeds a reaction up. It must therefore speed *both directions* up, by exactly the same factor — otherwise adding a catalyst would shift the equilibrium, which is a perpetual motion machine again.

This project enforces that structurally rather than by a check: a catalyst's rate-law exponent is applied to the forward reaction and to the derived reverse identically, so it cancels out of k_f / k_r exactly. A flask with no catalyst reaches the *same* equilibrium, infinitely slowly.

THE POINT

> That is why the esterification template writes its acid catalyst on **both** sides of the reaction SMARTS. It is not stylistic. Catalysing one direction only would make adding acid shift the equilibrium, and the failure would look like chemistry.

CHAPTER 7

Phases: boiling, and why there is no boiling point in the code

7.1 Vapour pressure

Put a liquid in a sealed flask with some empty space above it. Molecules leave the surface and rejoin it; at equilibrium the two rates match, and the gas above sits at a definite pressure that depends only on temperature and on what the liquid is. That is the **saturation vapour pressure** $P^{\mathrm{sat}}(T)$.

It rises steeply with temperature, because escaping costs an energy ΔH_{vap} and the fraction of molecules with that much energy is a Boltzmann factor. Equating the chemical potentials of the two phases and differentiating gives the **Clausius–Clapeyron** relation,

$$\frac{\mathrm{d}\ln P^{\mathrm{sat}}}{\mathrm{d}T} = \frac{\Delta H_{\mathrm{vap}}}{RT^2},$$

which integrates (with ΔH_{vap} taken constant) to $\ln P^{\mathrm{sat}} = -\Delta H_{\mathrm{vap}}/RT + \text{const}$. In practice a three-parameter empirical fit does better over a wide range, and the standard one is **Antoine's equation**:

$$\log_{10}\!\left(\frac{P^{\mathrm{sat}}}{\mathrm{bar}}\right) = A - \frac{B}{C + T}.$$

THE POINT

Every volatile species in this project carries exactly three numbers, (A, B, C), and Layer 5 asks them exactly one question: *given a liquid mole fraction x_i, what partial pressure is that in equilibrium with?*

7.2 Raoult's law, and the same array serving Henry's law

For an ideal liquid mixture, each component contributes its vapour pressure in proportion to how much of the liquid it is:

$$p_i = x_i\, P_i^{\mathrm{sat}}(T). \qquad \textbf{(Raoult)}$$

Engine output. A boiling point is where a curve meets the dashed line — it is not stored anywhere

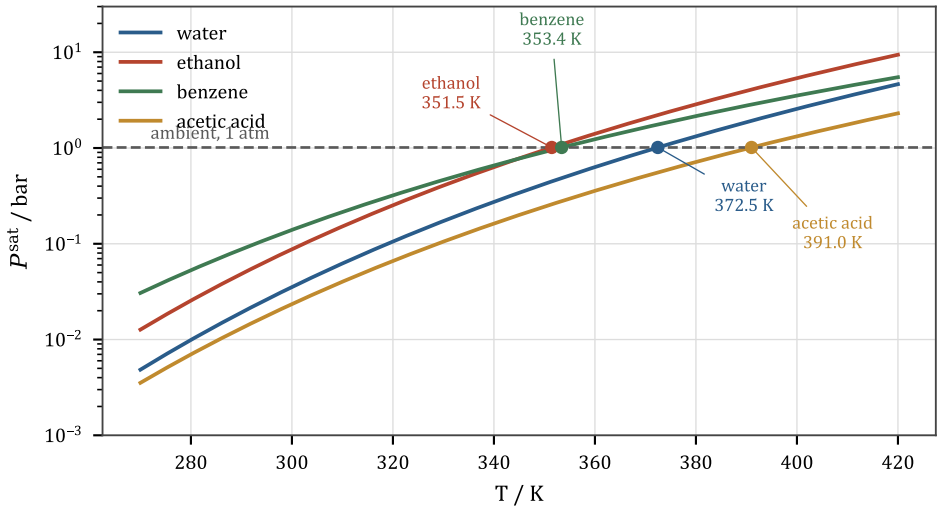

Figure 7.1: Vapour-pressure curves, computed by the engine's own provider.

For a **permanent gas** — O_2 or N_2, which are above their critical temperature and have no vapour pressure at all — the same *shape* of law holds, but the constant means something different: it is a solubility constant measured in a particular solvent.

$$p_i = x_i \, H_i(T). \qquad \textbf{(Henry)}$$

Henry constants follow van 't Hoff, $H(T) = H_{\text{ref}} \exp[-C(1/T - 1/T_{\text{ref}})]$, which is *already Antoine-shaped with* $C = 0$ — no fitting required, just algebra.

TWO PHYSICAL LAWS, ONE ARRAY

So `properties/volatility.py` emits one three-number record per species and the integrator evaluates $10^{A-B/(C+T)}$ for all of them at once. Raoult's law and Henry's law are the same line of code, and the hot loop cannot tell which species is which.

This is the setup/hot-loop split again, and it is the cleanest instance of it in the project.

The provider has three sources in preference order, each stamped on the result: curated Antoine constants (NIST) for the solvents that matter; curated Henry constants for permanent gases; and Lee–Kesler estimation from $T_b/T_c/P_c$, fitted to Antoine form, so a molecule nobody has ever tabulated still gets a curve. Chapter 13 is about that third route.

7.3 Boiling is a condition, not a number

Here is the payoff, and it is characteristic of the whole project.

A liquid boils when its vapour pressure reaches the ambient pressure — at that point a bubble can form in the bulk, because the vapour inside it can push the surroundings back. Ethanol's

normal boiling point is 351.4 K because that is where its Antoine curve crosses 1.013 bar (Figure 7.1).

There is no boiling point stored anywhere in this codebase. What exists is:

- an evaporation flux driven by the difference between equilibrium and actual partial pressure, $\dot{n}_i \propto k_{la}(x_i \gamma_i P_i^{\text{sat}} - p_i)$;
- an energy balance in which that flux carries latent heat away;
- a vent that opens when the total pressure exceeds ambient.

Run those together and a flask on a hotplate does this:

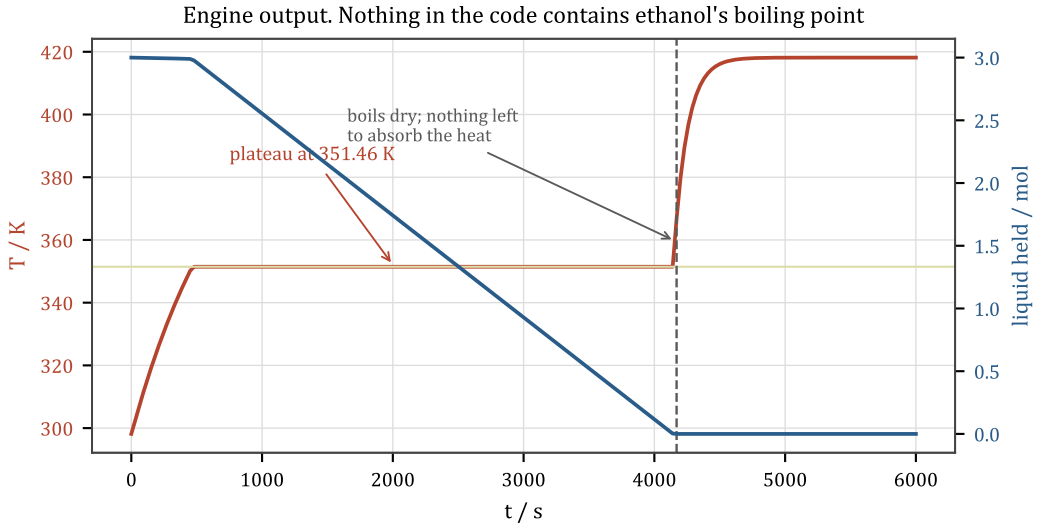

Figure 7.2: Engine output. Plateau, dryout, superheat.

The temperature climbs, then **pins at 351.46 K** — against a measured 351.4 — and holds there for as long as there is liquid. It holds because once $\sum_i p_i$ reaches ambient, any further heat goes into evaporation instead of into temperature: the evaporation term runs away, and latent heat absorbs the entire hotplate input. Then the liquid runs out, there is nothing left to absorb the heat, and the temperature rockets.

> **THE POINT**
>
> The boil-off rate is $(Q_{\text{in}} - \text{losses})/\Delta H_{\text{vap}}$ and that too is not written down anywhere. It falls out of the energy balance, and the test suite asserts it.

7.4 Distillation

If two components have different volatilities, the vapour is richer in the more volatile one. That is separation, and it is the oldest unit operation there is.

For an ideal binary mixture the vapour composition is

$$y_1 = \frac{x_1 P_1^{\text{sat}}}{x_1 P_1^{\text{sat}} + x_2 P_2^{\text{sat}}}.$$

A 50/50 ethanol–water liquid gives a vapour that is 71% ethanol — and the project reproduces that from Raoult alone, with no separation model whatsoever. Condense that vapour, boil it again, and you enrich further. Do it many times and you have **fractional distillation**; the project builds an eight-plate column that reaches 85.4% purity at a reflux ratio of 5 (Chapter 22).

A STILL WITH NO OPEN END IS A PRESSURE VESSEL

The project's first attempt at a plate column diagnosed its poor separation as a startup transient. It was not. The still had **no open end**, so it ran at 3–3.8 bar, at which the whole vapour–liquid equilibrium is different. The diagnosis was wrong and the fix was a vent. The recorded lesson is that when a separation underperforms, check the pressure before you check the model.

7.5 What is still missing from this picture

Everything above assumes the liquid is **ideal** — that a molecule cannot tell what it is surrounded by. Under that assumption distillation always runs to a pure product, because the vapour is always richer in the more volatile component. Reality disagrees, sometimes spectacularly, and fixing it is Chapter 8.

CHAPTER 8

Activity: when the liquid stops being ideal

8.1 What Raoult quietly asserted

$p_i = x_i P_i^{\text{sat}}$ says a molecule's tendency to escape the liquid depends only on how much of the liquid is that molecule — not on what the rest of it is. That is the assertion that **a molecule cannot tell what it is surrounded by**, and it is false in an obvious way: ethanol molecules hydrogen-bond to water, and to each other, and not equally.

Two things followed from that assumption in this project, wrong in the same direction:

- distillation always ran to a pure product;
- a solid dissolved as readily in a solvent it hates as in one it loves.

One model fixes both, because both were the same missing term.

8.2 Activity and the activity coefficient

Define the **activity** a_i as the thing that actually appears in every equilibrium expression, and the **activity coefficient** γ_i as the correction factor:

$$a_i = \gamma_i x_i, \qquad \gamma_i \to 1 \text{ as } x_i \to 1.$$

Then Raoult becomes $p_i = \gamma_i x_i P_i^{\text{sat}}$ and everything else carries through unchanged. $\gamma_i > 1$ means "this molecule is less comfortable here than in its own pure liquid, so it escapes more readily than its mole fraction says"; $\gamma_i < 1$ means the opposite.

> **IF YOU KNOW PHYSICS**
>
> γ is an excess free energy in disguise. Write $G^{\text{E}} = RT \sum_i n_i \ln \gamma_i$: the difference between the real mixture's free energy and an ideal one at the same composition. Then $RT \ln \gamma_i = \partial G^{\text{E}} / \partial n_i$ — it is a partial molar excess quantity, i.e. an interaction term in a mean-field free energy. Every activity-coefficient model is a model of G^{E}, and UNIFAC below is a particular functional form for it with the parameters fitted to group pairs rather than to species pairs.

8.3 UNIFAC

The model this project uses is **UNIFAC** (Fredenslund, 1975), and its organising idea is the same one that Chapter 12 will use for thermochemistry: a molecule is a bag of functional groups, and

the interactions are between *groups* rather than between *molecules*. That is the only way to get coverage — there are $O(N^2)$ molecule pairs and only $O(G^2)$ group pairs, and G is about 50.

$$\ln \gamma_i = \underbrace{\ln \gamma_i^{\mathrm{C}}}_{\text{combinatorial}} + \underbrace{\ln \gamma_i^{\mathrm{R}}}_{\text{residual}} .$$

The **combinatorial** part is entropic and depends only on molecular size and surface area (parameters R_k and Q_k per group): big molecules and small ones do not mix ideally even if they like each other perfectly. The **residual** part is enthalpic and comes from a matrix of group–group interaction parameters a_{mn}.

WHY THIS ONE MODEL WOULD NOT COLLAPSE INTO A POLYNOMIAL

Every other property in this project is a function of temperature alone, so it is evaluated once at setup and handed to the kernel as polynomial coefficients. An activity coefficient depends on **composition**, and composition *is the state vector*. Fitting it in advance would mean fitting a function of the solution.

So the setup/hot-loop split *moves* rather than vanishing. What is precomputed is the *parameter block* — the group-count matrix ν, the R_k/Q_k vectors, and the interaction matrix already expanded from main groups to a dense subgroup basis. What runs per RHS call is the evaluation. The contract is unchanged: numpy in, numpy out, no domain types. The arrays are simply bigger, and the loop finally does real work.

There is one implementation detail that is not cosmetic. The combinatorial part is written using $J_i = \Phi_i/x_i$ and $L_i = \theta_i/x_i$ rather than Φ_i and θ_i themselves. Algebraically identical — but the x_i cancels *analytically* instead of numerically, so a species at exactly zero concentration has a well-defined activity coefficient rather than a $0/0$. In a reaction network, most species are at zero for part of the run.

8.4 What it buys: azeotropes, from nothing

If γ can bend the vapour-liquid equilibrium line, it can bend it across the diagonal — and where $y = x$, distillation stops working. That composition is an **azeotrope**: a mixture that boils without changing composition.

Figure 8.1 is engine output. The dotted lines are Raoult's law applied to the *same* Antoine curves, so the only difference between dotted and solid is γ. Note two things: the real bubble-point curve dips *below both pure boiling points*, and the y-x curve crosses the diagonal.

result	chemsim	reference
ethanol/water azeotrope	$x = 0.899$	0.894 (95.6 wt%)
its boiling temperature	351.17 K	351.3 K
below both pure components?	yes (351.45 / 372.45 K)	minimum-boiling, as observed

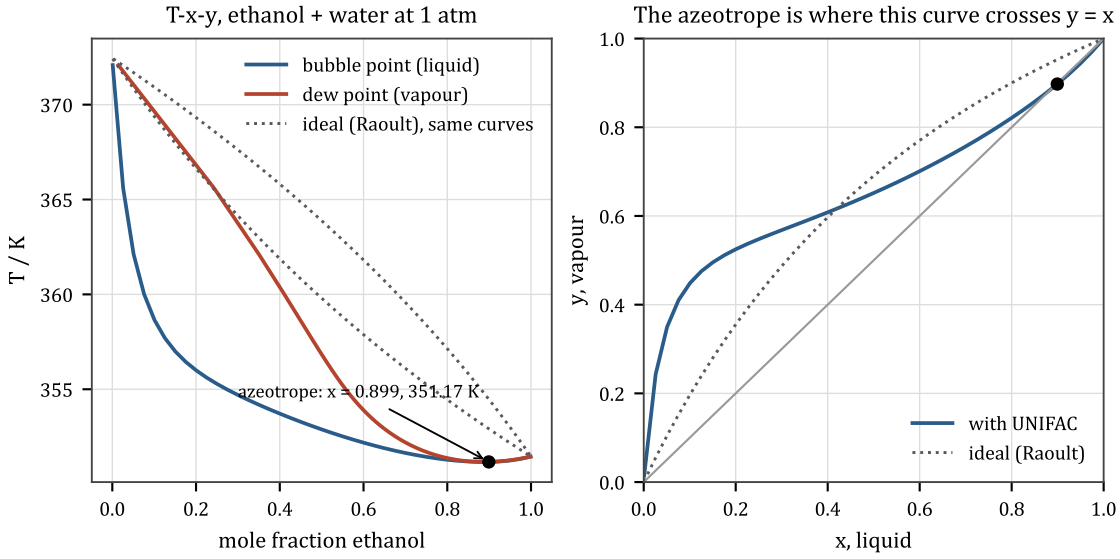

Figure 8.1: Ethanol and water, computed by the engine.

THE POINT

There is no azeotrope table in this project. The azeotrope is simply the composition where $y = x$, and it exists because γ bends the equilibrium line across the diagonal. Distillation of ethanol stalls at 95% here for the reason it stalls at 95% in a real column.

8.5 Two reference states, one expression

A condensable species uses the **symmetric** convention: $\gamma \to 1$ as the liquid becomes pure in it. A dissolved gas cannot — it has no pure liquid at these temperatures — so it uses the **unsymmetric** convention, referenced to infinite dilution in the solvent its Henry constant was measured in:

$$\gamma_i^* = \frac{\gamma_i(x)}{\gamma_i^{\infty}(\text{reference solvent})}.$$

That division is what transfers a measured constant to a *different* solvent, because the solute's hypothetical pure-liquid fugacity cancels out of the ratio: $H(S)/H(\text{ref}) = \gamma^{\infty}(S)/\gamma^{\infty}(\text{ref})$. In water the correction is 1 by construction and the calibrated number comes back untouched; anywhere else it is computed.

Every one of those solvents used to return water's 0.27 mM.

ASIDE

Standard UNIFAC has no group for a permanent gas, so the group table carries PSRK's gas extension — added as main groups UNIFAC *does not have*, so no existing parameter is overwritten and every previously validated result is unchanged to the last digit. The join is defensible be-

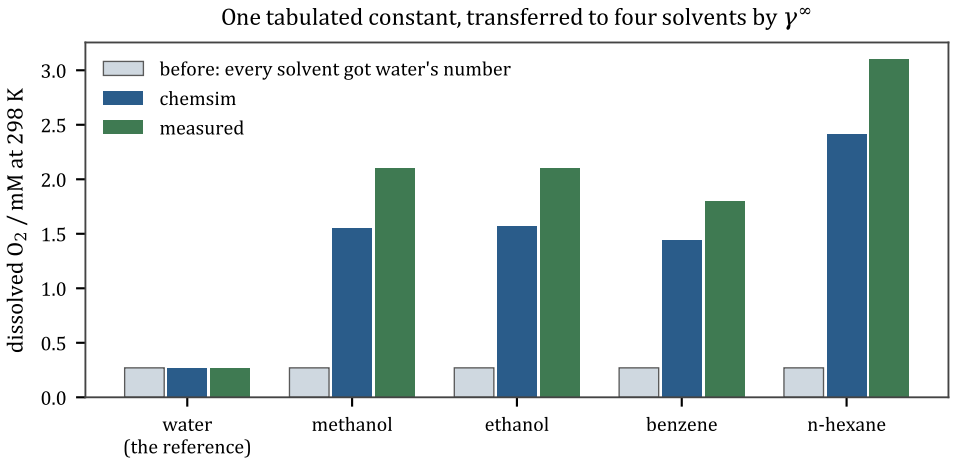

Figure 8.2: One tabulated constant, transferred to four solvents.

cause PSRK's organic backbone *is* UNIFAC's: 1124 of UNIFAC's 1174 pairs are bit-identical in PSRK. It is still a join of two regressions rather than one self-consistent fit, and the project says so in its limitations.

The divisor $\gamma^\infty(T)$ depends only on temperature, so it collapses to four numbers at setup like everything else — and it is fitted in $1/T$ rather than T, because it is a ratio of Boltzmann factors and van 't Hoff is the right basis. That choice is worth an order of magnitude: 0.15% error for N_2 against 2.5%.

8.6 Two liquid layers

If γ gets large enough, the free energy of mixing becomes non-convex, and the system lowers its energy by splitting into two liquids of different composition — oil and water. The equilibrium *condition* is easy and has the same form as every other phase equilibrium: equal activity on both sides,

$$\gamma_i(x^{\mathrm{I}})\, x_i^{\mathrm{I}} = \gamma_i(x^{\mathrm{II}})\, x_i^{\mathrm{II}}.$$

BUT THAT CONDITION CANNOT LIVE IN THE RIGHT-HAND SIDE

The condition above is *also satisfied by the two phases being identical*. That trivial solution always exists, it is where a relaxation started from a well-mixed flask sits, and no amount of integrating will leave it: **a single phase is a fixed point of its own splitting dynamics.**

Deciding whether that fixed point is a stable minimum or a saddle is a *global* question about the Gibbs surface, and answering it takes an iteration. So the work is split the way this project splits everything discrete: the smooth relaxation lives in the RHS, and the *decision* lives at an event boundary, between integrations. That is `numerics/lle.py`, and it is the only module in Layer 4 that is never called from inside a solve.

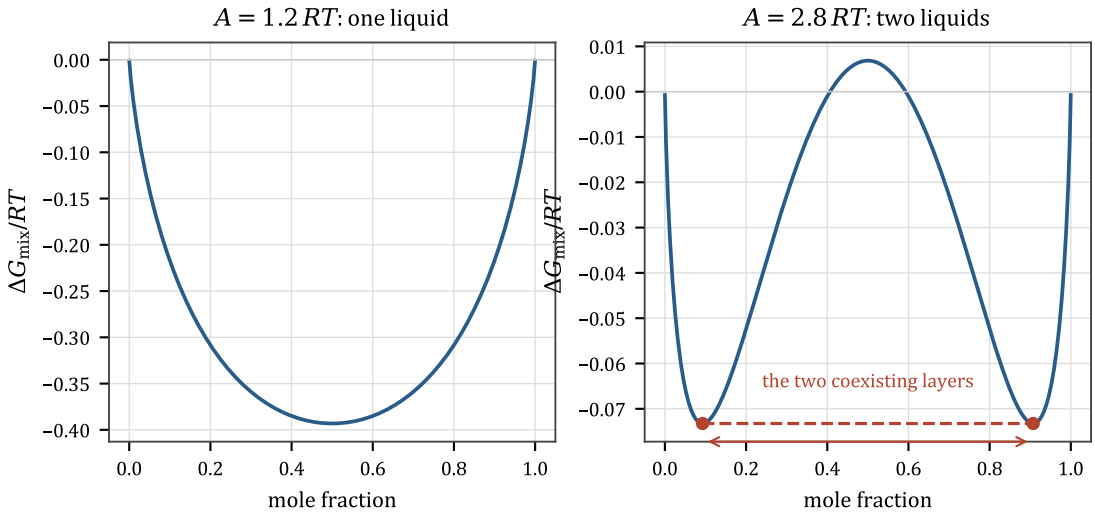

Figure 8.3: A miscibility gap is a non-convex free energy.

The test it uses is Michelsen's **tangent-plane criterion**: a phase of composition z is stable iff the Gibbs surface lies above its own tangent plane at z for every trial composition w, i.e. iff

$$\mathrm{tpd}(w) = \sum_i w_i \left[\ln w_i + \ln \gamma_i(w) - \ln z_i - \ln \gamma_i(z)\right] \geq 0$$

everywhere. Working with unnormalised W_i turns the stationary points of that surface into fixed points of a plain successive substitution, and because tpd can have several minima the iteration is run from several starting points and the deepest wins.

It does **not** flash. It reports "this liquid is unstable, and here is a composition that is better", and the caller seeds a second phase with a little material; the ordinary activity-equality term in the RHS then does the rest. That is deliberate — the discrete decision and the continuous relaxation stay separate.

8.7 What is stated rather than assumed

Three things can go wrong, and each is *named* rather than silently treated as ideal:

- a species has **no UNIFAC decomposition** (dissolved gases, ions, anything with an element the table does not cover). It gets $\gamma = 1$ and is listed in `vessel.activity_model.report()`;
- a species is an **ion**. UNIFAC is a non-electrolyte model; ions get $\gamma = 1$ *by policy*, not by accident — and see Chapter 10, where a Born term supplies what UNIFAC cannot;
- a main-group **pair** is absent from the published matrix. Roughly half are. A missing pair is not zero: zero is the strong claim that two groups mix athermally.

SILENCE ABOUT GAMMA = 1 WAS AN ARGUMENT, NOT A NEUTRAL DEFAULT

This is milestone M4 and it is the sharpest instrumentation finding in the project. A species with no decomposition got $\gamma = 1$ and nothing said so. In a *single* liquid that is a bounded error. In a **two-phase** calculation it is not an approximation at all — $\gamma = 1$ on both sides of an interface means every species partitions to equal mole fraction, i.e. **an ideal liquid never splits.** So the silent default was quietly asserting the answer to the question being asked.

Fixing the flag came before fixing the matcher, deliberately, because until the model says what it does not know, improving what it does know is unmeasurable. UNIFAC coverage over the corpus is now 54.1% and reported.

8.8 Cost

The RHS goes from about 140 µs to 231 µs per call for a four-species vessel — 1.7×. Notably the γ kernel is *flat* from 4 species to 25 (78 µs to 87 µs): at these sizes both numbers are numpy dispatch overhead on small arrays, not arithmetic.

That is a useful negative result. It means UNIFAC does **not** by itself justify dropping to a Rust kernel; the case for that rests on fixed per-call overhead, which was already there and which Rust would collapse for the whole RHS rather than for this part of it.

CHAPTER 9

Solids: melting, dissolving, and the law that is wrong for salt

9.1 One equation for two phenomena

Consider a solid in contact with a liquid. Equilibrium requires the solid's chemical potential to equal that of the same substance dissolved in the liquid. Working that out with an ideal solution and constant enthalpy of fusion gives

$$\boxed{\ln a_{\text{sat}} = -\frac{\Delta H_{\text{fus}}}{R}\left(\frac{1}{T} - \frac{1}{T_m}\right)}$$

where ΔH_{fus} is the enthalpy of fusion (the heat needed to melt it) and T_m the melting point.

Look at what happens at $T = T_m$: the right-hand side is zero, so $a_{\text{sat}} = 1$ — the solid becomes fully miscible with its own melt.

> **SO THERE IS NO SEPARATE MELTING MODEL**
>
> Melting and dissolving are the same equation evaluated at different temperatures. That is why melting shows a latent-heat plateau in this simulator for exactly the reason boiling does (Chapter 7), and why nothing in the code contains a melting point either. ΔH_{fus} and T_m come from group contributions or a curated table.

9.2 Melting and dissolving had to be separated after all

The equation above is written in **activity**, not mole fraction, and that distinction was a bug for a while. Written in x it was right for melting and badly wrong for dissolution. Written in a:

- **dissolution** divides by the activity coefficient, $x_{\text{sat}} = a_{\text{sat}}/\gamma$, so a solute that a solvent dislikes ($\gamma \gg 1$) dissolves far less;
- **melting** does not divide, because a pure solid in equilibrium with its own melt must not care how badly some solvent dissolves it.

The difference is enormous. Benzoic acid in water at 298 K:

	value
ideal law, $\gamma = 1$	1347 g/L

	value
with UNIFAC γ	3.24 g/L
measured	3.44 g/L

Both curves in Figure 9.1 are engine output and differ by exactly one factor.

AN HONEST LIMITATION, STATED WHERE IT WAS MEASURED

UNIFAC understates how fast an associating solute's solubility climbs with temperature. Benzoic acid in water comes out at 0.95× the measured value at 298 K but 0.48× at 333 K — the absolute scale is right and the slope is not. You can see it in Figure 9.1: the blue curve passes through the low-temperature squares and falls under the high-temperature ones. The project reports this rather than tuning it away.

9.3 Crystallisation is a rate, not an event

A solute above its solubility limit crystallises out. In the RHS that is a transport term next to evaporation: a driving force $(x_{\text{sat}} - x)$ times a rate constant, with the sign deciding whether crystals grow or dissolve. Growth from a supersaturated solution and dissolution of an existing crop are the *same term* running in two directions.

Cooling a solution therefore crops it, with no declaration anywhere:

T / K	benzoic acid dissolved	as solid
330.0	0.050000	0.000000
298.1	0.026681	0.023319
275.0	0.012236	0.037764

THERE IS NO NUCLEATION BARRIER, AND THAT IS A REAL GAP

Real solutions can be *supersaturated*: they hold more than the equilibrium amount because starting a crystal is harder than growing one. Real chemists exploit this constantly — you cool slowly to get big pure crystals, you seed, you scratch the glass.

Here, precipitation is ungated by design: "anything can nucleate", so a supersaturated solution crashes out immediately. A metastable solution that would not crop at the bench will crop here. `NUCLEATION` is a named engine gap in the project's own notes, arrived at in milestone S3.

9.4 Filtration and where yield actually goes

Once solids exist you can filter them, and this is where a simulator can either be honest or quietly generous. The project's rule is stated as a principle:

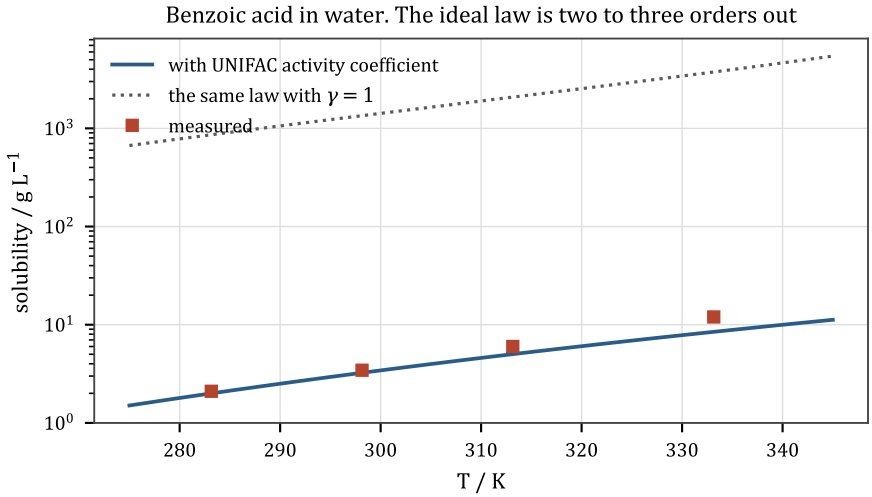

Figure 9.1: The same law, with and without one factor.

THE POINT

A loss must be a mechanic, never a tax. A yield loss has to come from something the model does, so that a player can act on it — not from a multiplier applied at the end.

Applied to the benzoic-acid preparation, that rule produced three findings, two of which were *refusals*:

- **the crystal crust is real and it is worth 9 points of yield.** Mother liquor clings to the crystal surface; it is a genuine holdup, and modelling it takes the preparation from 93% to 84%. Kept.
- **film holdup on the glassware is worth nothing here.** Measured, then dropped.
- **crystal occlusion — solvent trapped inside growing crystals — was killed by arithmetic before it was built.** Somebody sampled the trajectory, worked out the size of the effect, found it negligible at these crystal growth rates, and did not write the code.

That last one is the project's habit at its best: *sample the trajectory before implementing the term*, which is the same discipline that shaped the `wait_until` vocabulary in Chapter 23.

9.5 Ionic solids, and the law that does not apply to them

Now the hard part. Table salt is not a molecule. It is a **lattice**: a repeating three-dimensional array of Na^+ and Cl^- ions held together electrostatically. It has no molecular graph in any useful sense — `[Na+].[Cl-]` is a formula, not a structure.

And the fusion law does not describe it dissolving. Applied to an ionic lattice it is wrong by up to three orders of magnitude **in both directions**, measured against tabulated 298 K solubilities:

salt	error
NaCl	407× too **insoluble**
K_2CO_3	585× too insoluble
Na_2CO_3	251× too insoluble
KNO_3	2.6× too **soluble**
$CaCO_3$	11× too soluble

6,400X OF SPREAD WITH THE SIGN FLIPPING IS NOT A BIAS, IT IS THE WRONG LAW

T_m and ΔH_{fus} describe lattice → **melt**. Dissolution is lattice → **hydrated ions**, and the hydration energy — which is enormous, hundreds of kJ/mol — appears in neither quantity.

So `properties/thermochemistry.py` **refuses a lattice SMILES by name** rather than handing it to a law that is measurably wrong, and `mineral_data.py` is reference data rather than a provider tier. What a mineral in a flask actually *is*, is its ions, and that already works.

9.6 The solubility product

The right law for a lattice dissolving is a **solubility product**. For $M_aX_b(s) \rightleftharpoons a\,M^+ + b\,X^-$:

$$\Delta G_{\text{diss}} = a\,\Delta G_f(M^+) + b\,\Delta G_f(X^-) - \Delta G_f(\text{solid}), \qquad K_{\text{sp}} = e^{-\Delta G_{\text{diss}}/RT}.$$

Both halves are formation values from the elements, so **the subtraction means something only if both are on the same basis**. That sentence is the entire history of `properties/solubility_product.py` and it produced two of the project's most instructive findings.

A ZERO IS NOT DATA; IT IS AN ASSERTION ABOUT THE CONSUMERS

Before this module, the only ion values in the project were "spectator zeros" — $\Delta G_f(Na^+) = 0$, on the reasoning that a spectator ion cancels out of every equilibrium. Measured over 13 minerals, a naive K_{sp} built on those zeros returned a float for nine of them and was **25–29 decades out with the sign flipping**; blue vitriol came out at 76 mol/L, denser than the crystal.

The cause is exact and worth carrying: *a spectator cancels from an equilibrium only when it appears on both sides.* Every proton transfer has the cation unchanged across the arrow, which is why $[Na^+] = 0.0$ is exactly right there and why the project's five pH invariants hold. A solubility product is the one consumer where it appears **once**, so the entire hydration Gibbs energy that the zero stands in for lands in ΔG_{diss} — about 262 kJ/mol for sodium, which is 46 decades.

A REFUSAL FROM AN API IS NOT EVIDENCE THAT THE DATA IS ABSENT

The first measurement also recorded that the fix could not be automated, because the `chemicals` package "has no aqueous ion values and hands back the gas-phase ion" — and that was *measured*: `Hfs`, `S0s` and `Hfl` are all `None` for Na^+, and `Hfg` is +609,343 J/mol.

That was true of the functions and false of the package. `chemicals` 1.5.2 ships a file, *CRC Thermodynamic Properties of Aqueous Ions*, with 173 ions on the conventional scale — and no accessor function reads it. It is now `properties/ion_data.py`, 58 ions, each cross-checked by re-deriving its ΔG_f from its own ΔH_f and $S(\text{aq})$ against the element reference states, worst residual 0.85 kJ/mol against a tolerance of 1.0.

This is the exact mirror image of the project's older rule that *a successful call can be a wrong answer*, and it cost a milestone's worth of planning.

9.7 Reactions inside and at the surface of a crystal

A crystal can react while staying a crystal — a lime kiln, $CaCO_3(s) \rightarrow CaO(s) + CO_2(g)$, is matter changing identity without leaving the solid phase. Chapter 21 covers how that is represented; the chemistry to know now is one fact:

A pure solid has unit activity. It does not appear in the equilibrium expression at all. So calcite and quicklime sitting together fix p_{CO_2} at $K(T)$ regardless of how much of each is present.

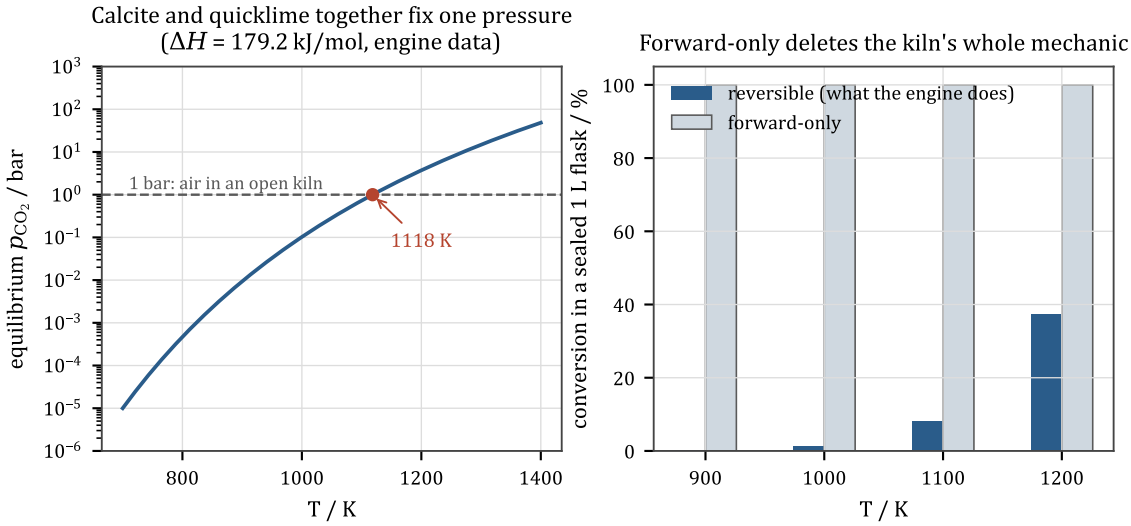

Figure 9.2: Engine data. The kiln has a threshold, and forward-only chemistry deletes it.

That is why a real kiln has a threshold temperature (Figure 9.2, left: about 1120 K under 1 bar of air on the engine's own curated data) and why sweeping the CO_2 away makes it go at lower temperature. The right-hand panel is the measured consequence of getting this wrong: a forward-only model calcines completely at any temperature you are willing to wait at, which deletes the kiln's entire mechanic.

CHAPTER 10

Water, ions, acids and bases

10.1 Ions

An atom or molecule that has lost or gained electrons carries a net charge and is called an **ion** — Na^+ (a sodium that lost one), Cl^- (a chlorine that gained one), SO_4^{2-} (a whole polyatomic group carrying two extra electrons).

Ions exist in water because water is extremely polar: each molecule has a partial negative charge on the oxygen and partial positives on the hydrogens, so water molecules cluster around an ion with the appropriate end inward and stabilise it enormously — hundreds of kJ/mol. That energy is called **solvation** or **hydration** energy, and it is the entire reason salt dissolves and the entire reason it does not dissolve in oil.

> **IF YOU KNOW PHYSICS**
>
> Quantitatively, the leading term is just electrostatics in a dielectric. Charging a sphere of radius r carrying charge ze inside a medium of relative permittivity ε costs (Born, 1920)
>
> $$G_{\mathrm{el}} = -\frac{N_A z^2 e^2}{8\pi\varepsilon_0 r}\left(1 - \frac{1}{\varepsilon}\right),$$
>
> which is the self-energy of a charged sphere with the medium's screening in it. Water has $\varepsilon \approx 78$; toluene has $\varepsilon \approx 2.4$. The difference between those two is why an ion stays in the aqueous layer when you shake a separating funnel.

10.2 Acids and bases

An **acid** is a proton donor; a **base** is a proton acceptor. In water:

$$\mathrm{HA} + \mathrm{H_2O} \rightleftharpoons \mathrm{A^-} + \mathrm{H_3O^+}$$

The equilibrium constant of that reaction is the **acid dissociation constant** K_a, and because it spans about twenty orders of magnitude it is always quoted as $\mathrm{p}K_a = -\log_{10} K_a$. Small $\mathrm{p}K_a$ = strong acid. Sulfuric acid's first proton is around -3; acetic acid is 4.76; water itself is about 15.7.

pH is $-\log_{10}[\mathrm{H_3O^+}]$. Pure water at 298 K has $[\mathrm{H_3O^+}] = 10^{-7}$ M, hence pH 7. Below 7 is acidic, above is basic.

THERE IS NO PH SOLVER IN THIS CODEBASE, AND THERE SHOULD NOT BE ONE

Acid dissociation is *chemistry*, so it enters as ordinary reversible reactions:

$$\mathrm{HA} + \mathrm{H_2O} \rightleftharpoons \mathrm{A^-} + \mathrm{H_3O^+}, \qquad 2\,\mathrm{H_2O} \rightleftharpoons \mathrm{H_3O^+} + \mathrm{OH^-}$$

and everything already built handles them. Detailed balance fixes each reverse rate from the thermochemistry; the stiff integrator resolves the (very fast) equilibrium; and the network builder's charge-balance check — which has been enforcing electroneutrality on every reaction since Layer 3 was written — finally earns its keep.

pH is then a *readout*, $-\log_{10}[\mathrm{H_3O^+}]$, not a state variable.

The results, all emergent:

result	chemsim	reference
pure water	pH 7.00	7.00
0.1 M acetic acid	pH 2.89	2.88
half-neutralised	pH 4.76	$= \mathrm{p}K_a$, exactly
equivalence point	pH 8.88	basic, as acetate is a weak base
0.05 M $\mathrm{H_2SO_4}$	pH 1.24	~1.1, with the correct $\mathrm{HSO_4^-}/\mathrm{SO_4^{2-}}$ split

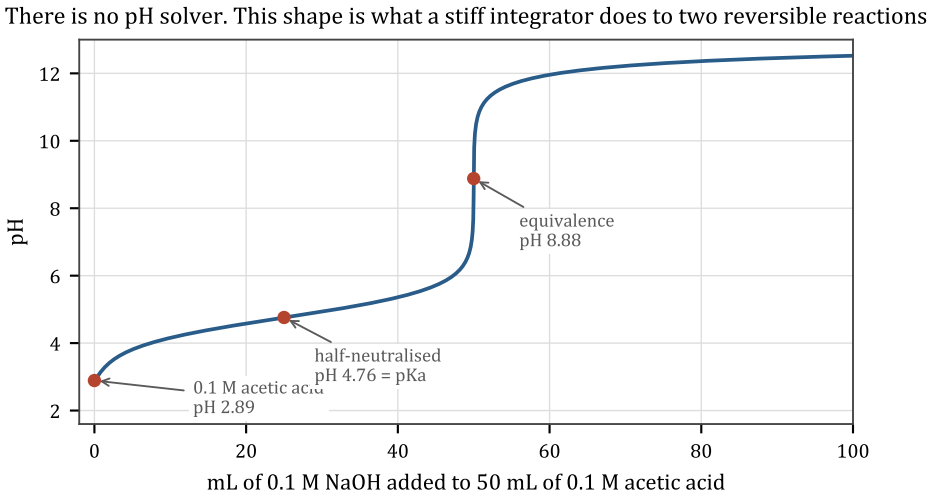

Figure 10.1: A titration curve, which is what a stiff integrator does to two reversible reactions.

10.3 Two decisions that make this work

Write dissociation with water on both sides. $\mathrm{HA} + \mathrm{H_2O} \rightleftharpoons \mathrm{A^-} + \mathrm{H_3O^+}$ has $\Delta n = 0$, where the more familiar $\mathrm{HA} \rightleftharpoons \mathrm{A^-} + \mathrm{H^+}$ has $\Delta n = +1$. That matters more than it looks: a mole-changing

reaction drags in the activity-to-molarity standard-state conversion (Chapter 4), and this project's formation data is ideal-gas while aqueous ion data is on the molarity scale. Writing it the balanced way makes the conversion **cancel exactly**, so the two unit systems never have to be reconciled.

Derive ion formation data from pKa, against this project's own water entry. Rather than importing tabulated aqueous ion values — which are referenced to liquid water and would silently disagree with our ideal-gas water — each ion's Gibbs energy is back-calculated so the measured pKa comes out right *with the water value this project already uses*:

$$\Delta G_{\mathrm{rxn}} = 2.303\,RT\,\mathrm{p}K_a, \qquad \Delta G_f(\mathrm{A}^-) = \Delta G_f(\mathrm{HA}) + \Delta G_{\mathrm{rxn}},$$

with the convention $\Delta G_f(\mathrm{H_3O^+}) = \Delta G_f(\mathrm{H_2O})$, i.e. the proton is the zero. The resulting numbers are not literature aqueous values and are not labelled as such. They are internally consistent constants that reproduce measured acidity.

TWO DIFFERENT ZEROS, THREE DECADES APART, THAT MUST NEVER BE SUBTRACTED

This project now has **two** ion tables on **two** different conventions, and the distinction is load-bearing:

- `properties/electrolyte.py` back-calculates each ion from a measured pKa against this project's water, with $\Delta G_f(\mathrm{H_3O^+}) = \Delta G_f(\mathrm{H_2O}, \ell)$. Right basis for a **proton transfer**. Chloride reads −111.73 kJ/mol.
- `properties/ion_data.py` is the conventional aqueous scale, anchored on $\Delta G_f(\mathrm{H^+}, \mathrm{aq}) = 0$. Right basis for a **lattice subtraction**. Chloride reads −131.20 kJ/mol.

Neither number is wrong. Mixing them costs **3.4 decades of** K_{sp}. There is no import between the two modules and there should not be one.

THE ANCHOR WAS THE ACID, AND FOUR ROWS OF THE TABLE ARE CATIONS

`electrolyte.py` anchored each ion on its *acid*, unconditionally. But four rows of the pKa table are **cation/neutral** pairs whose acid *is* the ion — an anilinium ion, for example — and the ordinary providers refuse to price a charge. Those four rows were dead.

The rule is now: a neutral acid anchors its anion, a neutral base anchors its cation, and the second is the first read backwards. And the neutral member must be taken in its **liquid** standard state, because a pKa is a solution-phase measurement — skip that and every pKa moves by about three units.

10.4 An ion in a two-phase system: the Born term

Everything above is inside one liquid. Across a **liquid–liquid interface**, $\gamma = 1$ for ions is not a bounded error: equality of activity with $\gamma = 1$ on both sides means an ion partitions to *equal mole fraction* between water and toluene. So the project's phase-splitting code refused to split

any electrolyte at all, and the most common workup in preparative chemistry — acidify, extract, wash the organic layer — was not expressible.

TWO IONIC GAPS, AND ONLY ONE OF THEM IS THIS ONE

- **(a) ionic strength *within* one phase** — Debye–Hückel / Davies. It is what makes a concentrated brine's ions less active than its concentration says, and it is what salting-out is. **Not modelled here.**
- **(b) ion transfer *between* phases — Born**. The electrostatic cost of moving a charge out of a high-dielectric medium into a low one. That is what holds an ion in the aqueous layer, and it is what `properties/dielectric.py` computes.

Conflating the two wastes the work, and the project says so at the top of the module.

Only *differences* of the Born energy are used. An ion's reference state here is infinite dilution in water, so the quantity wanted is the transfer:

$$\ln \gamma_i(\text{phase}) = \frac{A_i}{RT}\left(\frac{1}{\varepsilon_{\text{phase}}} - \frac{1}{\varepsilon_{\text{water}}(T)}\right), \qquad A_i = \frac{N_A z_i^2 e^2}{8\pi\varepsilon_0 r_i}.$$

In water this is exactly zero, at every temperature, by construction. That is why it is written as a transfer rather than as a solvation energy, and it is what makes the existing pH invariants safe — the anchors were derived at $\gamma = 1$ against water, and in water γ is still exactly 1. (Every pKa in the table was re-measured afterwards anyway, because "safe by construction" is a claim and not a check.)

A PERMITTIVITY IS A MIXTURE PROPERTY, SO IT CANNOT FULLY COLLAPSE

$\varepsilon_{\text{phase}}$ depends on composition, so the Born term lands in the same place UNIFAC did. The project's standard question — *what uniform array form does this collapse to?* — has a clean answer: an $(n, 4)$ block that is a function of temperature alone, $[A_i \mid \varepsilon_{\text{pure},i}(T) \mid v_i(T) \mid \varepsilon_{\text{water}}(T)]$. Three of those four columns already existed; the only thing left in the hot loop is the mixing rule, which is three array operations. The kernel still evaluates one polynomial form and has never heard the word "Born".

The mixing rule is Oster's (1946), Onsager's theory applied to a mixture: $f(\varepsilon) = \sum_i \phi_i f(\varepsilon_i)$ with $f(e) = (e-1)(2e+1)/9e$ and ϕ the **volume** fractions — permittivity is a bulk polarisation property, so volume is the right weighting.

AN UNCLIPPED GAMMA REPORTS SUCCESS WITH A 1E9 MOL DIPOLE

The Born exponential is unbounded. An ion in a very low-dielectric phase gets an enormous positive $\ln\gamma$, and the transfer term then drives a quantity that should be zero to something like 10^9 mol — a number with no physical referent at all, produced by a run that reports success. The fix is a ceiling on the exponent, justified as a resolution limit rather than as physics, and stated as such.

10.5 What protonation buys, and what it cannot fix

Milestone G5 added the ability for a molecule's *protonated form* to be a distinct species with its own reactivity — an aniline that becomes an anilinium ion in acid, and stops being nucleophilic. It is the right model, and it is honest about its own limit:

THE ENGINE'S PH FLOOR IS -0.79 AND THE CROSSOVER IT NEEDED IS -9.42

The anilinium split is the correct chemistry, and it **cannot** fix aniline. The crossover pH at which the split matters is -9.42; the engine's floor, set by the concentration of water itself, is -0.79. Real nitrating mixtures are *drier* than water — and a drier acid is a **less** acidic pot on this model, because the model's acidity is mediated by hydronium and there is not enough water to make it. The gap is a missing model of non-aqueous acidity (the Hammett acidity function H_0), not a missing parameter.

Reporting a bound like that, rather than fudging a constant until aniline behaves, is the project's standard.

CHAPTER 11

Electricity as a reagent

11.1 Oxidation and reduction

Some reactions transfer electrons rather than atoms. Losing electrons is **oxidation**; gaining them is **reduction**; the two always happen together, because electrons have to come from somewhere.

$$\mathrm{Zn} + \mathrm{Cu}^{2+} \rightarrow \mathrm{Zn}^{2+} + \mathrm{Cu}$$

Zinc is oxidised, copper reduced, and two electrons crossed over. If you physically separate the two halves and connect them by a wire, the electrons have to travel through the wire, and you have a battery.

11.2 The cell, and nFE

Run it backwards — push electrons through the wire from an external supply — and you can drive reactions that would not otherwise go. That is **electrolysis**, and it is how aluminium, chlorine, sodium hydroxide and pure hydrogen are made industrially.

The bookkeeping is one equation. Moving n moles of electrons through a potential difference E does electrical work

$$w_{\mathrm{el}} = nFE, \qquad F = N_A e = 96{,}485\ \mathrm{C/mol},$$

and that work is available to the reaction. So the criterion becomes

$$\Delta G_{\mathrm{effective}} = \Delta G_{\mathrm{chem}} - nFE,$$

and **a reaction whose chemistry costs less than the cell supplies runs**. The voltage at which the two balance is the **decomposition potential**:

$$E_{\mathrm{dec}} = \frac{\Delta G_{\mathrm{chem}}}{nF}.$$

THE WHOLE OF ELECTROCHEMISTRY HERE IS ONE SUBTRACTION

`ReactionTemplate` grew one field, `electrons`; `build_network(cell_potential=...)` says what the supply is set to; nFE joules come off the reaction's Gibbs energy. **No gate, no flag, no new term, and no Layer 4 code at all.**

Nothing declares a decomposition potential, and the numbers fall out of formation data alone: **1.441 V for water** against a book 1.229, and **2.362 V for brine** against 2.186.

11.3 A half reaction does not conserve charge, so every template here is a whole cell

$2\,Cl^- \rightarrow Cl_2 + 2e^-$ does not balance charge, and a species list that does not balance charge is exactly what the network builder rejects. There is no electron species in this project and there should not be: an electron here would be a state-vector entry with a concentration, and the electrons in a cell are not in the flask, they are in the wire.

So every electrode template is a **whole cell** — anode half plus cathode half, electrons cancelled, charge balanced. That is not a compromise; it is what the catalog rows already say. "sodium-chloride + water → sodium-hydroxide + chlorine + hydrogen" *is* the cell, not the anode. And it makes the arithmetic honest, because ΔG of a half reaction is not measurable without a reference electrode and ΔG of a cell is.

THE PRICE

A real cell's anode and cathode reactions are chosen independently by the electrode material and the potential. Here each pairing is a separate template that has to be written: four templates are four pairings, not two anodes times two cathodes. That is a representation limit, and it is stated.

11.4 What the barrier means on an electrode reaction, and why it is not invented

An electrode reaction still needs an E_a, and here the identity that makes it meaningful is nice enough to be worth spelling out.

Evans–Polanyi (Chapter 18) says $E_a = E_a^\circ + \alpha\,\Delta H$. Put the cell's electrical work inside ΔH, and

$$E_a = E_a^\circ + \alpha\,(\Delta H_{\text{chem}} - nFE)$$

which is **exactly the Butler–Volmer equation**, with α the electrochemical transfer coefficient at its conventional value of 0.5. So E_a° is the **activation overpotential** in energy units, $E_a = nF\eta_a$, where η_a is the extra volts a real cell needs on top of its decomposition potential before it passes appreciable current.

THE POINT

Those are measured quantities with a century of Tafel data behind them, and the two that matter here are wide apart: oxygen evolution is notoriously sluggish ($\eta_a \approx 0.5$ V on most anodes), chlorine evolution is not (≈ 0.1 V on a coated titanium anode).

That gap is the entire reason a brine cell makes chlorine rather than the oxygen its thermodynamics prefers, and declaring it as a barrier in joules is the only way this engine can express it. It is also a clean example of kinetics beating thermodynamics — the same phenomenon as Figure 5.3, in a different costume.

11.5 Where it stops, measured rather than assumed

`barrier` floors at zero, so far above E_{dec} every electrode reaction runs out of barrier at its own rate and the selectivity goes with it. A real cell is transport-limited there; this one is not limited by anything, because **nothing here budgets current**. That is reported by `validation/cell_potentials.py` rather than tuned away.

THE MILESTONE'S OWN HEADLINE CLASS DID NOT SURVIVE ITS ROW CHECK

M8 was scheduled because `electrolysis` was the top row of the coverage queue, worth "+3 routes". Reading its four rows found **three different mechanisms**: aqueous electrolysis (built), molten-salt electrolysis (a melt is not a phase this project has), and amalgam electrolysis (a marker with no molecular graph). Split on the project's own standard that *a reaction class names a mechanism, not an outcome*, the top row is worth **+1**.

The other +2 came from a different class entirely, which happened to be fully built. Net: +3 routes, from a curve that promised +3 from one class and was wrong about which one.

PART II

Where the numbers come from

CHAPTER 12

Estimating properties from structure

Every equation in Part I has constants in it. ΔH_f, ΔG_f, $C_p(T)$, T_b, T_m, ΔH_{vap}, ΔH_{fus}, Antoine coefficients, UNIFAC group parameters — and there are 1,583 compounds in this project's corpus. Measured values exist for a few hundred of them.

So the properties layer has to *estimate*, and this part is about how, and about how much that estimate can be trusted.

THE GENERALISATION MECHANISM, TWICE

Reaction templates generalise *reactions*: one SMARTS rule covers every substrate bearing the right functional group. Group contribution generalises *properties*: one table of group values covers every molecule made of those groups.

They are the same idea applied to the two halves of the problem, and together they are why a novel molecule — one nobody has ever tabulated, one this simulator just invented by applying a template — still gets a full set of numbers.

12.1 The idea

If a molecule's energy is roughly a sum over its bonds, then it is roughly a sum over its *parts*. Chop the molecule into a set of recognised fragments, count them, and add up tabulated contributions:

$$\Delta H_f \approx a_0 + \sum_k n_k \, h_k$$

where n_k is how many of group k the molecule has and h_k is that group's tabulated contribution.

IF YOU KNOW PHYSICS

This is a linear model, and the group values are its regression coefficients, fitted by least squares over a training set of measured compounds. Everything that is true of a linear regression is true here: it interpolates well inside its training distribution, it extrapolates badly, its residuals are not independent (a molecule with two errors of the same sign compounds them), and its failure mode on out-of-distribution inputs is a **confident wrong number** rather than a refusal.

Two properties are less obvious and both matter downstream. First, some functional forms are non-linear in the group sums — Joback's critical temperature is $T_c = T_b\,[0.584 + 0.965\Sigma - \Sigma^2]^{-1}$

— so errors do not simply add. Second, and this is the killer, *contributions that appear on both sides of a reaction cancel exactly*, which is a feature until it is a bug.

12.2 Step one: fragmentation

Before you can add group values you have to decide which groups a molecule is made of, and that is a covering problem with genuine ambiguity: the atoms of a carboxylic acid (`-COOH`) also look like an `-OH` next to a `C=O`.

`properties/fragmentation.py` is the shared machinery — Joback and UNIFAC use the same matcher with different tables — and its algorithm is:

1. **Try groups in priority order, highest first.** Priority encodes chemical specificity: `-COOH` must claim its atoms before `-OH` and `>C=O` can split it between them, and an ester's `CH3COO` must be tried before a bare `CH3`.
2. **Each match greedily claims a disjoint set of heavy atoms.** A match overlapping an already-claimed atom is rejected, so every atom belongs to exactly one group.
3. **Verify, twice over.** Every heavy atom must be claimed, *and* the summed atom tally of the assigned groups — including hydrogens, which the patterns never claim explicitly — must equal the molecular formula.
4. If step 3 refuses, **search**.

STEP 3 IS THE ONE THAT EARNS ITS KEEP

Group patterns are written to be readable, not airtight. A pattern intended for an ether will happily match an alcohol's oxygen and quietly lose a hydrogen. The formula check turns that into a loud failure instead of a plausible wrong number — which matters because **group contribution is at its most dangerous when it succeeds on a molecule it does not actually cover.**

Step 4 is a depth-first re-cover, and its design contains two rules worth carrying into any similar problem:

- **The search runs only after greedy has been refused, and that ordering is load-bearing rather than an optimisation.** Every decomposition the module returns today is the greedy one; a molecule that fragments now fragments identically afterwards. What the search can turn into an answer is a *refusal*, never a *different answer*. Measured over the corpus: 20 of 1,155 neutral organics fail greedy for exactly this reason and no other.
- **A search that runs out of budget refuses, and says which refusal it is.** "I did not find a cover" is not "there is no cover", and the two must not be reported as though they were the same statement.

12.3 Joback

Joback and Reid (1987) is the workhorse. Given the group counts it supplies ΔH_f, ΔG_f, $C_p(T)$ as a cubic, T_b, T_m, T_c, P_c, V_c, ΔH_{fus} and ΔH_{vap} — everything Part I needs, from a graph.

It has two structural limits that better data cannot fix.

JOBACK CANNOT DISTINGUISH HOMOLOGUES, AND THAT IS EXACT RATHER THAN APPROXIMATE

Group contributions are additive, so the $CH_3 \rightarrow C_2H_5$ difference **cancels exactly** between an alcohol and the ester it makes. Esterifying methanol and esterifying ethanol therefore came out of this project with an *identical* gas-phase ΔG_{rxn} of -7.35 kJ/mol.

No amount of downstream care recovers a distinction the estimator never made.

The second limit is coverage: Joback has **no groups at all** for whole functional classes — aryl aldehydes, formamides, sulfoxides, sulfones, anhydrides — and no metals, silicon, boron or phosphorus. Nine such species have curated records in this project; every other member of those classes was unreachable.

And its accuracy is a few kJ/mol, which Chapter 4 already told you is a factor of 2–4 in K. For methanol it is 17 kJ/mol — a factor of a thousand.

12.4 Benson

The second estimator sits above Joback and fixes both structural limits *by construction*, because a **Benson group is a polyvalent atom together with the types of its ligands**.

Methanol's carbon is `C-(H)3(O)`. Ethanol's are `C-(C)(H)3` and `C-(C)(H)2(O)`. Different groups — so the homologues cannot collapse onto each other.

Measured head to head on 82 curated ideal-gas species:

	median ΔG_f error
Benson	**1.6 kJ/mol**
Joback	2.8 kJ/mol

with the gains concentrated exactly where Joback is weakest: acetanilide 4.7 kJ/mol out against Joback's 89.4, a branched octane 7.7 against 30.6. On a bench-realistic target set Benson assigns 26 of 26 against Joback's 17.

Benson refuses about 12% of species (unmapped groups, heteroaromatics, anything under three heavy atoms), which keep Joback's estimate. So Benson improves **accuracy** rather than coverage — and note what it does *not* supply: T_b, T_c, P_c, V_c. Group additivity is a statement about *formation* quantities and says nothing about critical properties. That gap is Chapter 13.

WHY BENSON IS NOT A SMARTS TABLE

Joback and UNIFAC groups are overlapping substructure patterns, so something has to arbitrate, hence the priority-ordered greedy matcher. Benson groups are not patterns at all: **every**

polyvalent heavy atom contributes exactly one group, decided by its own element and bonding and its neighbours' types.

So there is nothing to arbitrate and no ambiguity to resolve, and expressing it as SMARTS would need one pattern per ligand combination — hundreds — and would *reintroduce* a matching ambiguity the scheme does not have. It runs off a plain topology description instead, which is why Layer 0 gained a `topology()` method and stayed sealed.

12.5 Four traps in the Benson data, all silent

The group values come from MIT's Reaction Mechanism Generator database, the open machine-readable form of Benson's tabulation plus later revisions. Building `benson_data.py` from it turned up four failure modes that are worth reading even if you never touch this code, because each is a general shape.

1. THE SOURCE MIXES UNITS INSIDE ONE FILE

Entries sourced from Benson are in **kcal**/mol; later revisions are in **kJ**/mol. `Cs-CsHHH` reads -10.2 and `Cs-OsHHH` reads -42.9 — values that agree to within a revision, printed a factor of 4.184 apart.

Assuming one unit throughout makes every oxygen group four times too large, which **validates fine on alkanes** (they have no oxygen groups) and then destroys anything with a functional group on it. The parser reads the declared unit per entry and raises on one it does not recognise.

2. LETTING FILE ORDER PICK BETWEEN DUPLICATE ENTRIES HAS MEASURED CONSEQUENCES

RMG has ~2700 entries against this project's ~750 keys: its tree carries second-order nodes and generic bracketed alternatives that all collapse onto one first-order key. The build picks the **most specific** candidate and prints every collision with its spread.

Letting file order decide instead made propylamine 15.8 kJ/mol too negative (a generic nitrogen node) and priced **every vinyl thioether as a sulfoxide** (a generic sulfur node).

3. S298 IS INTRINSIC ENTROPY AND NEEDS A SYMMETRY CORRECTION THE CALLER MUST APPLY

Benson's scheme does not intend S_{298} to include symmetry; the caller must apply $-R \ln \sigma$ for the molecule's symmetry number. Omit it and alkanes look fine while **every symmetric molecule is wrong** — benzene by $R \ln 12 = 20.7$ J/(mol K), which is 6 kJ/mol in ΔG_f.

4. A GROUP VALUE ONLY MEANS ANYTHING AGAINST THE BASIS IT WAS FITTED WITH

This is the general form of the other three, and it is the sentence the project records as the lesson: a fitted parameter carries its fitting context — units, reference state, symmetry con-

> vention, which other parameters it was regressed alongside. Transplanting a value out of that context produces a number that looks like data and is not.
>
> You will see this again in Chapter 18, where a Hammett ρ turns out to be meaningless without saying which σ scale it was fitted on.

Finally, RMG gives C_p at seven temperatures (300/400/500/600/800/1000/1500 K). Those are least-squares fitted here to the $a + bT + cT^2 + dT^3$ form the rest of the codebase uses — the same move as fitting Lee–Kesler to Antoine form, so that **one functional form reaches the kernel**.

CHAPTER 13

The physical half, and the boiling point nobody will estimate

13.1 Two halves that fail differently

The project splits every species' data into two halves and reports them separately, because they fail for different reasons and cost different things when they fail.

half	what it is	what an error there does
formation	ΔH_f, ΔG_f	sets every equilibrium constant. An error propagates into yields and never washes out.
physical	T_b, T_c, P_c, V_c	sets the vapour-pressure correlation. An error moves a boiling point and a headspace composition.

Averaging them into one coverage number would hide which of the two you are short of. Chapter 12 covered the formation half; this chapter is the physical one.

13.2 The chain

For a species with no tabulated Antoine constants, the project builds a vapour-pressure curve like this:

1. take a **measured boiling point** T_b;
2. get T_c and P_c from **Wilson–Jasperson**, which takes T_b as an input;
3. get V_c from **Fedors**, from structure alone;
4. invert **Lee–Kesler** at T_b to obtain the *acentric factor* ω — a one-number measure of how non-spherical the molecule is, defined so that $\omega = 0$ for argon;
5. evaluate Lee–Kesler's corresponding-states vapour-pressure curve over a temperature window and **fit it to Antoine form**.

Step 5 is the recurring move: whatever the correlation is, it is sampled and fitted at setup so that the kernel evaluates one functional form and has never heard of Lee–Kesler. The same is done for liquid molar volume (Rackett) and liquid heat capacity (Rowlinson–Bondi), both fitted to cubics in T.

WHY WILSON-JASPERSON AND FEDORS EXIST HERE AT ALL

The reason is architectural rather than numerical. Joback was the *only* source of critical properties in the project, and Benson — the better estimator above him — says nothing about $T_b/T_c/P_c/V_c$ because group additivity is a statement about formation. So **a species Joback refused had no physical half from anywhere, and its Benson formation half was unreachable no matter how well Benson priced it.**

Benson gets acetic anhydride's formation enthalpy to within 3.7 kJ/mol of measurement, and before `properties/critical.py` existed that value was computed, correct, and unusable.

Wilson–Jasperson takes T_b as an *input*, so the whole coverage problem collapses to one lookup.

Accuracy, measured on acetic anhydride against the CRC handbook:

	estimated	measured	error
T_c	600.9 K	606.0 K	0.8%
P_c	44.9 bar	40.0 bar	**12%**
V_c	290.3 cm^3	294.0 cm^3	1.3%

P_c is the weak link, and it feeds ω, which sets the entire vapour-pressure curve. So every entry enabled by this route has to pass a boils-at-1-atm cross-check.

13.3 Nothing here estimates a boiling point, and that is deliberate

THE POINT

The chain above needs T_b as an input and nothing in this project will estimate one. A species with no measured boiling point is **refused** rather than served a guess.

The reason is a specific trap, and it is the sharpest data-sourcing lesson in the repository.

A LIBRARY WILL HAND BACK YOUR OWN ESTIMATE LABELLED AS DATA

The `chemicals` package serves Joback *predictions* through the same accessor as its measured compilations. For metformin the only T_b source it offers is `JOBACK`, returning 609.52 K — **bit-identical** to what this project's own Joback implementation computes from the same groups, because it is the same method.

Looking that up would have closed a coverage gap by relabelling our own estimate as measured data: the number would move from the "estimated" column to the "measured" column having changed by exactly zero. Every estimated method is excluded and the species refused instead, which is why metformin and saccharin carry no T_b rather than a confident-looking number.

That trap has a second half worth stating separately.

BOILS-AT-1-ATM IS NOT AN INDEPENDENT CHECK IF THE FIT WAS ANCHORED ON TB

The natural cross-check on a vapour-pressure curve is "does it predict the measured normal boiling point?" — and if ω was obtained by *inverting the correlation at* T_b, that check is circular. It is guaranteed to pass and it measures nothing.

The project found this and fixed it in milestone S10 by making the fit *unanchored* for the species where an independent check was needed, at which point boils-at-1-atm becomes a real measurement again.

13.4 Two data tiers, and the empirical test that separates them

Measured data is not all the same kind of thing, and the project keeps a distinction on every value:

`experimental` — critically evaluated measurement: IUPAC, CRC, the NIST WebBook, Common Chemistry, Open Notebook melting points.

`compilation` — published, widely used, and **not auditable to a measurement**. The `chemicals` package says of one such source that "no data points are sourced in the work"; another mixes experimental with estimated values; a third is supporting material from an equation-of-state paper.

The decisive test for that distinction is empirical rather than bibliographic: one compilation source gives **saccharin a critical temperature of 968 K**, and saccharin decomposes near 500 K without ever boiling. That is not a measurement of anything.

So $T_c/P_c/V_c$ are taken from the experimental tier only; where none exists the record falls to Wilson–Jasperson and Fedors, whose error is *known* because `validation/physical_estimation.py` measures it. A number of unrecorded origin may itself be a group-contribution estimate from a method you cannot inspect, and a known error beats an unknown provenance. T_b gets no such choice — nothing here estimates one — so a compilation-tier T_b is accepted where it is the only source, and stamped.

13.5 The hand-typed list, and how big that gap turned out to be

A TABLE GENERATED FROM 37 HAND-TYPED NAMES, IN A 1583-COMPOUND CORPUS

`properties/physical_data.py` is a generated file. Until milestone S13 it was generated from a **hand-typed list of 37 species names**, and everything else in the corpus fell to Joback — silently, because a Joback record *resolves* without complaint.

Regenerating it from `data/catalog` itself gives **1,239 species, 896 with a measured boiling point**. The 881 estimates it replaced were off by a **mean of 6.1% and a worst of 111%**.

Two further findings came out of the same session, and both are about instruments rather than chemistry:

- **The gap was not exotic. It was the bench solvent.** Not obscure species — the things actually in the flask.
- **The instrument built to expose the gap undercounted it by 60%**, because the coverage audit's tier classifier was *parsing prose* out of provenance strings. The `thermochemistry` module had already written down why that would fail.

The effect on the two coverage halves, before and after:

	before S13	after
physical half **measured**	40 (2.5%)	**652 (41.2%)**
physical half falls back to Joback	964 (61%)	333 (21%)

13.6 What the two halves look like now

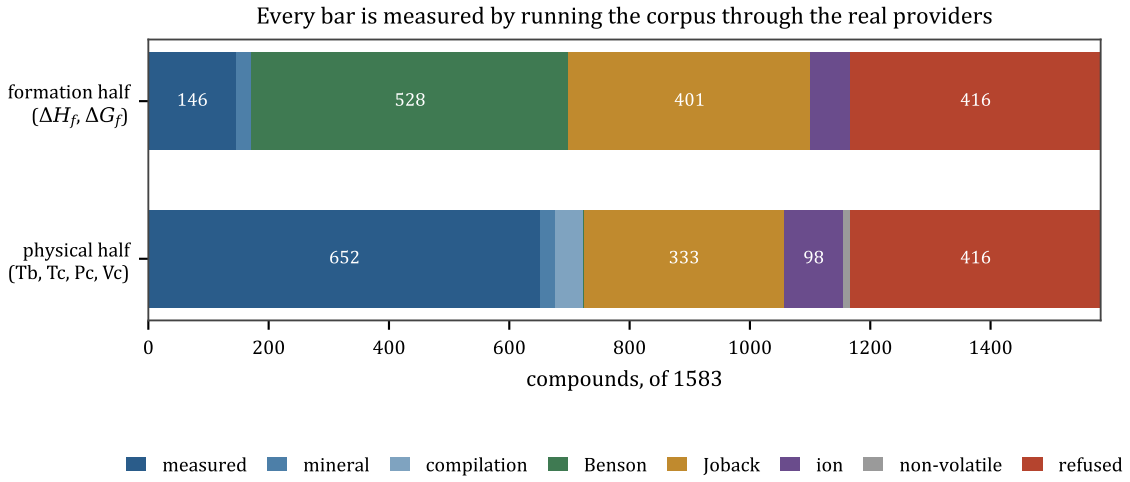

Figure 13.1: Both bars are measured by running the corpus through the real providers.

Figure 13.1 is the current state over all 1,583 compounds. Read the formation half first: a Joback-only formation half integrates without complaint and reports a confidently wrong equilibrium constant.

overall	count	of 1583
both halves resolve	1167	73.7%
formation half measured or Benson (not Joback)	766	48.4%
formation half falls back to Joback	401	25.3%
refused outright	416	26.3%
decomposes for UNIFAC (can enter a phase split)	857	54.1%

A NEGATIVE HEAT CAPACITY HAD BEEN IN THE ENGINE SINCE S4

Rowlinson–Bondi is a corresponding-states correlation for liquid C_p, good to about 5% for non-polar species and poor for hydrogen-bonding ones (it overestimates ethanol by ~40%, so alcohols, water and acids are curated). It can also, outside its domain, return a **negative** heat capacity — and it had been doing so for mercury since milestone S4. 103 rows still carry one.

A negative C_p means a vessel that cools when you heat it. It was found by an audit, not by anything going wrong, which is the usual way.

CHAPTER 14

Provenance: how a number earns the right to be used

This chapter has no equations in it. It is about the discipline that surrounds the numbers, and it is arguably the most transferable content in the project.

14.1 The rule

> **THE POINT**
>
> **Every value carries its source.** Not a comment — a field on the record, that downstream code and the user interface can both read. `ThermoData.source`, `Volatility.source`, and the tier column in the coverage report all exist so that "this came from a measurement" and "this came from a group-contribution estimate" are distinguishable at the point of use.

The reason is that group contribution is at its most dangerous when it *succeeds*. A refused species is a visible hole. An estimated species is an invisible one, and it will be quoted with the same confidence as a measured one by anything that does not ask.

14.2 The provider stack

Properties resolve through a tier list, highest first, and every tier is named on the result:

1. **curated measured data** — `formation_data.py`, `physical_data.py`, `element_data.py`, `mineral_data.py`;
2. **Benson group additivity** — better formation values, ~12% refusal rate;
3. **Joback** — broad coverage, several kJ/mol;
4. **refused** — with a reason.

Two of those tiers exist as *refusals with a route out* rather than as fallbacks: an element comes from `element_data` or is refused by name (Chapter 3), and an ionic lattice comes from `mineral_data` or is refused by name (Chapter 9).

14.3 Derive, do not transcribe

ΔG_f in `formation_data.py` is **derived** from that entry's own ΔH_f and S° against the CODATA element reference states, rather than transcribed from a table. That is deliberate: the two halves

of every entry are then thermodynamically consistent with each other by construction, which is what makes the entropy a caller derives from the pair the real one.

The same principle appears three more times:

- `ion_data.py` re-derives each ion's ΔG_f from its own row's ΔH_f and $S(\mathrm{aq})$, worst residual 0.85 kJ/mol against a tolerance of 1.0;
- `mineral_data.py` derives ΔG_f from ΔH_f and S° against the same element reference states;
- ΔH_{vap} is derived from the Antoine curve by Clausius–Clapeyron rather than taken from a second correlation, so both halves of the standard-state shift come from one source.

> **NEVER MIX SOURCES INSIDE ONE ENTRY**
>
> Both halves of a record come from the **same** database or the entry is refused. That rule bites on iron(II) sulfate, whose S° is 107.5 J/(mol K) from CRC and 120.93 from the WebBook — 13.4 apart, and worth 4 kJ/mol in ΔG_f. Taking the enthalpy from one and the entropy from the other produces a consistent-looking record that describes no substance.

14.4 Cross-checks that touch nothing the entry came from

A curated value is only as good as the independent thing it agrees with. Each tier here has a check built from correlations that never touched the formation tables:

For a species with both gas and liquid formation data, two identities must hold to within 3 kJ/mol:

$$\Delta H_f(g) - \Delta H_f(\ell) = \Delta H_{\mathrm{vap}}(298), \qquad \Delta G_f(\ell) - \Delta G_f(g) = RT \ln \frac{P^{\mathrm{sat}}(298)}{P^\circ}.$$

Agreement means three independent measurements line up. Of 102 candidate species, 83 gas and 5 liquid entries survived.

For an element whose reference state is condensed, shifting the ideal-gas value back down into its own phase must return zero:

$$\Delta G_f(g) + RT \ln \frac{P^{\mathrm{sat}}}{P^\circ} - \Delta H_{\mathrm{fus}} \left(1 - \frac{T}{T_m}\right) = 0$$

and nothing in that expression touched the formation table — P^{sat} comes from $T_b/T_c/P_c$ through Lee–Kesler, and ΔH_{fus} and T_m are separate measurements. Residuals: bromine -0.053 kJ/mol, mercury $+0.012$.

For a transcribed parameter table, the check is against an independent implementation. Every Joback group value, every UNIFAC R, Q and interaction parameter, and every PSRK gas parameter is cross-checked entry-by-entry against the `thermo` package in the test suite. The transcription is *verified*, not trusted.

14.5 Two rules about calls to other people's libraries

These are stated as a matched pair in the project's notes and they are worth learning together.

A SUCCESSFUL CALL CAN BE A WRONG ANSWER

The `chemicals` package returned a boiling point for metformin. It was our own Joback estimate. Chapter 13.

A REFUSAL FROM AN API IS NOT EVIDENCE THAT THE DATA IS ABSENT

The same package returned `None` for every aqueous ion property, which was recorded as "the data does not exist" and blocked a milestone. The data ships *inside the package*, in a TSV file no accessor function reads. Chapter 9.

14.6 An absent count is not a wrong count

One more distinction, from milestone S13, that is easy to get backwards when reporting a gap.

The hand-typed physical-data list was missing 1,202 species. That is **not** the same as 1,202 wrong numbers: an absent entry falls through to an estimate, and some estimates are fine. The honest measurement is *how far the estimates were off*, which is the mean-6.1%-worst-111% figure, not the count of absences.

Reporting the count would have overstated the finding. The project's own instrument did overstate a related one by 4.6×, by reading a tier out of prose.

14.7 What this discipline costs and buys

It costs a real amount: every provider has a refusal path, every generated table has a build script with an argument in its docstring, and several capabilities were *delayed* because their data could not be sourced honestly.

What it buys is that the project can say things like "31 of 173 named routes can run, and that is an upper bound rather than a measured count, and here is the route that proves the difference" — and be believed. A simulator that cannot distinguish its measured numbers from its estimated ones cannot make a claim like that at all.

THE POINT

The general form: **an estimator outside its domain is the class of bug here, and it does not announce itself.** Every guard in this project's properties layer exists to convert one of those into a refusal with a name on it.

PART III

The machine

CHAPTER 15

The architecture, and its two seams

15.1 Eight layers, strictly downward

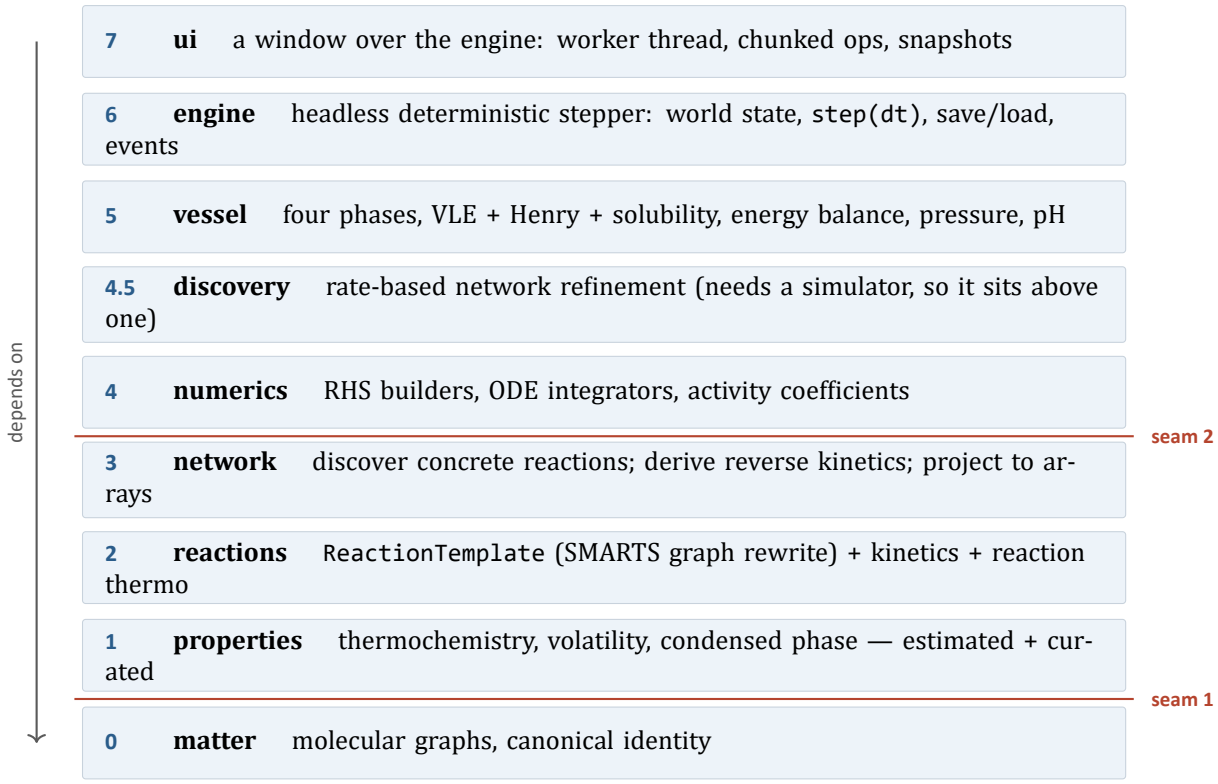

Figure 15.1: The stack. Dependencies point strictly downward, and the two red lines are the interfaces that were designed to be replaceable.

Nothing above a layer is imported by it. That is enforced by convention rather than by a tool, but it holds, and two of the boundaries are load-bearing enough to have names.

15.2 Seam 1: `matter` hides RDKit

Nothing above Layer 0 imports rdkit. Parsing, canonicalisation, substructure matching and template application all happen inside `chemsim/matter/molecule.py`, behind a domain type.

Two things this buys. The cheminformatics backend is replaceable — a Rust library, a different toolkit — without touching anything else. And RDKit types cannot leak into the engine, which matters more than it sounds: an RDKit `Mol` is unhashable, unpicklable and mutable, and a `Mo-`

lecule here is none of those. That is why a species can be a dictionary key and why a save file contains no molecule objects.

15.3 Seam 2: numerics sees only arrays

The hot integration loop consumes KineticArrays and PhaseArrays — numpy plus a list of species names — and knows nothing about molecules.

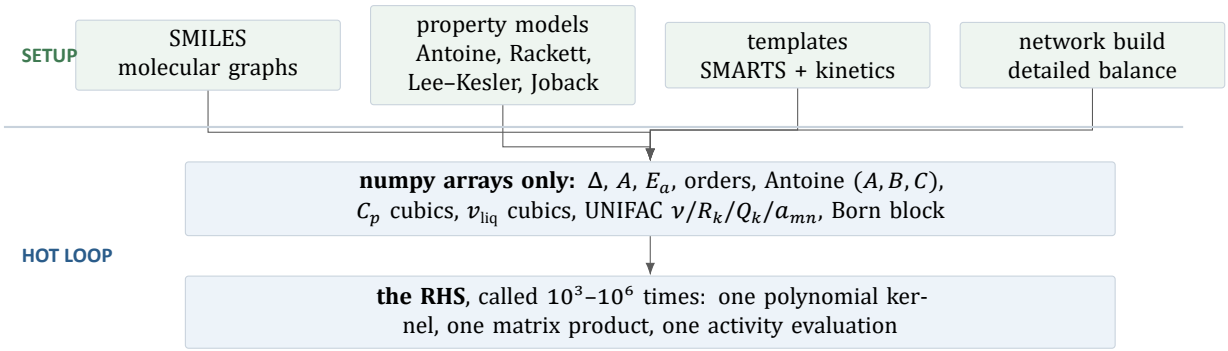

Figure 15.2: The setup/hot-loop split. Everything model-specific happens once, above the line.

CHEMINFORMATICS HAPPENS AT SETUP TIME, NEVER INSIDE THE LOOP

This is the discipline that makes the whole design tractable, and it is applied to property *models* as well as to molecules. Antoine, Lee–Kesler, Rackett and Rowlinson–Bondi are all evaluated and fitted to plain polynomial coefficients during assembly, so **the kernel evaluates one polynomial form and has never heard of any of them.**

Raoult's law and Henry's law arrive as the same array. A forward reaction and its thermodynamically derived reverse arrive as two rows of the same matrix. A melting model and a dissolution model arrive as one equation with one factor switched.

15.4 The one exception, and why it is not a leak

Activity coefficients depend on **composition**, and composition is the state vector — so unlike every other property they cannot be fitted in advance (Chapter 8).

The split *moves* rather than breaking. What is precomputed is the UNIFAC *parameter block*: group counts, size and surface parameters, the interaction matrix, all expanded to a dense subgroup basis at assembly time. What runs per step is the evaluation. Layer 4 still receives nothing but numpy; the arrays are simply richer, and the loop finally does real work.

The Born term (Chapter 10) lands in the same place for the same reason, and collapses to an $(n, 4)$ block that is a function of temperature alone plus a three-operation mixing rule.

15.5 Why Python, stated as a measurement rather than a preference

The hot loop's cost scales with the number of **species and reactions in a vessel** — a small, stiff ODE system, milliseconds in SciPy's C solvers — and **not** with the number of molecules. Python becomes a bottleneck for spatial gradients, large auto-generated networks, or many simultaneous vessels, and for those the `numerics` boundary lets a Rust kernel drop in surgically with nothing above it changing.

The measurement that bears on this, from Chapter 8: adding UNIFAC took the RHS from 140 µs to 231 µs, and the γ kernel itself is *flat* from 4 species to 25 — both numbers are numpy dispatch overhead on small arrays, not arithmetic.

> **THE POINT**
>
> So the case for Rust does not rest on any one model being slow. It rests on **fixed per-call overhead**, which a Rust kernel would collapse for the whole RHS rather than for one part of it. That is a different argument, and the project defers the work until real numbers justify it.

15.6 What lives where

you want to know	look in
how a molecule is represented	`matter/molecule.py`
where a number came from	`properties/*_data.py` and the `source` field
what reactions exist	`reactions/library.py`, `reactions/synthesis.py`
how a reaction becomes numbers	`network/builder.py`
the actual differential equations	`numerics/vessel_integrator.py`
what a flask does	`vessel/vessel.py`
how a run is made reproducible	`engine/world.py`, `engine/scenario.py`
how much chemistry is covered	`data/catalog/`, `validation/catalog_coverage.py`
why any decision was made	`MILESTONES.md`, and the module docstrings

Appendix C is a fuller version of that table.

15.7 A note on the module docstrings

They are long, and they are the primary documentation of this project. They do not describe what the code does — the code does that — they record *what was measured, what was tried and rejected, and what the alternative would have cost.* `numerics/jacobian.py` opens with a 200-line argument for a single inequality; `properties/solid_state.py` contains a four-row table showing why one modelling choice was rejected.

This manual is largely a reorganisation of that material into an order somebody can read it in.

CHAPTER 16

Layer 0: molecules in code

A short chapter, because the layer is short — 263 lines — and because Chapter 1 already covered what a molecule *is*. This is about what the type guarantees.

16.1 Molecule

```python
from chemsim.matter import Molecule

m = Molecule.from_smiles("CC(=O)Oc1ccccc1C(=O)O")   # aspirin
m.smiles          # the CANONICAL form, which is the identity
m.formula         # "C9H8O4"
m.topology()      # plain data: per-atom element, charge, H count, neighbours
```

Three guarantees, and each one is used somewhere upstream.

1. Identity is canonical SMILES. Two `Molecules` are equal iff their canonical SMILES match. That makes a `Molecule` hashable and usable as a dict key or set member — which is *how the network builder decides whether a freshly generated product is a new species*. Without a canonical form, applying a template twice by different routes would register the same substance twice, and the state vector would silently double-count.

2. Identity distinguishes stereochemistry. RDKit's canonical SMILES is isomeric by default, so R/S and E/Z are different species. That matters for steroids, chiral drugs, and anything where a template must not silently scramble a centre.

> **THE IDENTITY MODEL IS AHEAD OF THE REACTION MODEL**
>
> Templates do not yet *control* stereochemistry, so a graph rewrite can lose it. Distinguishing what you cannot control is the safer of the two failure modes, but it is still a mismatch — and it costs something measurable.
>
> **Not one template in the library spells stereochemistry on its product side** (0 of 50). So a rewrite that makes or touches a centre emits the *flat* species, and the flat species is not the one the corpus spells: hydrogenating isopulegol gives `CC1CCC(C(C)C)C(O)C1` where the catalog's menthol is `CC(C)[C@@H]1CC[C@@H](C)C[C@H]1O`. Both are real species by the rule above, and only one of them used to reach menthol's measured boiling point — the other fell to Joback, 43 K away, in the middle of a route the engine runs.

The *identity* half of that mismatch stands, deliberately. The *lookup* half is closed: a property table will now answer across an unspecified centre, under rules that never merge two species (Chapter 17). Making templates control stereochemistry remains future work.

3. No RDKit object escapes. Every method returns a string, a number, a `Molecule`, or plain data. `topology()` exists specifically so that Benson group additivity (Chapter 12) — which needs per-atom neighbour types rather than substructure matching — can be written above this layer without reaching through it.

16.2 Substructure matching and rewriting

Two more operations live here, both delegated to RDKit and both wrapped:

- **match a SMARTS pattern**, returning atom index tuples. Used by the fragmentation matcher (Joback, UNIFAC) and by template applicability tests.
- **apply a reaction SMARTS**, returning product `Molecules`, canonicalised.

IMPLICIT HYDROGENS, AND THE AMMONIA THAT WAS TWO DIFFERENT SPECIES

Chapter 1 mentioned that implicit hydrogens would cause exactly one serious bug. Here it is.

SMILES normally leaves hydrogens implicit, hung off their heavy atom's valence. But **hydrogen gas has no heavy atom** — `[H][H]` is two explicit hydrogen *atoms*, and any template that consumes H_2 must write it that way. Apply such a template and the ammonia it produces comes back as `[H]N([H])[H]`, which is a **different canonical string** from the `N` somebody charged into the flask.

Two state-vector entries for one substance, no reaction connecting them, and a mass balance that closes perfectly while the answer is wrong. `ReactionTemplate.run` now collapses explicit hydrogens after every rewrite. It is one line, and without it the entire Haber process is silently broken in a way no conservation check can see.

16.3 The one limitation that is deliberate

No tautomer resolution. Keto and enol forms are separate species. Good enough to bootstrap; revisit when tautomers matter. It is written in the module docstring so that nobody rediscovers it as a bug.

16.4 Why a wrapper at all

It would be simpler to pass RDKit `Mol` objects around. The reasons not to, in the order they bite:

1. **Serialisation.** A save file must not contain molecules, and RDKit objects are not serialisable. Because identity is a string, a `Scenario` (Chapter 23) stores SMILES text and rebuilds the network on load.
2. **Hashability.** Discovery is a fixpoint over a set of species. Sets need hashing.

3. **Replaceability.** The whole backend is one module's worth of calls.
4. **Determinism.** Species indices come from dict insertion order, and template application iterates lists, never sets — so a network built twice from the same inputs is identical, including the order of its rows. Chapter 23 depends on that completely.

CHAPTER 17

Layer 1: how a property actually resolves

Part II covered the estimation *methods*. This chapter is about the plumbing: what happens when something upstream asks for a number, and what the layer emits.

17.1 The providers

Layer 1 exposes a handful of provider objects, each of which answers one question about one species and stamps its answer with a source.

provider	answers	file
`ThermochemistryProvider`	ΔH_f, ΔG_f, S°, $C_p(T)$, T_b, T_m, T_c, P_c, V_c, ΔH_{fus}, ΔH_{vap}	`thermochemistry.py`
`VolatilityProvider`	Antoine (A, B, C), and whether it is a vapour pressure or a Henry constant	`volatility.py`
`CondensedProvider`	liquid molar volume, liquid C_p, as cubics in T	`condensed.py`
`UnifacProvider`	group decomposition, R_k, Q_k, the interaction matrix	`unifac.py`
`DielectricProvider`	permittivity, Born radii	`dielectric.py`
electrolyte provider	pKa-anchored ion formation data and the dissociation templates	`electrolyte.py`

Everything is keyed by **canonical SMILES**, which is the only identity in the system — and canonical SMILES is *isomeric*, so a spelling is part of the key.

A SPELLING USED TO SELECT A DATA TIER

Exactly one property table carries stereochemistry in its keys: the **generated** one, `MEASURED_PHYSICAL`, which inherited the corpus's spelling when it was built from the corpus. Every hand-typed table is flat, because a human types the simple form — 0 of 82 ideal-gas formation entries, 0 of 58 liquid, 0 of 50 curated records, 0 of 29 pKa rows. For 49 of the 212 corpus compounds spelled with stereochemistry that meant the two spellings of one substance resolved to *different sources*, and for lactic acid it meant neither spelling reached the best available record for both halves at once.

`properties/stereo_keys.py` lets a lookup cross that, under two limits that are the whole of its safety. It may cross an **ambiguity** and never a **difference**: a query naming no stereochemistry may take a record that names some, and a query naming some may take a flat record, but two differently specified spellings never share one — those are two species. And the unspecified side must be answered by **exactly one** record. That second guard fires: `MEASURED_PHYSICAL` holds seven skeletons with more than one stereoisomer, and without it a flat butenedioic acid would take maleic or fumaric acid's boiling point depending on dictionary order — **230 K apart**. Every value that arrives this way says so in its provenance string.

17.2 Resolution order

`ThermochemistryProvider.get(smiles)` walks a tier list:

1. is it an **element**? Then `element_data` or refuse by name. No estimator may price a reference state (Chapter 3).
2. is it an **ionic lattice**? Then `mineral_data` — and note that this *prices* it without making it *dissolvable*, because the fusion law is the wrong law for a lattice (Chapter 9).
3. is there a **curated measured record**? Use it.
4. can **Benson** fragment it? Use Benson for the formation half.
5. can **Joback** fragment it? Use Joback.
6. otherwise **refuse**, with a reason.

The physical half resolves on its own track — measured T_b where one exists, then Wilson–Jasperson and Fedors for the critical constants — which is why the coverage report has two independent columns and why a species can be Benson-priced and physically refused, or vice versa.

REFUSING TO DISSOLVE IS NOT REFUSING TO PRICE

This distinction was worth 16 routes and it was found by an audit rather than by a failure. `species-ready` — the coverage instrument's question "does every species in this route resolve?" — was asking only the three ideal-gas providers, which refuse an ionic lattice **by name and correctly**.

But `mineral_data` had been pricing those lattices on the solid basis since milestone M3. Nineteen compounds moved from *refused* to *mineral*, and sixteen routes with them — including the lime cycle, which the project had already declared complete end to end and whose example ran. **The audit was measuring the wrong provider set, and the number it produced was wrong in the safe direction, which is why nobody noticed.**

17.3 What the layer emits, and why it is polynomials

Nothing upstream ever sees a correlation. Layer 5 assembles `PhaseArrays` by asking each provider for each species and packing the answers into numpy:

```
Antoine (A, B, C)          one row per species    -> p_eq = x * gamma * 10^(A -
B/(C+T))
Cp_liq, Cp_gas              cubic coefficients     -> Cp = a + bT + cT^2 + dT^3
v_liq                cubic coefficients    -> liquid volume, hence concentration
Hfus, Tm                     scalars                -> the fusion law
latent heat                 cubic                 -> the energy balance's q_vap
UNIFAC nu, R_k, Q_k, a_mn  matrices              -> gamma, evaluated in the RHS
Born A_i, eps_i(T)         (n,4) block           -> ion transfer between layers
```

Anything non-polynomial — a corresponding-states correlation, in both the Rackett and Rowlinson–Bondi cases — is **sampled and fitted at setup**. That is not cosmetic. It means Layer 4 evaluates one polynomial kernel over one array and never learns that Rackett exists.

17.4 Two properties Layer 5 could not do without

`condensed.py` exists for two reasons that are easy to overlook:

- **liquid molar volume**, because the liquid volume is what turns moles into the concentrations the rate law uses — *and what lets a flask boil dry*;
- **liquid heat capacity**, because a vessel's temperature response is set by the liquid it contains, and ideal-gas C_p is roughly half the real value.

Estimation quality is stated honestly rather than assumed: Rackett is good to a few percent for most organics and about 10% low for water, which is anomalous; Rowlinson–Bondi is good (~5%) for non-polar species and poor for hydrogen-bonding ones, overestimating ethanol by ~40%. Since alcohols, water and acids are exactly the solvents this simulator cares about, those are curated and the correlation is the fallback for everything else.

17.5 A layering cycle, and how it was broken

A small but instructive piece of structure. `volatility` builds a provider and therefore imports `thermochemistry`. But `thermochemistry` needs an enthalpy of vaporisation for a record it assembles from *estimated* critical constants, and deriving that from Lee–Kesler would import `volatility` straight back.

The fix was to put the Lee–Kesler shape functions in `critical.py`, a module that depends on nothing but `matter`. That breaks the cycle — and it is where they belong anyway: they are a property *model*, not a *provider*. `volatility` re-exports them so its public surface is unchanged.

> **ASIDE**
>
> This is a general pattern worth naming: when two modules need each other, the thing they both need is usually a third, more primitive thing that has been misfiled inside one of them.

17.6 The report

Every provider can say what it did not know. `vessel.activity_model.report()` names the species held at $\gamma = 1$ and the main-group pairs missing from the published matrix; `electrolyte_report()` names ions with no data; `integrability_report()` names species that will make the solver unhappy.

Those strings are surfaced in the user interface rather than logged (Chapter 24), on the reasoning that "nothing is silently approximated" is only worth anything if somebody is shown what it said.

> **A CHANNEL THAT WAS REPORTED ALL ALONG AND THAT NOTHING READ**
>
> The refluxing rig destroyed 0.34 mol of its air for months. The loss was reported by `conservation_report` the whole time. Nothing was reading it.
>
> That is why the reports panel exists in the UI, and why several validation scripts now assert on report contents rather than merely printing them.

CHAPTER 18

Layer 2: reaction templates

18.1 The core idea

Instead of storing "acetic acid + ethanol → ethyl acetate + water", store a *transformation*:

```
[CX3:1](=[O:2])[OX2H1:3] . [OX2H1:4][CX4:5]
      >>  [CX3:1](=[O:2])[O:4][CX4:5] . [OH2:3]
```

That is this project's Fischer esterification template, verbatim. It says: find a carboxylic acid group and an alcohol group; make a bond between the acid's carbonyl carbon (atom 1) and the alcohol's oxygen (atom 4); release the acid's original hydroxyl (atom 3) as water.

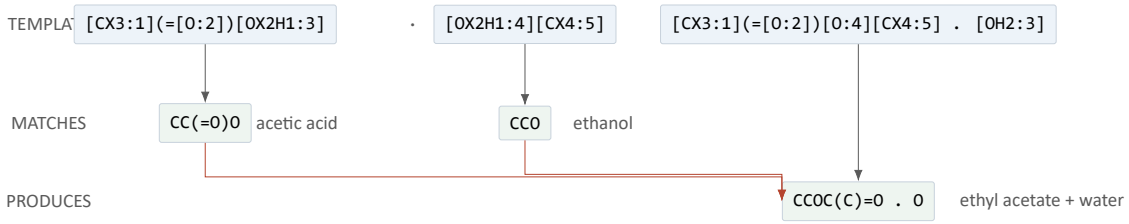

Figure 18.1: A template is a rule over graphs. The same rule fires on any acid and any alcohol, with no extra code and no entry in any table.

THE POINT

A few hundred templates generate an effectively unbounded space of concrete reactions, so the project never enumerates a combinatorial product dictionary. That is the whole reason this design is tractable, and it is the reason the simulator can produce a reaction nobody anticipated.

18.2 What a template carries, and how honest each field is

The project states this explicitly and it is the right calibration for anything the simulator outputs.

field	honesty
`smarts`	**real.** The transformation is chemistry, and its *specificity* is the selectivity mechanism.
`Ea`	**sourced.** Each barrier carries the literature band it came from, and the ordering *between* templates is the load-bearing part.

field	honesty
`alpha`	**derived where used.** Evans–Polanyi ties the barrier to the reaction enthalpy the network already computes.
`A`	the remaining **hand-authored** parameter, and the honest weak point. Order-of-magnitude choices inside the range physically sensible for the molecularity.
`reversible`	**never a free parameter.** The reverse Arrhenius pair is derived by detailed balance. There is no hand-typed reverse rate anywhere in this project.
`orders`	declared only where the template writes a *global* stoichiometry.
`phase`	`"liquid"`, `"gas"`, or `"any"`.
`solid_catalyst`	the name of a crystal that must be present.
`electrons`	how many cross the external circuit; non-zero makes it an electrode reaction.

18.3 Selectivity is SMARTS specificity

The oxidation template is written `[CX4;!H0:1][OX2H1:2]` — a carbinol carbon that still has a hydrogen to lose — and that one clause is the entire selectivity model for the family:

- methanol → formaldehyde, ethanol → acetaldehyde, propanol → propanal;
- a **secondary** alcohol gives a ketone (isopropanol → acetone), because the pattern never said how many hydrogens, only "not zero";
- a **tertiary** alcohol is refused, because there is no hydrogen on the carbinol carbon and you cannot make a carbonyl there without breaking a C–C bond;
- glycerol yields **both** the primary and the secondary oxidation product, from one template, because the pattern matches at two different sites.

None of that is enumerated. It follows from the pattern — which is why growing this library is cheaper than it looks, and why writing the pattern carelessly is the main way to get a confidently wrong answer.

A LIBRARY WITH ONE TEMPLATE CANNOT PRODUCE A SIDE PRODUCT

Before `reactions/library.py` existed, templates lived inline in whichever example needed one. The cost was not tidiness: **nothing ever competed.** A network with one template has purity 100% by construction, and every "impurity" the simulator could report was unreacted starting material or an ion.

The project's founding claim is that *yields, side products and temperature sensitivity emerge*, and two thirds of that was untested. The Phase-0 spike had hand-written three competing reactions and demonstrated exactly this; the real code had never reproduced it.

18.4 Barriers, and why the *ordering* matters more than the values

Ethanol over sulfuric acid gives **diethyl ether at 140 °C** and **ethylene at 180 °C**. Both are dehydrations of the same alcohol; which one you get is decided by temperature, because the alkene route has the higher barrier.

THE POINT

Reproducing that ordering is the whole test of whether two barriers are defensible, and it is asserted in the test suite. Get the ordering wrong and the temperature response is backwards, however good the individual numbers look.

18.5 Evans–Polanyi: rates that respond to thermochemistry

One template handing the same barrier to every substrate it matches means the author, not the model, decides which of two competing products forms faster. The fix is an empirical linear relation between barrier and reaction enthalpy within a family:

$$E_{a,i} = E_a^\circ + \alpha\,\Delta H_i, \qquad \alpha \in [0, 1]$$

so a more exothermic member of the family is faster. ΔH_i is *computed* by the network from formation data, not declared.

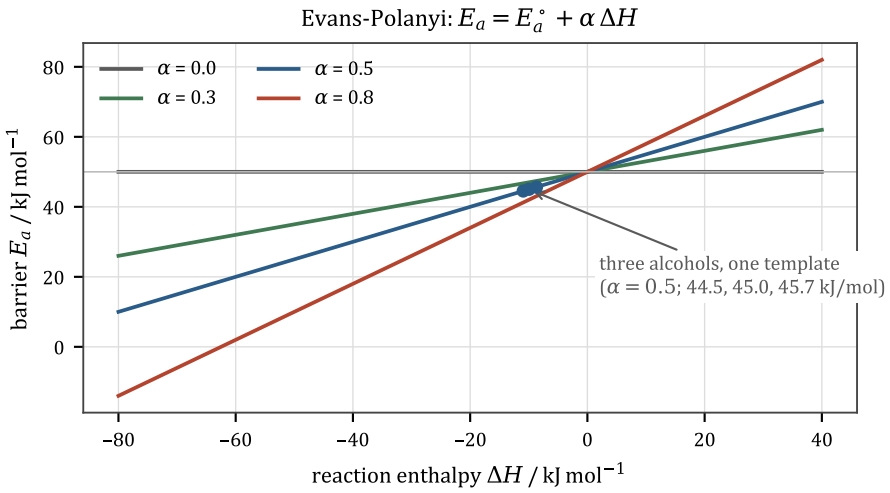

Figure 18.2: One template, three alcohols.

alcohol	ΔH / kJ mol^{-1}	E_a / J mol^{-1}
isopropanol	−10.94	44,532
methanol	−9.96	45,018
ethanol	−8.69	45,655

With $\alpha = 0$ all three get 50,000, which is the old behaviour exactly.

The reverse direction needs nothing extra: detailed balance gives $E_{a,\mathrm{rev}} = E_a - (1-\alpha)\Delta H$, which is the Evans–Polanyi relation for the reverse with transfer coefficient $1-\alpha$. It is still an empirical relation with a fitted α — it is just no longer a free parameter *per substrate*.

A POSITIVE ALPHA NAMES THE WRONG MAJOR PRODUCT WHEN KINETICS FIGHT THERMODYNAMICS

Measured in milestone S11 on the oxo process, where two templates race for the same substrate. α hands the lower barrier to the more exothermic route — which is the *thermodynamic* product — and real selectivity there is kinetic. So α is 0.0 on both hydroformylation templates, deliberately.

The general statement: **α is a claim that the barrier tracks the enthalpy, and that claim is false exactly where a reaction is under kinetic control.**

18.6 Hammett: what the substituents already on a ring do to the next step

Here is a specific, measured gap and its fix, and it is the best worked example in the project of choosing the *right* mechanism rather than the convenient one.

The gap. The nitration template gives one A and one E_a to every nitration on every substrate, so 2,4-dinitrotoluene nitrates exactly as fast as toluene. The consequence is not subtle: 1.0 mol of toluene and 3.5 mol of nitric acid reach **96% TNT in ten seconds at room temperature**, and the endpoint does not move with temperature at all — 300, 340 and 380 K all land on 1.0000 mol of trinitro. There is no stage to catch and nothing for an addition rate to control, and real TNT manufacture is a three-stage process with escalating acid strength and temperature.

The wrong fix, and why. Raise α. But α scales the barrier with the reaction *enthalpy*, and on this network the two point in opposite directions:

step	ΔH / kJ mol^{-1}
benzene → nitrobenzene	−141.2
nitrobenzene → 1,2-dinitrobenzene	−268.1

The **deactivated** ring's step is the *more* exothermic one, so any positive α makes the second nitration **faster** than the first — exactly backwards. `ReactionTemplate` refuses the two together for this reason.

The right fix. A substituent effect on an aromatic ring is an *electronic* property of the substrate, not a function of ΔH, and the tabulated quantity for exactly this is the **Hammett relation**:

$$\log_{10}\frac{k}{k_0} = \rho\sum\sigma \qquad \Longrightarrow \qquad \Delta E_a = -\ln(10)\,R\,T_{\mathrm{H}}\,\rho\sum\sigma$$

where σ is a property of the *substituent and its position* and ρ is a property of the *reaction* — so ρ is declared per template.

A RHO IS MEANINGLESS WITHOUT SAYING WHICH SIGMA SCALE IT WAS FITTED ON

This table is on σ^+ (Brown and Okamoto 1958), not on the ordinary aqueous σ, and that is not a detail. Electrophilic aromatic substitution builds positive charge on the ring in the transition state, so a resonance donor stabilises it far more than its ionisation-based σ says:

substituent	σ	σ^+
methoxy	−0.27	−0.778
amino	−0.66	−1.30
nitro (meta/para)	0.71 / 0.78	0.674 / 0.790

A ρ fitted against σ^+ applied to σ constants is two bases multiplied together — which is Chapter 12's fourth Benson trap wearing different clothes. Note that for electron *acceptors* the two scales nearly agree, because there is no lone pair to donate, which is what makes two proxy rows in the table tolerable.

AND THE REFERENCE TEMPERATURE IS 298.15 K, NOT THE NETWORK'S BUILD TEMPERATURE

σ^+ and ρ are tabulated from rate ratios measured at 25 °C, so 25 °C is the only temperature at which this conversion reproduces the number it came from. Using the network's own T_{ref} instead would make the same template give different barriers in networks built at different temperatures, with no measurement anywhere saying it should.

Ask what a fit was anchored on.

And the line saturates. A Hammett plot is a straight line fitted over a bounded abscissa — here, arenes with $|\sigma^+| < 0.4$. Extrapolating it to a trinitro ring is asking a linear fit for a number three times outside its range, and it produces barrier shifts of 90 kJ/mol.

WHY THE CLAMP IS NOT THE RATE CEILING

The obvious fix is the absolute rate ceiling of Chapter 5. It does not apply: that ceiling is derived for an *elementary* collision, and a Hammett-corrected nitration is not an elementary rate law — it is already a clamped, aggregated expression. Applying a collision-limit argument to it would be a category error.

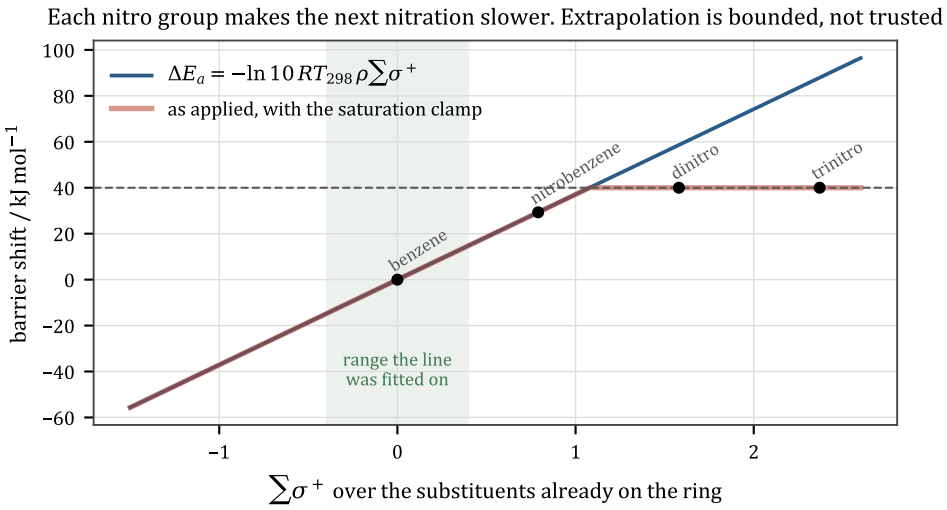

Figure 18.3: The line, and the clamp that bounds its extrapolation.

So the saturation is its own bound, justified by the fit's own range, and it came from a *second* literature source that supplies the lower bound. In the process it cost milestone G5 its headline coincidence, which the project recorded rather than quietly kept.

18.7 Catalysis, made explicit

A catalyst multiplies the rate and changes nothing else. A **homogeneous** catalyst is written into both sides of the SMARTS: the extra reactant slot raises its mass-action exponent to 1, and the identical product slot cancels it out of the stoichiometry. `builder.to_arrays` adds 1 to its `order` as a reactant and then cancels it in `delta`. Nothing in Layer 3 or Layer 4 needed a line of code.

WHAT IT DID NEED WAS HONESTY ABOUT THE PRE-EXPONENTIAL

An apparent rate is

$$\text{rate} = A_{\text{apparent}}\, e^{-E_a/RT}\, [\text{acid}][\text{alcohol}]$$

and an explicit one is

$$\text{rate} = A_{\text{intrinsic}}\, e^{-E_a/RT}\, [\text{acid}][\text{alcohol}][\text{H}_3\text{O}^+]$$

so the two agree at exactly one catalyst concentration, and $A_{\text{apparent}} = A_{\text{intrinsic}} \times [\text{H}_3\text{O}^+]_{\text{folded}}$. **That folded concentration was invisible for three sessions and is now declared** as `CATALYST_REFERENCE = 0.1` M. The catalysed and uncatalysed forms of each template are therefore the *same rate* at the reference loading — asserted in the tests — and away from it, "add more acid" is a real lever with the right slope.

The barrier does not change, and that is deliberate: E_a here has always been the *catalysed* apparent barrier, so re-declaring the catalyst does not license re-declaring the barrier.

A **heterogeneous** catalyst — iron in the Haber process — cannot be written that way, for two reasons that are both about representation:

- **a lattice is not a graph.** `[Fe]` has no bonds for a rewrite to match, so there is no SMARTS slot to put it in;
- **it is on a different basis.** A crystal lives in the solid block, which is an inventory in *mol*; every other exponent in this project is on a concentration in mol/L.

So it is *declared*, as `solid_catalyst`, and becomes a second exponent matrix.

WHAT IT IS NOT IS A NEW PHASE, AND THAT WAS MEASURED

Labelling $N_2 + 3H_2 \rightarrow 2NH_3$ a "solid"-phase reaction because iron is involved moves it off the ideal-gas standard state, because `reaction_deltas` applies the pure-liquid shift to anything that is not `"gas"`. Measured: ΔG **moves by** -99.7 **kJ/mol and** K **at 500 K by a factor of** 2.6×10^{10}**.**

The reaction is a gas-phase reaction. Every participant that *has* an activity is a gas, and a pure solid's activity is 1. The catalyst multiplies the rate and touches nothing else. `PHASE_INDEX` still has two entries.

And the same reference-charge bargain applies: `SOLID_CATALYST_REFERENCE = 0.1` mol, which is 5.6 g of iron — an ordinary bench charge — so that `ammonia_synthesis()` and `ammonia_synthesis(catalyst=None)` are the *same reaction* at that charge rather than two unrelated calibrations.

WHAT A SOLID CATALYST STILL DOES NOT DO

A real surface saturates: doubling the catalyst stops doubling the rate once the sites are full. That is a Langmuir–Hinshelwood form, and the kernel cannot express it — it evaluates $AT^n e^{-E_a/RT}$ times a product of powers, and nothing else. So the rate here is strictly first order in catalyst amount, for ever: **ten times the iron is ten times the rate at any loading.** Right at low coverage, wrong at high, and stated rather than approximated.

18.8 What the library refuses, and why refusals are content

Twenty templates were added in one milestone to make named historical routes runnable. Six candidate reaction *classes* were refused rather than credited, on the standard that **a reaction class is a mechanism claim, not an outcome**:

refused class	why
`fermentation`	glucose → acetone + butanol + ethanol + CO_2 + H_2 by *Clostridium*. A metabolic **network**, not a transformation.

refused class	why
`pyrolysis`	rows read "coal-marker → coal-tar-marker". Lumped decompositions of things with no molecular graph.
`isomerisation`	three mechanisms wearing one label.
`thermal-cracking`	a lumped product slate from a radical chain.
`catalytic-air-oxidation`	ranked third by routes unlocked; its four rows are at least three mechanisms.
`separation`	a distillation is not a reaction class, and the engine genuinely fractionates anyway.

A seventh, `catalytic-hydrogenation`, was **split** into five mechanism labels instead — because unlike the others, every one of its rows *is* a clean mechanism.

THE POINT

Crediting a class the engine cannot actually run makes the coverage audit *less* truthful, and the number it produces is then a description of the label rather than of the engine. Chapter 29 is about how much this discipline is worth.

CHAPTER 19

Layer 3: building a reaction network

19.1 Discovery, as a fixpoint

Give `build_network` a set of starting molecules and a set of templates. It:

1. matches each template's reactant patterns against the available species;
2. applies the graph rewrite;
3. canonicalises the products, registering any that are new;
4. **iterates to a fixpoint**, so that products can themselves react.

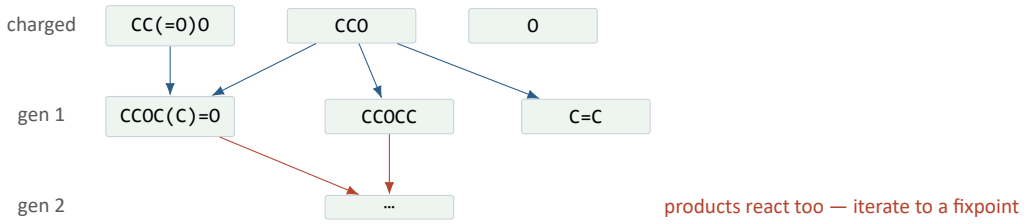

Figure 19.1: Discovery. Nothing here was enumerated in advance; every species below the top row was produced by applying a template to one above it.

Two guardrails run during this, and they have been there since the layer was written:

- **element and charge conservation.** A template that does not balance is *rejected*, not silently integrated (Chapter 2).
- **determinism.** Species indices come from dict insertion order and template application iterates lists, never sets. A network built twice from the same inputs is bit-identical, including row order. Chapter 23 depends on this completely.

19.2 Bounded discovery

Structural expansion enumerates every reachable species. For a polymerising feed — a diacid and a diol — that is an unbounded oligomer series: correct chemistry, fatal computation.

Two changes fix it:

- **incremental expansion.** Only combinations involving a newly-added species are tried. Re-running old pairs every round was quadratic waste.
- **`max_molar_mass`**, an explicit bound, with everything dropped **named in the log** — and a diagnosis attached saying that a growing series usually means the system polymerises, which species enumeration cannot represent properly.

The polyester case went from 24 s with a silent truncation to 0.04 s with an explicit report.

> **THE POINT**
>
> A network that silently omitted a pathway would be far more dangerous than one that is merely incomplete, because the omission would look like a chemical result.

19.3 The third bound, and the one that was silent

There is a third bound and for a long time it was the exception to the rule above. `generations=n` stops expansion after *n* rounds instead of running to a fixpoint. `max_species` reported when it bit, `max_molar_mass` reported and named what it dropped, a mixed standard state reported — and the generation limit broke out of the loop with a **non-empty frontier and said nothing.**

It is the strongest of the three claims, not the weakest. The other two concern species that were never *registered*; this one concerns species that **are in the flask** and whose onward chemistry was never looked for. If A + B makes C and C would go on to D, one generation shows C and never D — an approximation that changes the *contents* of a vessel rather than the completeness of a catalogue.

It matters because step-by-step play runs `generations=1` on every single step, and that is not a compromise but the mechanic: *what can the things in this flask do, once.* Measured across the full template library, five ordinary bench reagents explored two generations deep hit a 400-species cap in twelve seconds, while twelve reagents explored one deep cost under half a second.

So the limit now reports, in two forms:

- a **notice**, naming the count and the species left unexpanded; and
- `ReactionNetwork.unexpanded`, the same set as data, because a frontend asking *does this flask have more to give* needs an answer it can act on rather than a sentence it has to parse.

The frontier is taken on **either** exit from the expansion loop, which was a correction to the first version of this: the two bounds compete, and at `generations=2` the species cap bit first, so reading the frontier only on the generation branch reported an empty one for a 400-species network that had been truncated mid-round. Against a species cap the set is a lower bound and the cap's notice says so — the interrupted round left combinations untried as well.

> **THE POINT**
>
> A limit the player can see and lift is not an approximation; it is a choice. The same notices are carried on the network itself, because `print` is a channel a script reads and a windowed application does not have one.

19.4 Rate-based refinement, and why it lives at Layer 4.5

Structural discovery answers *what can form?* The question that actually matters is *what forms in meaningful amounts?* — and that is not a structural question at all. It depends on concentrations, on temperature, and on how long you wait.

`chemsim/discovery/refine.py` answers it the only way it can be answered: **by simulating.** The loop is the standard rate-based generation scheme:

1. take the current *core* species and build one generation outward;
2. integrate the core-only network from the actual feed concentrations;
3. with those concentrations, evaluate the rate of every *edge* reaction — the ones that would introduce a new species;
4. promote the edge species whose formation rate clears a threshold;
5. repeat until a round promotes nothing.

Step 2 is why this module sits at Layer 4.5 rather than inside `network`: refinement needs an integrator, and Layer 3 must not import Layer 4. Rather than invert the dependency for one call, **the layer that needs both sits above both.**

19.5 Where the thermodynamics gets imposed

Build time is also where thermodynamic consistency is enforced. For every reversible template, the reverse Arrhenius pair is derived here from the forward pair plus the reaction thermochemistry (Chapter 6):

$$A_{\text{rev}} = A_{\text{fwd}}\, e^{-\Delta S/R}, \qquad E_{a,\text{rev}} = E_{a,\text{fwd}} - \Delta H,$$

with the $T^{\Delta n}$ conversion landing in the modified-Arrhenius exponent n, and a floor at $E_{a,\text{rev}} \geq 0$ that prints a notice.

The reverse then enters the network as **an ordinary reaction**. That is the whole trick: Layer 4 stays a mass-action integrator with no concept of reversibility.

19.6 The output: `KineticArrays`

This is the clean numeric handoff — pure numpy plus a species-name list, no molecules, no RDKit. It is the seam a Rust kernel would sit behind.

array	shape	meaning
`delta`	(m, n)	stoichiometric matrix, integers
`order`	(m, n)	rate-law exponents, one row per reaction
`order_solid`	(m, n)	exponents on the *solid* block (a declared solid catalyst)

array	shape	meaning
A, Ea, n	$(m,)$	modified-Arrhenius parameters
phase	$(m,)$	which block's concentrations this reaction reads
species	$(n,)$	canonical SMILES, in index order

PHASE_INDEX is {"liquid": 0, "gas": 1} and **raises on anything else**, with a comment naming a solid-phase reaction as the case it was written to refuse loudly rather than swallow.

> **THAT REFUSAL HAS HELD TWICE, FOR TWO DIFFERENT REASONS**
>
> Both times the brief said to add PHASE_INDEX["solid"] = 2, and both times measurement said no:
>
> - **a reaction inside a crystal** cannot be written as mass action at all — the rate law comes out the wrong *shape*, not merely the wrong size (Chapter 21);
> - **a solid catalyst** would move its gas-phase reaction onto the pure-liquid standard state, worth 2.6×10^{10} in K at 500 K (Chapter 18).
>
> So both are *terms* rather than phases, and the two-entry index stands. A guard that has correctly refused two different attempts to widen it is a good guard.

19.7 What cell_potential does

build_network(cell_potential=...) is the one other knob. It multiplies each template's declared electrons into nFE joules of electrical work, which come off that reaction's Gibbs energy before K is computed and before detailed balance runs. A reaction whose chemistry costs less than the cell supplies then simply *has a favourable K*, and everything downstream is unchanged (Chapter 11).

19.8 Cost, and the one engineering consequence

Building a network is not free: 0.45 s for a four-species case, longer for a rich one. Since mixing two things in a sandbox means *building a network at charge time*, a UI that lets a player pour freely needs that cached and bounded or it stalls on every pour. That is noted in the project's own capability assessment as the single engineering consequence of "the player can mix anything".

CHAPTER 20

Layer 4: the numeric core

20.1 The isothermal case, which is the whole idea in six lines

```
rate_j(T, C) = k_j(T) * prod_i C_i ** order_ji
k_j(T)       = A_j * T**n_j * exp(-Ea_j / (R T))
dC/dt        = delta^T @ rate   (+ an optional source term)
```

That is `chemsim/numerics/integrator.py`. No `Molecule`, no RDKit, no unit objects — `KineticArrays` in, trajectory out. It is deliberately ignorant of chemistry, and that ignorance is exactly what makes it the clean swap point for a Rust/PyO3 kernel.

The optional `source(t, C) -> dC` hook is where higher layers inject non-reactive flux — a vessel's oxygen ingress, inter-phase transport, dosing from a dropping funnel — without the core needing to know what any of it means.

20.2 Solving it

The system is a set of coupled non-linear first-order ODEs. SciPy's `solve_ivp` integrates it, and the method is **BDF** — backward differentiation formulae, an implicit multistep method — for reasons that are Chapter 25's subject. The short version: chemical systems are *stiff*, meaning they contain timescales many orders of magnitude apart, and an explicit method's step size is bounded by the fastest one even after it has died away.

An implicit method needs the **Jacobian** $\partial f_i / \partial y_j$ at each step, and for a mass-action system that is analytically available:

$$\frac{\partial}{\partial C_k}\left(\Delta^{\mathsf{T}}\mathbf{r}\right)_i = \sum_j \Delta_{ji}\,\frac{\alpha_{jk}}{C_k}\,r_j .$$

In practice the project lets SciPy difference it numerically for the full vessel RHS, because the vessel's right-hand side (Chapter 21) contains phase-transfer terms, gates and an energy balance whose analytic derivatives would be a large and fragile piece of code. That decision has a cost, and Chapter 26 is entirely about it.

20.3 Sparsity: a measurement that came out backwards

A large reaction network's Jacobian is sparse — most species do not appear in most reactions — and SciPy accepts a `jac_sparsity` argument to exploit that.

PASSING JAC_SPARSITY TO A RIG WAS COSTING 10X

It was measured, and it was ten times *slower*. Two reasons:

- sparsity buys only **column groups**: SciPy differences several columns simultaneously when their non-zero patterns do not overlap. If the pattern does not partition well, you save nothing and pay the bookkeeping.
- **the temperature row was blocking all of them.** T appears in every rate constant, so the temperature column is dense — and one dense column prevents the grouping from finding any usable partition at all.

The general lesson is that a sparsity optimisation is a *measurement*, not a deduction, and a single dense row or column can destroy it.

20.4 Tolerances, and an audit whose instrument was wrong first

`rtol` and `atol` control the solver's local error. The project ran a full tolerance audit (milestone S2) across every example, sweeping from the default down to 10^{-8}, and it produced four findings, three of which are about methodology.

1. THE INSTRUMENT INVENTED A FINDING BEFORE IT WAS FIXED

The audit's first version reported a real-looking movement that was an artefact of how it re-ran examples. The instrument had to be audited before its findings could be.

2. "TIGHT IS FASTER" DOES NOT GENERALISE

There is a genuine effect where a tighter tolerance can be *faster*, because the solver stops rejecting steps. It was observed once and then over-generalised into a rule. Swept properly, it does not hold across examples.

3. A ONE-POINT TOLERANCE SWEEP CANNOT TELL NEWLY-BROKEN FROM ALREADY-BROKEN

If you only compare "default" against "tight", a number that moves might be a regression you just introduced or a pre-existing convergence problem. You need at least three points to see which.

The one *real* finding was a number that moved, and it was in the panel that exists to show exactly that.

20.5 Where the layer's boundary actually is

Layer 4 contains four things and nothing else:

module	what it is
`integrator.py`	the isothermal mass-action kernel
`vessel_integrator.py`	the full $4n$+1 flask RHS (Chapter 21)
`rig_integrator.py`	several coupled vessels as one system (Chapter 22)
`activity.py`	UNIFAC and Born evaluation, per RHS call
`lle.py`	the phase-split *decision*, called only between integrations
`jacobian.py`	one bound SciPy lacks (Chapter 26)

Every one of them takes numpy arrays and floats. None of them can name a chemical species except by integer index.

THE POINT

The clearest test of whether a layering claim is real: `numerics` could be handed to somebody who has never heard of chemistry, with the array shapes and the rate law, and they could reimplement it. That is what "the Rust seam" means in practice, and it is why the project can defer the Rust question indefinitely without the design rotting.

CHAPTER 21

Layer 5: the vessel, term by term

This is where the simulation stops being a reaction and starts being an *experiment*. It is also the largest single piece of physics in the project — `numerics/vessel_integrator.py` is 3,100 lines — so this chapter goes through it term by term.

21.1 The state vector

$$\mathbf{y} = \big[\ \underbrace{n_{L1}}_{n}\ \big|\ \underbrace{n_{L2}}_{n}\ \big|\ \underbrace{n_{G}}_{n}\ \big|\ \underbrace{n_{S}}_{n}\ \big|\ T\ \big], \qquad \text{length } 4n+1.$$

Four blocks — two liquid layers, the headspace, and the solid — plus one temperature, solved as **one stiff system**.

> **MOLES, NOT CONCENTRATIONS, AND ONE SYSTEM RATHER THAN SEVERAL**
>
> **Moles**, because concentration needs a volume and the liquid volume is itself a state-dependent quantity that shrinks as things boil off or crystallise out. Moles stay meaningful when the flask boils dry.
>
> **One system**, because the feedback loops are the entire point and operator-splitting them across separately-stepped subsystems would smear them and make the answer depend on the stepping interval:
>
> - an exothermic reaction heats the vessel;
> - the higher temperature accelerates it (Arrhenius) *but also lowers its equilibrium constant* (detailed balance);
> - heating raises the vapour pressure, so the solvent evaporates and latent heat pulls the temperature back down;
> - it also raises solubility, so a product that had crystallised redissolves.
>
> None of that is scripted. It is four terms in one right-hand side and a solver.

21.2 Where matter can go

Reaction runs in liquid 1, in liquid 2, in the headspace, inside a crystal, and at a crystal's surface. Transport moves matter between blocks without changing its identity, which is why element conservation is exact across all four.

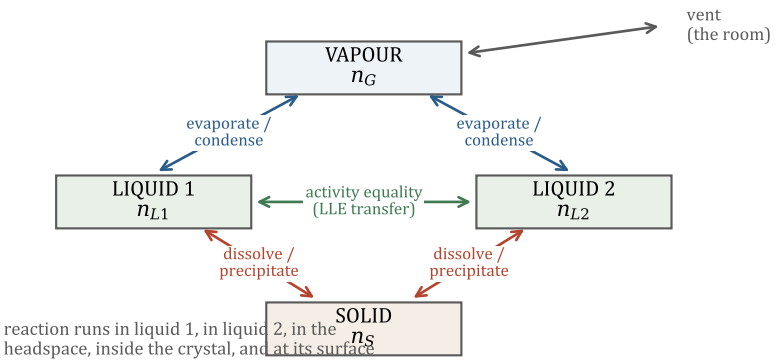

Figure 21.1: The four blocks and every path between them.

21.3 The terms

Here is the RHS assembly, essentially verbatim from the source:

```
return np.concatenate([
    dn_rxn1 - evap1 - evap_dry + solute1 - lle - precipitate,   # liquid 1
    dn_rxn2 - evap2              + solute2 + lle,               # liquid 2
    dn_gas_rxn + dn_gas_srxn + dn_gas_surf + evap - vent + ingress,   # vapour
    -solute + precipitate + dn_solid_rxn + dn_solid_surf,       # solid
    [dT],
])
```

Term by term:

`dn_rxn` — mass action in each liquid layer and in the gas, at that block's concentrations. Layer 2's contribution is multiplied by a gate that is zero when the second layer is empty.

`evap` — the phase-change flux, and it is one expression doing two jobs:

$$\dot{n}_i^{\text{evap}} = k_{la}\left(a_i\, P_i^{\text{sat}}(T) - p_i\right), \qquad a_i = x_i \gamma_i.$$

Positive means evaporating, negative means condensing. **Nothing in the code knows what a condenser is**: vapour arriving in a cold vessel finds $p > p_{\text{eq}}$ at that temperature, so this term runs backwards, the latent heat term changes sign and *releases* heat, and a thermal edge carries it to the coolant. That is Chapter 22's entire content.

`solute` — dissolution and crystallisation, driven by $(x_{\text{sat}}\, N - n_L)$ with x_{sat} from the fusion law divided by γ (Chapter 9). One term, two directions.

`precipitate` — ionic precipitation against a solubility product. A separate term from `solute` because a lattice is not priced by the fusion law and its ions are not priced on the same basis (Chapter 9).

`lle` — transfer between the two liquid layers, driving $\gamma_i^{\text{I}} x_i^{\text{I}} - \gamma_i^{\text{II}} x_i^{\text{II}}$ toward zero. The *decision* that a second layer should exist is made outside the RHS.

`vent` — gas leaving when $P > P_{\text{ambient}}$, carrying the headspace composition, with backflow from the room when the pressure drops.

`ingress` — the leaky-flask boundary flux. Contamination as a term rather than as a special case.

`dn_solid_rxn`, `dn_gas_srxn` — a reaction happening *inside* a crystal.

`dn_solid_surf`, `dn_gas_surf` — a gas reacting *at* a crystal's surface.

Those last two are separate, and the reason is instructive enough to have its own section below.

21.4 The energy balance

$$\frac{\mathrm{d}T}{\mathrm{d}t} = \frac{q_{\text{rxn}} + q_{\text{vap}} + q_{\text{fus}} + q_{\text{solid}} + q_{\text{surf}} + q_{\text{loss}} + q_{\text{vent}} + Q_{\text{input}}}{C_{p,\text{total}}}$$

with

$$C_{p,\text{total}} = C_{p,\text{vessel}} + (n_{L1} + n_{L2} + n_S) \cdot C_{p,\ell}(T) + n_G \cdot C_{p,g}(T),$$
$$q_{\text{loss}} = -UA\,(T - T_{\text{env}}), \qquad q_{\text{vap}} = -\dot{\mathbf{n}}^{\text{evap}} \cdot \Delta H_{\text{vap}}(T).$$

Every mechanic in Chapter 7 is in those two lines. The plateau is q_{vap} growing until it cancels Q_{input}; the dryout superheat is q_{vap} going to zero when there is nothing left to evaporate; the flask that boils dry and then *stops* climbing is q_{loss} catching up.

> **CROSSING BETWEEN LIQUID LAYERS IS ATHERMAL, DELIBERATELY**
>
> This project models no excess enthalpy of mixing anywhere — the energy balance is a sum of pure-component heat capacities — so charging a heat of mixing to this one transfer would be the only place it existed, and would disagree with the enthalpy every other transfer carries. Consistency beats a partial improvement.

21.5 The gates, and a bug that created matter

Several terms have to switch off smoothly as a phase disappears. Doing that badly is how you manufacture matter, and this project did.

> **THREE OVERLAPPING GATES ON ONE SCALE, AND A FLASK REPORTING 111% YIELD**
>
> The dryout gate was written as a ramp, $w = N/(N + \varepsilon)$, which is non-zero for **every** $N > 0$. So layer 1's evaporation at strength w and the dry-flask branch at strength $1 - w$ were **both live** inside the band. Worse, mole fractions were floored on the *same* scale, $x = n_L / \max(N, \varepsilon)$, so inside the band they summed to **less than one** — 0.57 at $N = 5.7 \times 10^{-7}$ — and every activity was understated as well.
>
> Measured on a sulfur burner, which walks into this because sulfur boils at 717.8 K and a burn near that holds only a trace of condensate:

T / K	liquid held / mol	oxygen created, relative
550	6.85×10^{-3}	1.8×10^{-12}
650	1.52×10^{-3}	1.1×10^{-9}
675	8.29×10^{-7} *(in band)*	2.3×10^{-3}
690	5.43×10^{-7} *(in band)*	1.1×10^{-1} **— reads 111% yield**

Three things came out of the fix and all three are general:

1. **The bug was the $0/0$ clamp sharing the gate's scale.** Two different guards, one constant.
2. **Disjoint gates are wrong here.** The obvious fix — make the two branches never overlap — creates a *dead zone* in which neither runs, and a dead zone stalls a condenser.
3. **A green test suite is no evidence the invariants table holds.** The suite passed throughout.

The scale that survived is 10^{-6} mol — 18 µg of water, far below anything a bench would call a pool, and *three decades above the solver's own* 10^{-9} *atol*, which is the gap that makes the transition resolvable at all. A constant's **units** are what make its value defensible.

WITH THE SECOND LIQUID BLOCK EMPTY, EVERY TERM REDUCES EXACTLY TO THE ONE-LIQUID RHS

Not approximately — term by term. `gate2` is zero, layer 2's volume is below the minimum so no reaction runs in it, its dissolution pool is zero, and the liquid–liquid flux carries the product of both gates.

That is deliberate and load-bearing: it is what lets a vessel that never splits reproduce every number the project has ever measured, so **a moved invariant means a real phase split and never an accounting change.**

21.6 A reaction inside a crystal

$\mathrm{CaCO_3(s) \rightarrow CaO(s) + CO_2(g)}$ is a lime kiln, and it is the first reaction in this project that neither the liquid block nor the gas block could write. Not a liquid-phase reaction, not a gas-phase one, and not a transport term either: **matter changes identity while staying a solid.**

The question was whether to add a third `PHASE_INDEX` entry or write a term. It was decided by arithmetic.

A pure solid has **unit activity**, so its equilibrium is a statement about the gas *alone*. Write the pair as mass action on the solid amounts and you get

$$k_f \, n(\mathrm{CaCO_3}) = k_r \, n(\mathrm{CaO}) \, p_{\mathrm{CO_2}} \quad \Longrightarrow \quad p_{\mathrm{CO_2}} = \frac{k_f}{k_r} \cdot \frac{n_A}{n_B}$$

which sweeps from infinity to zero as the charge converts. **That is not a perturbation of the right answer, it is a different shape of answer**: real calcite either decomposes completely ($p < K$) or not at all ($p > K$), and the mass-action form always stops somewhere in between.

And forward-only is not a way out, as Figure 9.2 showed. So the form used is an **affinity**:

$$\text{flux} = k(T)\left[\,\text{units}_{\text{fwd}} - \text{units}_{\text{rev}}\, e^{\,\ln Q - \ln K}\,\right] \text{ mol/s}$$

with $\ln Q$ summed over the *gas* participants only (solids contribute 1) and $\ln K$ from van 't Hoff.

21.7 A gas arriving at a crystal's surface

$2\,\text{ZnS(s)} + 3\,\text{O}_2\text{(g)} \rightarrow 2\,\text{ZnO(s)} + 2\,\text{SO}_2\text{(g)}$ is a roaster, and it is a *different* term.

> **THE LINE THAT HOLDS IS REVERSIBLE OR NOT**
>
> The affinity form is only a rate law while every gas participant is a **product**. Put a gas on the reactant side and its pressure lands in the *denominator* of Q, so an atmosphere depleted of it drives the reverse flux without bound — measured on a roasting declaration at 2.6×10^{15} formula units per second as $p_{\text{O}_2} \rightarrow 0$.
>
> So the two solid tables split on a rule the project already had:
>
> - **reversible, exponents forced** by detailed balance → `solid_state.py`;
> - **irreversible, orders declarable** → `surface.py`.
>
> which is the standing invariant *a declared rate order may never be reversible*, arriving as a module boundary.

The surface form is mass action, and its **basis is mixed**, which is the one thing that module must get right:

$$\text{rate} = k(T) \prod_{\text{solid}} n_{S,i}^{\alpha_i} \prod_{\text{gas}} C_i^{\alpha_i} \quad [\text{mol/s}]$$

- **a solid's "concentration" has no referent.** The solid block is an inventory in mol and its nominal volume is a convention. n_S/V would be a number divided by a convention.
- **a gas's amount is not what a surface sees.** The flux of molecules onto a crystal face goes with the collision rate, i.e. with **concentration**. Written on n_G instead, compressing the flask would not speed the reaction up — and a roaster is a machine for blowing air through a bed.

So the rate is *extensive* in the solid and *intensive* in the gas, and one consequence is a mechanic: with order 1 in the solid, $\tau = n/\text{rate} = 1/(k\,C_{\text{gas}})$ does **not** depend on how much ore is charged. Doubling the bed doubles the throughput and does not change the time.

A PHYSICALLY BETTER LAW, REFUSED

$n_S^{2/3}$ — the shrinking-core law, which accounts for a crystal's surface area falling as it is consumed — is physically better and is **refused**, because its slope at $n_S \to 0$ is infinite and that makes the solver's life impossible near the end of a burn. The refusal is recorded with the constant that would have been needed.

21.8 Two things that emerged from putting those two terms in the same flask

A ROUTE NOBODY DECLARED

Ore + coke + air → metal. Neither term declares that reaction. One term burns the coke to carbon monoxide; the other reduces the ore with it. Charge all three into one vessel and a *smelter* emerges from two independent declarations — and the project's coverage audit credited a catalog route that nothing in the code had ever named.

That is the strongest single piece of evidence for the emergent-design thesis in the repository.

And an equally instructive negative: a carrier-free lead chamber is now **inert** — both walls it found are closed — and a zinc retort evolves zinc *vapour* and condenses it in a cool receiver at 1180.15 K, which is a real Belgian retort's actual mechanic, **with no engine code changed at all.** What changed was one data entry: zinc moved out of the lattice table (where it could react and never boil) into the element table (where it can).

"A LATTICE MAY NEVER BOIL" WAS A STATEMENT ABOUT AN ENTRY

That sentence had been recorded as a fact about metals. It is a fact about how the *record* was filed. Zinc has a monatomic vapour, one condensed form and a measured sublimation curve, so it passes every test mercury was admitted on. Moving it took a route from thermodynamically-correct-but-static to a working distillation, for +0 on every coverage column and zero lines of engine code.

CHAPTER 22

Rigs: glassware as a graph

22.1 A vessel that can only see the room cannot reflux

Every piece of glassware that matters — condenser, still head, rotovap, Dean–Stark trap, dropping funnel, cold trap — is **two containers with something flowing between them**, and the flow is what the apparatus is *for*.

> **THE PHYSICS A CONDENSER NEEDS WAS ALREADY WRITTEN**
>
> Vapour arriving in a cold vessel finds $p > p_{\text{eq}}$ at that temperature, so the existing evaporation term runs backwards, q_{vap} changes sign and **releases** latent heat, and a thermal edge carries it to the coolant.
>
> Nothing in `vessel_integrator` had to learn about condensers. What was missing was only a way for two vessels to see each other.

So `Rig` adds exactly that, and "condenser" is not a class:

```
rig   = Rig()
flask = rig.add("flask",     Vessel(net, volume=1.0, Q_input=80.0))
cond  = rig.add("condenser", Vessel(net, volume=0.5, T_env=288.0, UA=40.0))
rig.vapour("flask", "condenser", k=2.0)   # vapour rises
rig.drain("condenser", "flask",  k=0.5)   # condensate runs back
rig.run(1800.0)
```

That is reflux. A condenser is a cold vessel with a vapour path in and a liquid path back — built the same way a flask is, differing only in its parameters and its edges.

22.2 One state vector, not several vessels stepped in turn

$$\mathbf{y} = \left[\, \text{vessel}_0\ (4n{+}1) \,\middle|\, \text{vessel}_1\ (4n{+}1) \,\middle|\, \cdots \,\middle|\, \text{vessel}_{m-1}\ (4n{+}1) \,\right]$$

> **THE POINT**
>
> Reflux is a **feedback loop** — boil, rise, condense, return, reboil — with latent heat coupling the two temperatures. Operator-splitting a loop like that across independently-stepped vessels smears it and, worse, makes the answer depend on the stepping interval. That is precisely the non-determinism Layer 6 exists to prevent.

So the rig solves one system and lets BDF resolve the loop.

Blocks are uniform because **every vessel in a rig shares one reaction network**, which Layer 5 already required for pouring one flask into another. The cost is that a condenser carries state for species it will never see; the benefit is that the coupled system is a uniform array rather than a ragged one.

Each vessel's own physics enters *unchanged*: `VesselIntegrator.make_rhs` already closes over a whole $4n+1$ state, so the rig calls it on a slice and adds edge terms on top. There is no second copy of the vessel RHS to keep in step with the first.

22.3 The four edges

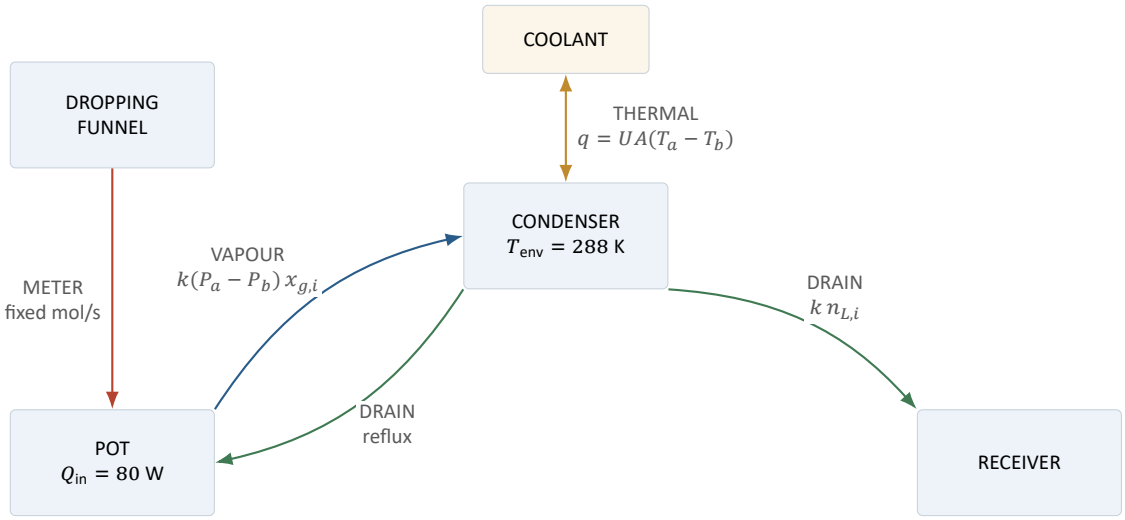

Figure 22.1: The whole vocabulary. Every apparatus in the project is some arrangement of these four edge types.

`VAPOUR` — bidirectional, pressure-driven, carrying the *donor's* headspace composition: $\dot{n}_i = k\,(P_a - P_b)\,x_{g,i}$. Venting to the room is the same law with a fixed far end, which is why the existing vent constant did not need generalising.

`DRAIN` — one-directional liquid, first order in the donor's holdup, $\dot{n}_i = k\,n_{L,i}$. This is a *drain*, not a level-driven flow: no geometry is modelled and none is implied. It is what returns condensate down a reflux column.

`THERMAL` — $q = UA\,(T_a - T_b)$. Jackets, coolant, a flask in a bath.

`METER` — one-directional liquid at a fixed molar rate: a dropping funnel or a syringe pump. The rate is a **parameter an event sets**, deliberately *not* a time window evaluated inside the RHS — a hard on/off in t is a discontinuity mid-solve, whereas an event at a step boundary is not.

After `rig.arrays()` nothing downstream knows what a condenser is.

22.4 What a still turns out to need

The physics of distillation worked immediately. The *protocol* did not, and the gap is worth stating because it is a general shape.

THE ENRICHMENT WASHES BACK OUT, AND THAT IS THE POINT

Measured on 2 mol ethanol + 2 mol water at 300 W:

t / s	pot T	head T	x(EtOH) in the head
200	302.43	300.62	**0.655**
600	303.63	299.57	0.618
1200	313.00	290.00	**0.500**

The enrichment is real and then it *disappears*, because everything comes over eventually and there was no way to stop and change the receiver.

Fractional distillation IS taking a cut, and the cut could not be expressed. Not merely unimplemented — unsayable: `World` had no rig at all, no `SWAP_RECEIVER` verb, and `wait_until` could only watch the vessel it was given while the head was not a vessel `World` knew about.

That was the single largest gap between "the physics is there" and "you can play it", and it was plumbing rather than science.

22.5 The plate column

Once cuts are expressible, a real fractionating column is a stack of vessels — each one a *plate*, with vapour rising and liquid falling. Eight plates at a reflux ratio of 5 reach **0.8544** mole fraction.

THE FIRST ATTEMPT'S DIAGNOSIS WAS WRONG, AND THE REAL CAUSE WAS PRESSURE

The first plate column separated poorly and the finding was recorded as a startup transient. It was not. **The still had no open end**, so it ran at 3–3.8 bar — and at 3.8 bar the entire vapour–liquid equilibrium is different.

Check the pressure before you check the model.

22.6 The dropping funnel

Adding a reagent *slowly* is one of the most important controls a preparative chemist has: it keeps an exotherm manageable by making the reaction addition-rate-limited rather than kinetics-limited.

THE MECHANIC ALREADY EXISTED; WHAT WAS MISSING WAS A WAY TO SAY IT

`METER` was already an edge type. What milestone G1 added was the ability to express a **conditional** drip — "add at 0.5 mmol/s until the pot reaches 340 K, then stop" — which needed a save-format version bump and nothing else.

And it turned up a real physical finding: **sensible heat alone cannot make an addition rate matter.** If the added reagent only has to be warmed up, the pot's thermal mass swamps it. The rate matters when the addition *drives a reaction* whose heat is large — which is the actual reason a dropping funnel exists.

Drip it too fast and the pot runs away. That is `examples/dropping_funnel.py`, and the yield of the route it belongs to moved **4.5**× on that change — with no species and no template touched. Chapter 30 is about why that matters for measuring what the simulator can do.

A RIG SINGULARITY THAT WAS A PUMP RUNNING DRY

The most recent numerics session was handed "the rig integrator goes singular" and expected a solver problem. It was a **METER edge pumping from a dry donor**: the funnel had emptied, the meter kept demanding a fixed molar rate, and the donor's composition — which is scale-invariant — reported back the solver's own probe size rather than anything physical.

Two lessons recorded: a composition is scale-invariant, so `num_jac` can end up measuring its own perturbation; and **an RHS is not only evaluated on its trajectory** — the solver probes states the physical system never visits, and every guard has to hold there too.

CHAPTER 23

Layer 6: the world, and what makes a run reproducible

The top of the stack, and the smallest layer in it. A `World` owns vessels and a clock, applies player events at step boundaries, and can write itself to a dict and read itself back. **It contains no chemistry**: every physical question is delegated downward.

It exists to guarantee three properties.

23.1 Headless

`step(dt)` is the entire interface. A real-time frontend calls it each frame, a batch experiment calls it in a loop, a test calls it once. None of them are privileged, and the engine has no opinion about rendering or wall-clock time.

23.2 Deterministic

> **THE POINT**
>
> A run is a pure function of **(scenario, script)**.

Player actions are **timestamped events** — the only thing that may mutate a vessel — and they fire strictly *between* integrations.

```
CHARGE   SET_HEAT   SET_ENVIRONMENT   SET_VENT   SET_STIRRING
SET_SHAKING   FILL_HEADSPACE   TRANSFER   FILTER   ...
```

An event is instantaneous relative to the chemistry; interleaving it with the solver's internal timesteps would make the outcome depend on the solver's adaptive step size, which is precisely the non-determinism this design exists to prevent.

Randomness — for future stochastic effects — comes from a single seeded generator that is *itself saved*, so a reload continues the same stream rather than starting a new one.

23.3 `wait_until`: the verb that makes a recipe a recipe

> **A REAL PROCEDURE HAS NO DURATIONS IN IT**

"Heat until it refluxes." "Cool until crystals appear." "Distil until the pot reaches 110 °C." "Stir until it all dissolves."

Not one of those is a time. A recipe written against fixed durations encodes the wrong shape into every screen built on top of it.

Every one of those is a **root of a function of the state**, located by `solve_ivp` to solver tolerance, so the instant is **discovered** rather than declared:

```
out = w.wait_until("pot",    boils(),                    timeout=7200.0)
out = w.wait_until("beaker", crystals("OC(=O)c1ccccc1"), timeout=14400.0)
out = w.wait_until("head",   temperature_steady(0.01),   timeout=3600.0)
out.elapsed     # how long it ACTUALLY took -- the clock moves by this
```

The sign convention is uniform and that is the whole contract:

$$f(\text{state}) < 0 \;\Rightarrow\; \text{not yet}; \qquad f(\text{state}) \geq 0 \;\Rightarrow\; \text{satisfied},$$

upward crossings only. Not cosmetic: with a direction flag per condition there are two ways to write each one and one of them is silently backwards, and "is it already true?" stops being a single comparison.

23.4 Four conditions that are not what they look like

Each condition was **sampled along a real trajectory before it was implemented** — the same discipline that killed crystal occlusion in Chapter 9 — and three of the four findings changed the vocabulary.

1. A DERIVATIVE APPROACHING ZERO IS NOT A ROOT

"The temperature has stabilised" is $\mathrm{d}T/\mathrm{d}t \to 0$, and it gets there **asymptotically**: approached and never crossed, so `dT/dt == 0` would wait forever.

What *is* a root is a **tolerance** on the derivative — "the thermometer has stopped moving" — which is what a chemist actually means and is a number a player can be given. Hence `temperature_steady(rate)` takes the rate, and there is no zero-derivative form at all.

2. AN AMOUNT THAT STARTS AT EXACTLY ZERO LEAVES ZERO RATHER THAN CROSSING IT

"Crystals appear" is n_S departing from exactly 0. At 10^{-9} mol the crossing is *inside* the solver's own atol. So the threshold is a **micromole** — three orders clear of atol and still far below anything a bench could see.

`SOLID_VISIBLE` is a **resolution limit, not a claim about nucleation**, and it is labelled as one.

3. A CONDITION ALREADY TRUE IS NOT A ROOT EITHER

SciPy locates *sign changes*. "Wait until it is above 300 K" asked of a flask at 340 K would hang. Layer 4 checks before integrating and reports it as already-satisfied.

4. A RATE TOLERANCE FIRES ON THE FIRST TRANSIENT, NOT ON THE PLATEAU

This one the probe was too coarse to see and a test caught instead. A flask whose headspace has just been filled evaporates hard for a moment: $\mathrm{d}T/\mathrm{d}t$ starts at -24 K/s, crosses zero inside a second, and only *then* climbs to its steady $+0.096$ K/s.

So `temperature_steady` on its own fires at 298 K rather than at the boil. The fix is not in the code: **say what you meant** — `boils()` first, `temperature_steady()` after — which is what a chemist does anyway.

23.5 The script, and why it stores conditions rather than instants

THE POINT

The determinism guarantee used to read "(scenario, **event list**)", and mending that sentence was the deliberate half of adding `wait_until`. An event is an *instant*; a wait is a *span whose end is discovered by a solver root.* The event list alone no longer says when anything happened.

The **script** is the ordered record of everything asked of the world — events scheduled, intervals stepped, conditions waited on — and it stores the **condition** and never the instant it resolved to.

The reason is a principle the project applies everywhere: *the instant is derived data, and this project declines to store derived data beside its source.* A `Scenario` keeps templates, not the network they generate; a script keeps "until it boils", not "at $t = 1183.7$ s".

23.6 Save and load

A save stores:

- the **scenario** — templates as SMARTS text, feed species, vessel config;
- the **script** — everything ever asked of the world;
- **moles and temperature**.

and never the derived network, which is rebuilt deterministically on load.

THE POINT

So a run is a pure function of (scenario, script), and `World.replay` re-runs one from its recipe alone. Saves are small, readable JSON. No RDKit object is ever serialised. The format is version-stamped and **refuses an incompatible reader rather than mis-mapping fields** — because

> the state vector will keep growing as lower layers gain phases, and a save written today must fail loudly against a future reader instead of silently shifting one block into another.

The version has been bumped for real, once, when the dropping funnel's conditional drip became expressible (Chapter 22).

23.7 What this buys, concretely

Because `(scenario, script)` determines everything:

- **a replay reproduces a run exactly**, including what the player saw;
- **a bug report is a JSON file**, not a description;
- **a test can assert on a trajectory** rather than on an endpoint;
- **the UI's recipe panel is the artefact**, growing as the player works, rather than something assembled at Save time (Chapter 24);
- **a run can be re-run at a different solver tolerance** to see whether any of its numbers were resolution rather than chemistry — which is exactly what the tolerance audit of Chapter 20 does.

CHAPTER 24

Layer 7: the window

`python -m chemsim.ui` opens a Tkinter window over a `World`.

The hard part of a frontend here is not layout and not chemistry. Layer 6 already guarantees a run is a pure function of (scenario, script), so a UI is an **event producer and a state renderer**. The genuinely hard part is that **an operation cannot be a blocking call**, and the reason is a measurement.

24.1 Cost is concentrated in stiff transients, not in elapsed simulated time

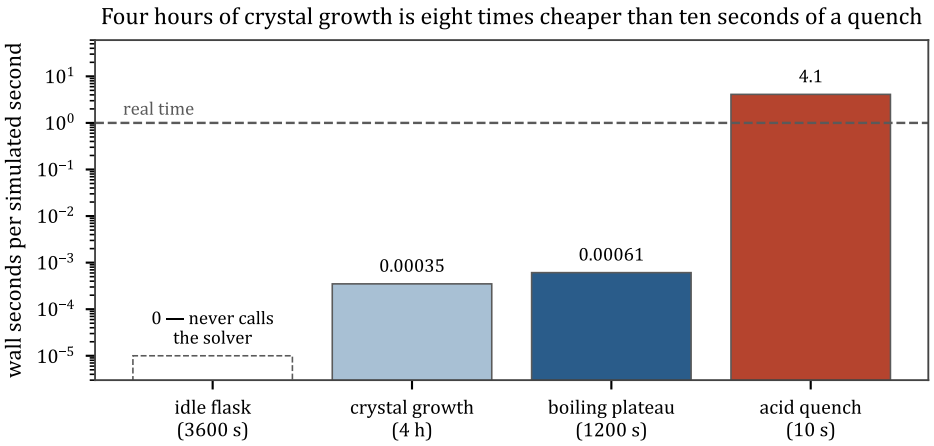

Figure 24.1: Four hours of crystal growth is eight times cheaper than ten seconds of a quench.

operation	simulated	wall	ratio
an idle flask	3600 s	0.00 s	never calls the solver at all
crystal growth	4 h	~5 s	$\sim 3 \times 10^{-4}$
a boiling plateau	1200 s	0.73 s	6×10^{-4}
an acid quench	**10 s**	**40 s**	**4.1**

THE POINT

The expensive moments are exactly the ones somebody is watching. A frontend that calls `world.step` on its own thread freezes precisely when the chemistry gets interesting.

That single constraint shapes everything in this layer, and each part of it is a decision.

24.2 One worker thread owns the `World`

Not the view, not a callback, not a "just read the temperature". Every command — including instantaneous ones like charging a reagent — goes through the same queue and is executed in submission order by that thread.

There is therefore **no lock around the engine at all**, and no possibility of reading a half-applied event. The only shared object is a `Snapshot`, which is immutable, and publishing one is a single attribute assignment.

24.3 Chunking is part of the recipe, not a rendering trick

A one-hour step runs as a sequence of shorter steps with a snapshot published after each, which is what lets a thermometer climb rather than teleport.

> **AND THAT IS NOT FREE, AND IT IS NOT HIDDEN**
>
> Freezing the layer permittivity at integration boundaries — an optimisation made elsewhere — made the caller's `dt` **weakly load-bearing**. Chunking therefore changes the answer slightly.
>
> So `World.script` records stepped intervals as well as waits, and a replay of what the player did reproduces what the player **saw**. The chunk size is offered as a visible setting for exactly that reason: it is a knob on the recipe, not on the graphics.
>
> A chopped `wait_until` is still a SciPy root, so chopping costs resolution nowhere.

24.4 Chunking bounds the update interval in *simulated* time, not wall time

Thirty simulated seconds of crystal growth is instant; thirty of the acid quench is two minutes. Nothing here can promise otherwise, so the module does not pretend to:

- it reports the wall time the current chunk has been running, and the measured cost ratio;
- `stop()` takes effect at the **next chunk boundary** rather than immediately.

A SciPy solve cannot be interrupted from outside, and claiming a cancel that does not cancel would be worse than an honest one that arrives late.

24.5 Three things on screen exist because of measurements

The cost meter — wall seconds per simulated second, live. This project's sharpest performance finding is that cost has nothing to do with elapsed simulated time, and a player who cannot see that will read a slow moment as a hung program.

The reports panel — `conservation_report`, `integrability_report`, `lle_report`, `electrolyte_report`, `solid_state_report`, `crust_report`, `holdup_report`, `surface_report`, and everything `build_network` said while discovering the reaction set. The engine's rule is that nothing is silently approximated, which is only worth anything if somebody is shown what it said.

> The refluxing rig destroyed 0.34 mol of its air for months, on a channel that was reported all along and that nothing read.

The builder's notices were on exactly such a channel until this panel took them: `print`, to a stdout a windowed application does not have. That is tolerable for a validation script and useless for a game, where a single step can generate hundreds of them — 397 for five bench reagents explored two generations deep, measured. They are now carried on the network, published in the `Snapshot`, and rendered here beneath the vessel's own reports, under a rule that took a scrollbar to keep honest: **showing the first seven of four hundred is the same failure as printing them where nobody looks.**

One of them is a fact about the flask rather than a note about it, so it is promoted out of the panel and into its heading: the count of species that were discovered and never reacted further (Chapter 19). *This flask has more to give* is the state a **react further** control exists to offer, and it must be legible without scrolling.

The recipe panel — the script, growing as the player works. A run is a pure function of (scenario, script), so this *is* the artefact, and it is visible rather than hidden behind a Save button.

24.6 A refusal is content, not an error dialog

Every guard in this project is written to name a cause and a fix: an overfilled vessel, a species with no Born radius, a solve that failed with a diagnosis attached. Those strings are the most useful thing the engine produces when something goes wrong, so they are carried on the snapshot and rendered — never reduced to "operation failed".

24.7 Why Tkinter

Because it is in the standard library. This project installs numpy, scipy and RDKit and nothing else it did not have to, and a browser stack for a desktop laboratory would be a server, a build step and a websocket to say what `root.after` already says.

24.8 The split that makes it testable

`chemsim/ui/session.py` is the half that can be wrong and has **no widgets in it**. `chemsim/ui/app.py` is the view, and **it never calls the engine** — not once, not for a temperature, not for a species list. It submits commands and reads `Session.snapshot()`.

`tests/test_ui.py` drives the session and never opens a window.

> **THE POINT**
>
> That is the same inversion as the two seams in Chapter 15, applied one more time: put everything that can be wrong on one side of an interface that can be driven headlessly, and leave nothing but rendering on the other.

PART IV

Numerics

CHAPTER 25

Stiffness, and why the solver is what it is

25.1 What stiffness is

A system of ODEs is **stiff** when it contains processes on timescales many orders of magnitude apart, and you care about the slow one.

A stiff system. The interesting answer is on the blue curve; the red one sets the step size

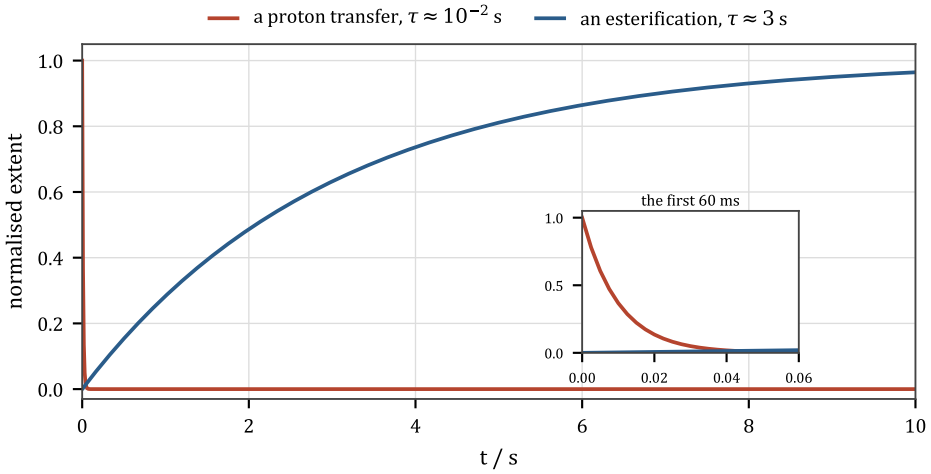

Figure 25.1: Two timescales in one flask.

Chemistry is stiff essentially always, and the reason is Chapter 5's exponential. Rate constants go as $e^{-E_a/RT}$; barriers in one flask range from ~0 (a proton transfer, which is diffusion-limited) to 150 kJ/mol (a bond being broken). At 300 K that is a spread of $e^{60} \approx 10^{26}$ in rate.

IF YOU KNOW PHYSICS

Formally, stiffness is about the eigenvalues of the Jacobian $J = \partial f/\partial y$. The stiffness ratio is $\max_i |\mathrm{Re}\,\lambda_i| / \min_i |\mathrm{Re}\,\lambda_i|$, and it is the ratio of the fastest decaying mode to the slowest.

An **explicit** method's stable step is set by the fastest mode, $h \lesssim 2/|\lambda_{\max}|$, *whether or not that mode is still doing anything*. So an acid–base equilibrium that reaches steady state in a microsecond forces microsecond steps for the whole hour you want to simulate. That is 3.6×10^9 steps to watch an esterification.

25.2 The fix: implicit methods

An implicit method evaluates the right-hand side at the *end* of the step, $\mathbf{y}_{k+1} = \mathbf{y}_k + h\,f(t_{k+1}, \mathbf{y}_{k+1})$, which requires solving a non-linear system at every step but is stable for arbitrarily large h on decaying modes. Fast modes that have already relaxed to their quasi-steady state stop costing anything.

The project uses SciPy's **BDF** — backward differentiation formulae, variable order 1–5, variable step. That is the standard choice for chemical kinetics and the reason the numbers in Chapter 24's table are as small as they are.

25.3 What that costs: the Jacobian

Solving the implicit step means Newton iteration, which needs $J = \partial f / \partial y$ — an $(4n+1)\times(4n+1)$ matrix for a single vessel, and m times that for a rig.

The project lets SciPy difference it numerically rather than supplying an analytic one. That is a real trade: an analytic Jacobian for the pure mass-action kernel is easy, but the *vessel's* RHS contains evaporation gates, dryout gates, dissolution limiters, a liquid–liquid transfer, a vent with backflow, an activity model and an energy balance — and an analytic derivative of all that would be a large, fragile piece of code that would have to be kept in step with every term added.

Chapter 26 is about what that choice cost, and it is the most interesting numerical story in the project.

25.4 Where the time actually goes

Three observations from the project's measurements, all of them slightly counterintuitive:

1. An idle flask is free. `VesselIntegrator.run` short-circuits when nothing is happening, and never calls the solver at all. An hour costs 0.00 s.

2. Cost tracks *events*, not duration. A boiling plateau is cheap because after the transient it is a smooth quasi-steady state. An acid quench is expensive because it is a genuine fast transient with real structure in it.

3. The RHS is dominated by fixed overhead at these sizes. The activity kernel is *flat* from 4 species to 25 (78 µs to 87 µs). Both numbers are numpy dispatch on small arrays, not arithmetic — which is why the case for a Rust kernel rests on per-call overhead rather than on any model being slow (Chapter 15).

25.5 Tolerances, and what a number means

`rtol` and `atol` bound the solver's *local* error per step. They do not bound the global error and they certainly do not bound the model error — which, given Chapter 12, is several kJ/mol in ΔG_f and therefore a factor of 2–4 in every equilibrium constant.

SO: HOW MANY DIGITS OF A SIMULATED NUMBER MEAN ANYTHING?

The honest answer here has three levels, and they are far apart:

- **solver resolution**: many digits. The tolerance audit swept every example at rtol $= 10^{-8}$ and most numbers do not move.
- **rate parameters**: one digit, generously. Pre-exponentials are order-of-magnitude choices, so *times* are order-of-magnitude.
- **equilibrium positions**: a factor of 2–4, dominated by group-contribution formation data.

A branching *ratio* between two templates in the same family is better than any of those, because the shared errors cancel. That is why the project's strongest claims are comparative — "ether above 480 K, ester below" — rather than absolute.

A ROOT IS ZERO TO SOLVER PRECISION, AND THAT IS NOT THE SAME AS ZERO

`wait_until` locates a root of a scalar function of state. What it returns is a state where that function is zero **to solver precision** — which for a condition like "the crystals have appeared" means the amount is at the threshold, not comfortably past it.

Anything that reads the state immediately after a wait has to tolerate that. It is the same class of issue as the micromole threshold in Chapter 23: a numerical tolerance masquerading as a physical statement, and the fix is always to make the physical statement explicit.

A DOCUMENTED TRAP THAT RESTED ON A WRONG BOILING POINT

One of the project's recorded numerical traps turned out to be visible only *because* a species had a wrong T_b. When milestone S13 fixed the boiling-point table, the trap stopped reproducing.

The finding was real; the reason recorded beside it was not. That pattern — "half the reason written down beside a refusal was about something the code never did" — has now happened three times in this project, and it is the argument for re-measuring a recorded refusal rather than trusting it.

CHAPTER 26

The Jacobian bound, which is one inequality and a long argument

This chapter is about a single line of code. It is here because the reasoning that produced it is the best worked example in the project of *how a numerical bug is actually found*, and because the shape of the answer — a bound derived from the state itself rather than a constant somebody chose — is transferable.

26.1 The failure

BDF is handed a NaN Jacobian. The LU factorisation fails with "array must not contain infs or NaNs", several tens of seconds after anything actually went wrong.

26.2 The mechanism

SciPy's `num_jac` differences column j at a step $h = \texttt{factor}_j \cdot \max(\text{atol}, |y_j|)$. When the difference it gets back is small compared with the rates elsewhere in the system, it concludes it probed too gently and **multiplies that column's `factor` by ten** — on every Jacobian, for ever.

SciPy *floors* `factor` at a minimum and **never ceilings it**. So a column it cannot difference is probed harder without bound until `factor` overflows to `inf`, h becomes `nan`, and the Jacobian is poisoned.

26.3 Why the column could not be differenced

This is the part worth reading.

The column that overflows is the **second liquid layer's SO_2**, holding 8.21×10^{-29} mol. Its f is the layer-reabsorption drain, which is strictly *negative* for any positive holding — so `num_jac` takes the sign as -1 and steps **downward**, straight into the RHS's own `np.maximum(y, 0.0)` clamp.

Every downward step, at every size, lands on the same clamped state:

h	-2.2×10^{-24}	-2.2×10^{-19}	-2.2×10^{-14}	-2.2×10^{-9}	-2.2×10^{-4}	-2.2×10^{6}
max |diff|	8.84e-29	8.84e-29	8.84e-29	8.84e-29	8.84e-29	8.84e-29

Constant over **thirty decades of step size**, against a scale of 8.37×10^{-14} taken from another species' row. So "the difference is too small" is true no matter what, the factor climbs a decade

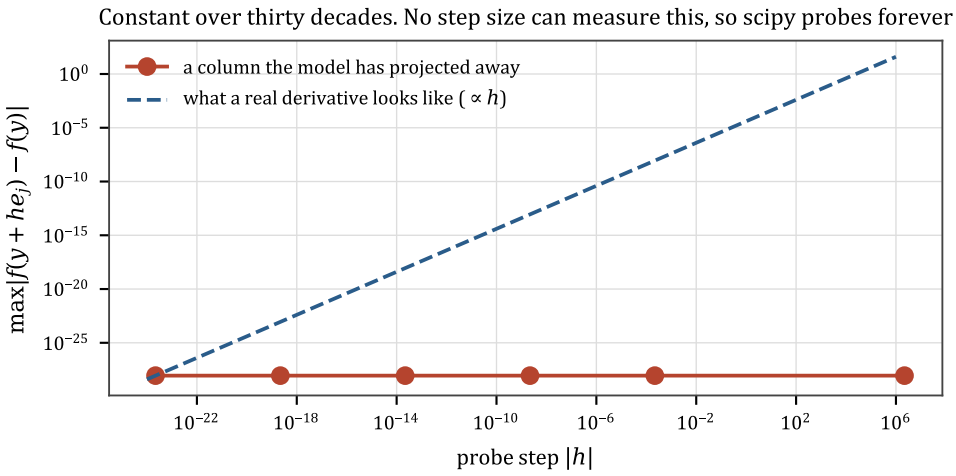

Figure 26.1: Constant over thirty decades. No step size can measure this.

per Jacobian — twenty-eight of those calls at one unchanged state — and about two hundred later it reads 2.220×10^{307}.

THE POINT

No step size can measure a derivative the model has deliberately projected away.

Probing harder is not always a question that can be answered, and a loop that assumes it is will run until it overflows.

26.4 Four wrong answers on the way to the right one

This bug had been met **three times** before and named three different ways — a vessel at rest, an empty second liquid layer, a sealed flask with no headspace — and worked around three times, each time *in the chemistry*.

1. THE FIX WAS SCHEDULED FOR THE WRONG BLOCK

It was planned as an "honest diagonal" on the **gas** block, on the reasoning that an absent gas species has a flat column. That is true — the gas block *is* a route in. But of the five recorded triggers, **the only one that still reproduces overflows in liquid layer 2**, and a diagonal on the gas block could not have reached it.

2. THE NAMED PRECEDENT WAS THE CAUSE

The fix was to copy the layer-reabsorption drain's approach. That drain is **what makes f negative** and so points the probe at the clamp in the first place. The precedent to copy was the thing to remove.

3. THE FIRST BOUND WAS WRONG, AND THE EXAMPLES SAID SO

The obvious cap is `factor` ≤ 1, on the reading that `factor` is the step as a fraction of the variable's own scale, so 1 moves the variable by all of itself.

That reading is false exactly where it matters. The scale is $\max(\text{atol}, |y_j|)$, so for a species at or below atol the fraction is of **atol**, not of the variable. `factor = 149` on an absent species is a 1.5×10^{-7} mol probe of a 0.1 mol flask, which is a perfectly good probe.

Measured on a real example:

ceiling	SO_2 in the flask	what the solver wanted
`inf`	0.000201 mol	1.490×10^2
10^6	0.000201 mol	1.490×10^2
10^2	0.000201 mol	1.490×10^2
1.0	**0.000197 mol**	clamped — **and the answer moved**

A ceiling of 1.0 moved a quotable digit in a *healthy* run, and eight of the sixteen examples moved under it.

4. FOUR OF THE FIVE RECORDED TRIGGERS DO NOT REPRODUCE

They had been fixed in the chemistry, or the conditions had changed. Re-measuring the list before acting on it is what located the one that was live.

26.5 The bound that survived

So the bound cannot be on `factor`. It has to be on the **step**, and the honest statement is:

THE POINT

A difference quotient is a derivative of *this* system only while the probe stays inside it. You cannot learn anything about a state by moving one of its components further than the whole state extends.

$$|h_j| \leq \max_i |y_i| \qquad \Leftrightarrow \qquad \texttt{factor}_j \leq \frac{\max_i |y_i|}{\max(\text{atol}, |y_j|)}$$

A bound per column and per call, computed from the state, **with no constant in it at all.**

On a single vessel it never binds: the largest factor any single-vessel example asks for is 1.49×10^9, against a bound of order 10^{11}–10^{12} for a state carrying a temperature at atol $= 10^{-9}$. Where it

binds *by design* is the runaway itself — the sulfur burner's frozen column reaches the bound at 6.9×10^{13} (690 K over atol $= 10^{-11}$) and stops there instead of at `inf`.

26.6 Where the fix belongs

Not in the chemistry. `chemsim/numerics/jacobian.py` sees a callable and two float arrays, like everything else in Layer 4, and the whole module is that inequality plus the argument for it.

> **THE GENERAL SHAPE**
>
> Three times, a numerical failure was worked around by changing a physical model — adding a drain, adding a short-circuit, avoiding a flask configuration. Each workaround was locally reasonable and each one made the *next* occurrence harder to find, because it moved the symptom rather than the cause.
>
> The bound belongs in the layer that owns the differencing. Ask, when a physical model grows a term whose only job is to keep a solver happy, whether the fix is in the wrong layer.

26.7 What it does not fix, stated

The bound stops the overflow. It does not make the column's derivative *correct* — it is still zero, because the model really has projected that direction away. What it buys is that BDF gets a finite, honest zero instead of a NaN, and the run continues with the same answer it would have had.

The species in question is at 10^{-29} mol. If it ever mattered, the model would be wrong for a different reason.

CHAPTER 27

Conservation: what holds exactly, and the one thing that does not

27.1 Matter is exact everywhere

Chapter 2 set this up: because $\Delta E = 0$ holds in *integers*, the quantity $\mathbf{n}^{\mathsf{T}} E$ — moles of each element, plus total charge — is conserved by the reaction terms to machine precision, not to solver tolerance.

Transport terms move moles between blocks of the state vector without changing species identity, so they conserve the same quantity trivially. Therefore:

> **THE POINT**
>
> A sealed vessel conserves every element **across all four phase blocks**, and a rig conserves them across every vessel in it. `conservation_report()` measures this on demand, and the test suite asserts it.

The guardrail that makes this true is upstream, at network build: a template that does not balance is rejected rather than integrated. Everything after that is bookkeeping.

27.2 Energy is not

> **AN INSULATED FLASK DESTROYED 495 J AFTER A PRECIPITATION EVENT, AND CONSERVATION_REPORT COULD NOT SEE IT**
>
> This is milestone M12 and it is the most important known limitation in the numerics.
>
> `conservation_report` checks **matter**. There is no equivalent invariant on energy, because energy is not a linear function of the state the way element counts are — it involves $C_p(T)$ integrals, latent heats, heats of reaction and boundary fluxes, and the RHS assembles $\mathrm{d}T/\mathrm{d}t$ from those rather than tracking a conserved total.
>
> So a leak is invisible to the instrument that exists to catch leaks.

The M12 investigation is worth following because of how the wrong answers were eliminated.

Four candidate causes were measured and refuted: the precipitation term's enthalpy accounting, the dryout gates, the vent, and the activity model's temperature dependence. Each was instrumented and each was clean.

The actual cause was a derived rate constant 9.4×10^7 times the collision limit. Not one somebody typed — one that came out of detailed balance from a forward rate that was itself only order-of-magnitude. At that speed the transient is faster than anything the solver can resolve, and the energy balance loses the heat of a reaction it never took a step inside.

The fix must scale both pre-exponentials. Capping only the offending direction would change k_f/k_r, i.e. change K — a thermodynamic falsehood introduced to fix a numerical problem. So the cap scales the pair.

THE POINT

The general form: **an unphysical rate constant is a numerical bug wearing a chemistry costume**, and it will show up somewhere other than where it is. Here it showed up as an energy leak, several layers away from the reaction that caused it.

`validation/rate_ceiling.py` now audits every network in the project, including the reverse that each reversible template implies.

27.3 Two things about reading the energy report

REPORT THE GROSS HEAT BESIDE THE NET

`q_rxn` for a fast reversible pair is a *difference of two large terms*. The project measured one case reading -4.69×10^6 W frozen at one state and -5×10^{-3} W frozen at another — nine orders apart, for the same physical situation, because the two directions nearly cancel and the residue depends on exactly where you froze it.

Reporting only the net makes a catastrophically ill-conditioned number look like a small one. The gross terms go beside it.

ENERGY_TERMS LIES UNLESS IT IS GIVEN THE RUN'S OWN BOUNDARY STATE

The energy report needs to know what the boundary was doing — ambient temperature, heat input, vent state — and it will happily compute a plausible-looking balance against *default* boundary conditions that the run never had. The interface now requires the run's own state to be passed in.

27.4 A leak whose driver was a blended composition

One more, because the mechanism is subtle and generalisable.

THE VENT LEAK, AND A DONOR COMPOSITION THAT WAS AN AVERAGE

A refluxing rig destroyed 0.34 mol of its air over a long run. The vent term carries the *donor's* headspace composition, $\dot{n}_i = k(P_a - P_b)x_{g,i}$ — and where the flow could reverse within a step, the composition being carried was a **blend** of the two ends rather than whichever end was actually donating.

A blended composition is not a small error in the flux; it is matter of the wrong *identity* crossing the boundary. The fix is that the flux carries a one-sided composition chosen by the sign of the driving force, which is the same algebraic shape as the fix in Chapter 21 that made the reversible solid–gas term work: **write a two-sided expression as two one-sided products, so that nothing is ever divided by — or averaged across — a quantity that can change sign.**

27.5 What this means for reading a result

THE POINT

- **Element counts and charge**: trust them completely. They are exact and they are checked.
- **Volumes and mole fractions**: trust them to solver tolerance.
- **Temperature trajectories**: trust the *shape* — plateaux, runaways, crossovers — and treat the absolute energy budget as approximate unless `energy_report` has been read with the run's own boundary state.
- **Any number from a run in which the rate-ceiling audit reports a capped reaction**: read the report first.

And the standing rule the project applies to all of this, from the dryout-band finding in Chapter 21:

> A green test suite is no evidence that the invariants table holds.

The suite passed throughout the 111%-yield bug, throughout the 495 J leak, and throughout the months of vented air. Each was found by an audit written to ask one specific question, and each audit is now a file in `validation/`.

PART V

What comes out

CHAPTER 28

Behaviour nobody wrote down

The whole point of the design is that things happen which nobody coded. This chapter is a catalogue of them, in roughly the order they were discovered, with the mechanism named in each case. Every one has been measured.

28.1 A liquid pins at its boiling point

What happens. Ethanol under a hotplate holds **351.46 K** for as long as there is liquid, then rockets.

Why. Evaporation runs away when $\sum_i p_i$ reaches ambient; latent heat absorbs the input. There is no boiling point anywhere in the code (Chapter 7).

28.2 The boil-off rate is $(Q - \text{losses})/\Delta H_{\text{vap}}$

Falls straight out of the energy balance. Asserted in the tests, written down nowhere.

28.3 A flask boiled dry superheats

The plateau lasts exactly as long as there is liquid, not one second longer.

28.4 An insulated exotherm gets *less* product

Why. Self-heating raises T; T lowers K via detailed balance for an exothermic reaction. Le Chatelier arriving from a heat-transfer coefficient (Chapters 4 and 6).

28.5 50/50 ethanol/water gives 71% ethanol vapour

Raoult alone. Distillation with no separation model.

28.6 Air-saturated water holds 0.28 mM oxygen

Henry's law through the *same array* as Raoult. Measured value ≈ 0.27 mM.

28.7 There is an azeotrope at $x = 0.899$, 351.17 K

No azeotrope table. It is simply where $y = x$, and it exists because γ bends the equilibrium line across the diagonal (Chapter 8). Reference: 0.894 and 351.3 K.

28.8 Which product forms depends on temperature

One flask, one charge, three templates, one hour:

T / K	ethyl acetate	diethyl ether	ethene
320	1.476	0.000	0.000
400	1.408	0.023	0.00003
480	0.421	0.751	0.017

Nobody wrote “if hot, make ether”. The barriers differ, so the branching does.

28.9 A catalytic cycle turns over, and can be lost

The lead chamber process runs a genuine NOx catalytic cycle: **80.3 turnovers on a 0.5 mmol charge**, watchable and losable. Not a rate multiplier — a species that is consumed and regenerated, with its own inventory that a badly-run process can destroy.

And a **carrier-free** chamber is now inert. Both walls it found are closed, which is a way of saying the mechanic is real rather than incidental.

28.10 A cooling solution crops crystals

Benzoic acid under water, nothing declared:

T / K	dissolved	solid
330.0	0.050000	0.000000
298.1	0.026681	0.023319
275.0	0.012236	0.037764

The fusion law against γ (Chapter 9). Filtration, cake porosity and the crystal-crust loss are built on top of it.

28.11 Toluene and water separate, and steam-distil

Two layers and a boiling point at **358.31 K** — below *both* components — without a single reaction. “Nothing happens” is not the same as “the flask is inert.”

28.12 A kiln has a threshold temperature

Calcite does not calcine below about 1120 K under 1 bar of air, and sweeping the CO_2 away makes it go lower. Both are consequences of a pure solid having unit activity (Chapters 9 and 21).

28.13 A gas-shift reaction peaks and falls away

The water-gas shift peaks at **620 K** and declines above it — exothermic, so K falls with temperature while the rate rises. The reformer beside it is inert until **900 K**. Deacon's ceiling and its rate cross near **650 K**.

Three different shapes, from three sets of $(\Delta H, \Delta S, E_a)$ and one integrator.

28.14 A Claus flask recovers 100.0% of its sulfur at exactly the stoichiometric air rate

Why: burning a third of the feed is what leaves the 2:1 ratio the second template wants. Two templates, one flask, and an optimum nobody declared.

28.15 A smelter emerges from two independent declarations

Ore + coke + air → metal. One term burns the coke to carbon monoxide; a different term reduces the ore with it. **Neither declares the route**, and the coverage audit credited a catalog route that nothing in the code had named (Chapter 21).

28.16 A zinc retort distils its product

Zinc evolves as a vapour and condenses in a cool receiver at **1180.15 K**, which is a real Belgian retort's actual mechanic — from moving one data entry between two tables, with no engine code changed.

28.17 Two templates racing cross from kinetic to thermodynamic control

The oxo process's two products swap ranks at a computable temperature. The crossing is a prediction of the two barriers and two prefactors, not a declaration (Chapter 5).

28.18 An electrolysis cell has a decomposition potential

1.441 V for water (book: 1.229) and **2.362 V for brine** (book: 2.186), out of formation data alone. Nothing declares a threshold; it is where nFE overtakes ΔG_{chem} (Chapter 11).

28.19 A brine cell makes chlorine rather than oxygen

Thermodynamics prefers oxygen. Kinetics — the two activation overpotentials, declared as barriers — decides otherwise. This is the industrial fact that makes the chlor-alkali process possible, reproduced from two numbers.

28.20 An oxidant that becomes one of its own reagents

In the Skraup synthesis the oxidant's *reduction product* is itself a substrate for the reaction, so the system feeds itself. The network found that; nobody wrote it.

28.21 An open flask loses 98% of the yield

Same Skraup preparation, vented instead of sealed. The volatile intermediate leaves. That is a real preparative fact and it arrives here as a boundary flux.

28.22 Drip too fast and the pot runs away

The dropping funnel (Chapter 22). And the negative result beside it: **sensible heat alone cannot make an addition rate matter** — the rate matters when the addition drives a reaction whose heat is large.

28.23 Nitration stages

With ring deactivation, a nitration becomes a *process* rather than an event: each nitro group makes the next substitution slower, which is why real TNT manufacture escalates its acid strength and temperature through three stages (Chapter 18). Before that, one flask reached 96% trinitro in ten seconds at room temperature and the endpoint did not move with temperature at all.

28.24 An inert spectator moves a yield

Found in milestone C5. A species that participates in no reaction still changes the answer — it dilutes, it carries heat capacity, it shifts mole fractions and therefore activities and therefore partial pressures. “Inert” is a statement about chemistry, not about the state vector.

> **THE PATTERN**
>
> Look at what those twenty-four have in common. Almost none of them came from adding a *feature*. They came from putting two existing terms in the same flask and integrating.
>
> That is the return on the design decision in Chapter 1 — and it is also why the project’s coverage work (Chapter 29) is about *templates and data* rather than about engine capability. The engine has been finished, in a real sense, for a long time; what is short is chemistry to put in it.

28.25 And two that went the other way

Not everything emergent is welcome, and both of these were caught by audits rather than by anything failing:

- **a flask reported 111% yield**, by manufacturing oxygen inside a dryout band three overlapping gates all thought they owned (Chapter 21);
- **an insulated flask destroyed 495 J**, because a derived rate constant was 9.4×10^{7} times the collision limit (Chapter 27).

Both are exactly as emergent as the twenty-four above. That is the cost of the design, and it is why Part IV exists.

CHAPTER 29

Measuring how much chemistry this covers

29.1 Why there is a corpus

"How much chemistry does this simulator cover?" is the question that decides what to build next, and it is very easy to answer by feel and be wrong.

So `data/catalog/` holds a hand-authored corpus: **1,583 compounds and 173 named synthetic routes over 377 steps**, from the lime cycle and Tyrian purple through the lead chamber and Leblanc to the Hock process, SOHIO ammoxidation and PLA.

THE POINT

It is **data, not code**. Nothing in `src/chemsim` imports it. It exists to be pointed at the simulator, and the numbers it produces are deliberately unflattering.

```
python tools/catalog.py                 # structural validation
python tools/build_route_index.py     # feedstocks -> intermediates -> products
python validation/catalog_coverage.py  # the audit
python tools/build_playable.py         # the tech tree (runs things; ~1 min)
```

29.2 Three reports that answer three different questions

AND THEY ARE ROUTINELY CONFUSED FOR ONE

report	asks	is a property of
`COVERAGE_REPORT.md`	*can the engine do this chemistry?*	the **engine**
`ROUTE_INDEX.md`	*what is a feedstock here?*	the **corpus**
`PLAYABLE.md`	*can a player get to it from a rock?*	**neither**

The third is neither, because a route's yield was measured moving **4.5×** on a change that touched no species and no template.

A route can be fully covered, fully indexed, and unreachable.

29.3 Three readiness tests, and the one to quote

A route is judged on three independent questions:

- **species-ready** — does every species in it resolve to a full property set?
- **template-ready** — is there a template for every reaction class it uses?
- **BOTH** — the intersection.

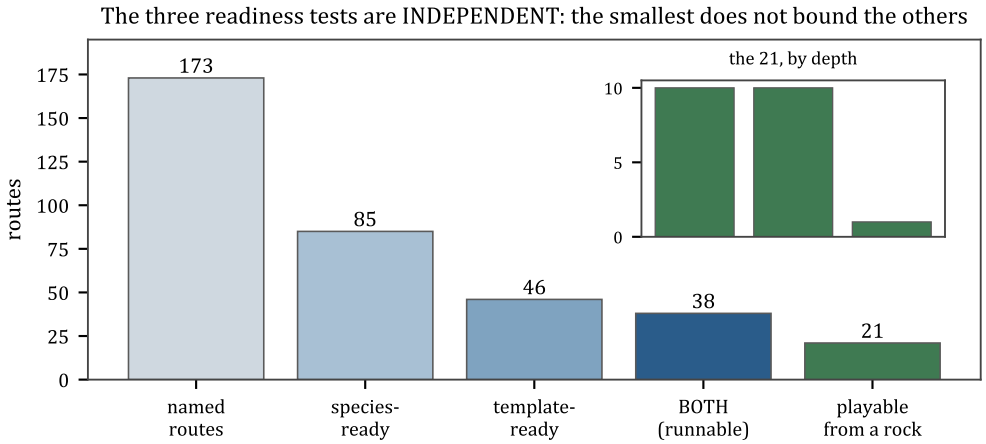

Figure 29.1: The funnel, and it is not a funnel.

	routes
named routes in the corpus	173
species-ready	85
template-ready	46
BOTH — the one to quote	**38**
playable from natural materials	21

46 IS NOT WHAT COULD RUN; 38 IS — AND EVEN 38 IS AN UPPER BOUND

The three columns answer **independent** questions and the smallest does not bound the others: a route needs a template for every step *and* a price for every species. **Eight of the 46 template-ready routes have a refused species** and cannot run.

Nothing computed the intersection until milestone S6.

And 38 is an *upper bound on what runs*, not a measured count: a class is credited when a template would fire on the right substrate **at all**, which is not the same as running. One route is the standing proof of that difference.

29.4 Five instrument findings, which are the real content

The coverage audit has been wrong more often than the engine has, and each correction is a lesson about measuring your own work.

1. A REACTION CLASS MUST NAME A MECHANISM, NOT AN OUTCOME

A template is *SMARTS on a mechanism*. So a class named for what a step *achieves* cannot answer "is there a template for this", and four classes in the original corpus were outcome labels spanning several mechanisms each. **32 rows were re-labelled** to the mechanism their own reactants and products show.

The cost of not doing it: crediting six proton-transfer templates was supposed to take covered steps from 21 to 46 in one line. That arithmetic needed `deprotonation` (6 steps) to be proton transfer. **Five of its six rows are carbanion generation** — precisely the capability with no template. Crediting the class would have made the audit *less* truthful.

Two rules follow. **The class says what the mechanism is; whether a reagent is priced is a species question**, counted separately, or one gap is double-counted as two. And **a step's NAME can lie; its reactants cannot** — one route's step 1 is called "alkoxide formation" and reads `phenol + NaOH -> sodium-phenoxide`, which is a phenoxide, which *is* covered.

2. THE CLASS DENOMINATOR MOVES, AND THAT IS CORRECT

Because a class is a mechanism claim, reading a class's rows sometimes splits it: the denominator went 212 → 224 over five milestones and is 240 now. A coverage percentage whose denominator is itself a live measurement has to be quoted with its date.

One split had a **negative** effect on the headline — `combustion` turned out to be six rows and five mechanisms, credited since the first milestone to a template that fires on two of them — and that is a split doing its job.

3. SPECIES-READY WAS BLIND TO A WHOLE PROVIDER

It asked only the three ideal-gas providers, which refuse an ionic lattice **by name and correctly**. But `mineral_data` had been pricing those on the solid basis for milestones. 19 compounds and 16 routes moved on that one line — including a route the project had already declared complete and whose example ran (Chapter 17).

4. THE RECORDED SIZE OF A GAP WAS ITSELF WRONG, BY 60%

Twice. Once because a comparison was made on *raw* rather than canonical SMILES; once because a tier classifier was **parsing prose** out of provenance strings, which the properties layer had already written down would fail.

An instrument's own output is data and needs the same auditing as the thing it measures.

5. NOTHING HAD EVER CHECKED THAT A CATALOG ROW BALANCES

75 of 367 rows do not. The corpus is hand-authored data and had never been subjected to the conservation check that every template has passed since Layer 3 was written.

29.5 And two findings about generated files

ROUTE_INDEX.MD WAS STALE BY THREE MILESTONES AND NOBODY NOTICED

It is the one generated file no audit reads — the coverage audit parses the raw route steps directly — so a stale index changes no measured number and produces no failure. It had not been regenerated since the initial commit while the underlying data was re-labelled three times. Regenerating it moved **21 class labels**.

Anyone who read that index to find a step's class over that period got a stale answer, silently.

AND COVERAGE_REPORT.MD WAS NOT BYTE-STABLE

`sorted(covered, ...)` sorted a *set* with no tie-break, so regenerating it produced about 17 lines of `PYTHONHASHSEED` noise with every number identical — enough to hide a real one-line change in review, which is what regenerating it is *for*.

It is now byte-identical across hash seeds. If a regeneration ever produces a diff again, that diff is real.

29.6 The finding the report itself does not give

The coverage report ranks missing classes by **frequency**. That is the wrong ranking for deciding what to build, because a class used in many routes may unlock none of them — those routes each need three other things too.

Ranked by **marginal unlock** instead:

- 127 routes have at least one gap, and **44 of them are one class away — from 35 different classes.**
- The best single class unlocks **3 routes** on the template column, and **2** that could actually run.
- Twelve templates take template-ready from 46 to 68, and the gain falls away fast after the first few.

AND THAT CURVE OPTIMISES THE OVERSTATED COLUMN

Its totals are template-ready, and a route also needs every species priced. Its top three rows are `isomerisation` at +3 unlocked / **2 runnable**, `catalytic-air-oxidation` at +3 / **1**, and `pyrolysis` at +2 / **1**.

So the greedy curve's own ordering is *not* the work order — which is why the report prints a `RUNNABLE` column beside the `ALONE` column and says to read the second one.

And the two rankings barely overlap:

ranked by routes it would make RUNNABLE	ranked by steps it covers
isomerisation (2)	nucleophilic-substitution (6 steps)
catalytic-air-oxidation (1)	radical-polymerisation (6)
metal-ion-aldehyde-oxidation (1)	carbanion-generation (5)
molten-salt-electrolysis (1)	polycondensation (5)

Several of the most-used missing classes unlock **zero** routes on their own, because the routes needing them each need three other things too.

THERE IS NO LEVER

"Full template coverage" is a ~150-template grind **with no bottleneck to attack**. Twenty templates in one milestone took the route count from 7 to 25, and the reason it took twenty rather than five is the finding: *the gap is a long tail*. Before that milestone, 63 routes sat one class away from 50 distinct classes; today 44 sit one class away from 35.

Eighteen routes cost twenty templates, and the next eighteen will cost about the same. So: **plan for a target ("twenty playable routes"), never for completeness.**

29.7 The species side pays as a multiplier, not as a headline

One more measured shape, and it changed how work is scheduled.

Curating nine bare elements moved `species-ready` from 63 to 77 and moved the intersection by **exactly zero** — and that was *predicted before it was done*, because none of the 15 routes it unblocked were template-ready.

What it did move is the **shape** of the queue: six reaction classes went from 0 runnable routes to 1, and one went from 1 to 2.

THE POINT

Species work is a **multiplier on template work** rather than a headline of its own. A species job should follow its template, not precede it.

CHAPTER 30

The tech tree: what can you make starting from a rock?

Chapter 29's reports ask whether the *engine* can do a reaction and whether the *corpus* has a route. Neither asks the question a player would ask.

`PLAYABLE.md` does, and it is the only artefact in the repository whose generation actually **runs** chemistry.

30.1 The answer

tier	routes	meaning
1 — from the ground	10	every feedstock and every catalyst is a natural material
2 — one step up	10	needs the output of a tier-1 route
3 — two steps up	1	needs the output of a tier-2 route
runnable but unfed	23	the engine can run it; nothing can supply it
not runnable	129	see the coverage report
	173	

21 of 173 named routes are playable from natural materials, against a stated goal of about 40. The deepest chain in the corpus is **three tiers**.

> **THE TECH TREE IS A SHALLOW BUSH, NOT A TREE**
>
> Ten of the 21 playable routes are tier 1 — they touch nothing another route made. The corpus is not a connected progression that happens to be short; it is a **fan of one-step routes off the ground with one thin chain hanging off it.**
>
> That is a different problem from "not enough routes", and it changes what to build: the shortage is *connections*, not leaves.

30.2 The one hand judgement, printed so it can be argued with

A species is NATURAL if a player could obtain it without running any chemistry — dig it, pump it, breathe it, press it out of a plant, or scrape it off an animal. Nothing about the engine or the catalog decides this. It is a game-design decision about where the tech tree starts, and it is the single input that most changes every number in the file.

45 species are declared natural, in four groups:

- **air and water** (4): CO_2, N_2, O_2, water.
- **native elements and rocks you can dig** (26): limestone, gypsum, rock salt, pyrite, galena, cinnabar, sphalerite, haematite, pyrolusite, saltpetre, native sulfur, graphite, gold, silver, fluorspar, cryolite, sand, bauxite, phosphate rock, borax, magnesite, anhydrite, covellite, pyrrhotite, melanterite, Chile saltpetre.
- **pressed, fermented or scraped off something living** (15): turpentine's α-pinene, clove oil's eugenol, citronellal, glucose, cellulose, hide collagen, coal.

The goal says ~10, so this list is already generous by a factor of four — and therefore **21 is an upper bound on playability, not a lower one.**

30.3 The deep chain, run end to end

This is the part that could not be answered by a static scorer, and running it produced four findings.

Tier 1 — the zinc retort. 10 L sealed, 1400 K. Sphalerite plus graphite plus air:

species	mol	
zinc	0.032793	the target
carbon monoxide	**0.054290**	**a byproduct, and the whole of tiers 2 and 3**
sulfur dioxide	0.032793	a byproduct

> **THE RETORT MAKES MORE CARBON MONOXIDE THAN ZINC, AND NOTHING ELSE MAKES ANY**
>
> It is the only carbon-monoxide source a player can reach; it is **not charged** — carbon burns in the blast and the Boudouard reaction hands the CO back — and **three tier-2 routes and one tier-3 route all want it.**
>
> A reachability scorer says "carbon monoxide is on the shelf" and attaches no quantity to that at all.

Tier 2 — the copper smelter, on the retort's own CO.

CO charged	copper
0.054290 (one retort)	0.039995
0.108580 (two retorts)	0.039996

Doubling the CO changes the copper in the sixth decimal, so this charge is **ore-limited, not CO-limited**. That is the *opposite* of what the contention above suggests, and only running it settles which.

Tier 2 — the water-gas shift, on the same CO. Gives 0.053445 mol hydrogen — and **consumes** the carbon monoxide to get there. The shift, the smelter and the methanol synthesis are not three routes sharing a shelf entry; they are three claims on one retort's gas.

Tier 3 — methanol, where the catalyst is the gate.

copper / mol	methanol / mol	conversion
0	0.000000	**nothing at all**
0.0001	0.000209	0.38%
0.0010	0.001669	3.08%
0.0100	0.004127	7.60%
0.039995	0.004154	7.65% — *smelted at tier 2*
0.1	0.004157	7.66%

> **THE ENTIRE THIRD TIER OF THIS CORPUS IS ONE COPPER CATALYST**
>
> Methanol needs no tier-2 *reagent*: its carbon monoxide is tier 1, and its hydrogen is tier 1 too, because the chlor-alkali cell throws hydrogen off as a byproduct of making caustic soda from rock salt.
>
> It is tier 3 for exactly one reason — **its catalyst has to be smelted first, and smelting it needs the byproduct of smelting a different metal.** Grant a player free copper and methanol moves to tier 2 and the corpus has no third tier left.

And the gate **saturates well below the smelter's output**: 0.01 mol of copper already reaches 99.3% of the reference rate, so one ore charge yields about four times more catalyst than the route needs. The catalyst is a *gate*, not a rate multiplier, so a player needs to *reach* copper and does not need to stockpile it.

> **WHAT DOES BITE IS SCALE, AND IT IS THE POINT OF RUNNING ANY OF THIS**
>
> At the retort's own scale the methanol conversion is **7.7%**. The same route, same template, same catalyst loading, at the corpus's own declared charge of 3 mol CO + 12 mol H_2 gives **2.994 mol — 99.8%.**

Methanol synthesis is pressure-driven, and one zinc retort is not a pressure vessel.

"Reachable" and "worth doing" are different questions, and a static scoreboard can only answer the first.

30.4 A warning printed on every number in that file

A TRAP THIS PROJECT ACTUALLY FELL INTO

Every yield in `PLAYABLE.md` is a property of *the declared constants on the day* and of *the conditions beside it*, not of the route. One milestone changed no species, no template and no route, and moved one substrate's rate by **2400**× while leaving three nitration yields identical to four decimal places.

A yield there is evidence that a route **works**. It is not a corpus property, and it will move under sessions that were not about it.

30.5 Measuring the scoreboard itself

The scoring rules are hand-written judgements, and a granularity audit went looking for how wrong they are.

THE BOTH COLUMN UNDERSTATES BY FIVE, AND FIVE IS SMALL

137 of 142 gaps are real work. That is the useful finding: the instrument is approximately right, so the queue it produces can be trusted.

Three specific faults, all caught by *running* things rather than by reading:

1. THE SCORER SCORES ROWS WHILE A ROUTE IS A DAG

A route is a directed graph of steps with shared intermediates. Scoring it as a list of rows loses the structure, and structure is exactly what tier depth is.

2. A SCORER THAT LETS YOU CHARGE THE TARGET CREDITS EVERY RECYCLE LOOP

If the feasibility check may charge the route's own product as a reagent, then any route with a recycle loop trivially passes. Three false credits came from this, and all three were caught by running the route rather than by scoring it.

3. FIXING ONE SCORING RULE MASKED ANOTHER

Two rules were wrong. Fixing the first hid the second, because the first's error compensated for it. **Measure two rules as a grid**, not one at a time — which is the same reasoning as the three-point tolerance sweep in Chapter 20.

30.6 What the queue looks like now

The project's current work order is a per-class table in `PLAYABLE.md` §8b: 22 rows, projected to take playability from 21 to about 45.

Six sessions have worked through it — oil of vitriol from a rock, phosphate rock digested in sulfuric acid, vanillin from clove oil, the ABE fermentation, the sugar-to-furan dehydrations — and the shape of the findings has been consistent enough to be worth stating as a pattern:

WHAT A CONTENT SESSION ACTUALLY FINDS

1. **The blocker is rarely where the work order says.** One route was blocked on a *price for a species that is not in its chemistry*; another on a pKa sitting in a different table entirely.
2. **A class refused in an earlier session is often refused on the evidence of one of its two rows.** This happened three sessions running.
3. **A +0 row is what makes the +1 row mean anything.** Doing the work that changes no headline number is how you find out that the headline number is measuring the right thing.
4. **Granting a work-order row makes the work order longer**, because a newly reachable route unblocks routes that then become visible as blocked.

And one finding from that series that is pure engine, and the best single illustration of why an integration test is not enough:

THE ENGINE COULD NOT FERMENT SUGAR IT HAD INVERTED ITSELF

A flag on the template machinery meant a template could not run on a species that *another template had made*. Invisible to every single-template test, because every such test charges its own substrate.

Removing it removed an accidental generation cap, exposed a preparation that was making an ester in caustic soda, and turned up a green test that had been resting on the order of two identical rows.

PART VI

Limits

CHAPTER 31

What it cannot do, and what each gap would cost

Every item here is deliberate, measured, and has a stated route out. They are grouped by what kind of thing is missing, because that determines what fixing one would take.

31.1 Missing accuracy

Group-contribution formation data is the dominant error in K. With the standard state fixed, Fischer esterification sits at 8.1 against a measured ≈ 4. Joback's ΔG_f carries several kJ/mol of uncertainty, which is a factor of 2–4 in K (Chapter 4). *Route:* more curated ΔG_f, or wider Benson coverage. Ongoing.

UNIFAC understates how fast an associating solute's solubility climbs with temperature. Benzoic acid in water is 0.95× measured at 298 K and 0.48× at 333 K. The absolute scale is right; the slope is not.

Gas solubility is transferred, not measured. Only the aqueous Henry constants are experimental; every other solvent is predicted through γ^{∞}, and runs about 25% low for the common solvents and 2.6× high for acetone.

Carbon monoxide's reference state fits poorly — 3.6% against 0.15% for O_2, because PSRK's parameters for it are strongly quadratic in T. Reported by the activity model rather than absorbed.

The gas extension mixes two regressions. Organic pairs are UNIFAC-VLE, gas pairs are PSRK. Sound because PSRK's organic backbone *is* UNIFAC's (1124 of 1174 pairs bit-identical), but it is a join, not one self-consistent fit.

31.2 Missing models

Equilibrium is on a concentration basis, not an activity basis. γ corrects phase equilibria and solubility but **not the reaction quotient**, so a strongly non-ideal mixture equilibrates to the wrong quotient. *Route:* rates on activities — which redefines every rate constant's units, and is a distinct project.

No electrolyte activity model. Ions sit at $\gamma = 1$ within a phase, so ionic strength does not affect anything and salting-out does not exist. Debye–Hückel or a UNIFAC extension is the fix. Note this is a *different* gap from ion transfer between phases, which the Born term does cover (Chapter 10).

No nucleation barrier or metastable zone. Precipitation is ungated by design, so a supersaturated solution crashes out instantly. No seeding, no supersaturation, no oiling-out. Named as an engine gap.

Rate laws are power-law mass action, so Langmuir–Hinshelwood and Michaelis–Menten have nowhere to live. The measured consequence: a solid catalyst is strictly first order in catalyst amount for ever, so ten times the iron is ten times the rate at any loading. Right at low coverage, wrong at high (Chapter 18).

Non-aqueous acidity. The engine's pH floor is -0.79, set by the concentration of water itself, and a real nitrating mixture needs about -9.4. Fixing it means a Hammett acidity function H_0, not a parameter (Chapter 10).

No melt phase. A molten salt is not a phase this project has, which is why two industrially important electrolysis cells cannot be expressed at all regardless of their species.

Energy is not conserved to an invariant. There is no equivalent of `conservation_report` for energy, and a leak is therefore invisible to the instrument that exists to catch leaks (Chapter 27).

31.3 Missing representations

Polymers and extended solids need a different representation entirely — chain-length *distributions*, not graphs. That is 12 routes in the corpus, and the design has met the problem three times: bounded discovery, the lattice question, and the coal/cellulose markers that have no molecular graph at all.

No tautomer resolution. Keto and enol forms are separate species.

Stereochemistry is distinguished but not controlled. A template rewrite can scramble a centre.

Joback gaps: no anhydride, sulfoxide/sulfonyl, formamide or aryl-aldehyde groups, and no metals, Si, B or P. Curated data covers the common reagents; Benson is the general fix and it inherits the critical-property gap that Chapter 13's Wilson–Jasperson chain exists to close.

A cell's two electrodes cannot be paired freely. Each pairing is a separate whole-cell template.

31.4 Missing budgets

Nothing budgets current. Far above the decomposition potential every electrode reaction runs out of barrier and the selectivity goes with it. A real cell is transport-limited there; this one is not limited by anything.

Nothing budgets surface area. The shrinking-core law was refused for numerical reasons (Chapter 21), so a crystal's reactivity does not fall as it is consumed.

31.5 The honest headline

THE POINT

The mechanism is fully general; the library is not. A player can mix two arbitrary things and the machinery will discover what happens — that is architecture, and it works. What is short is *content*: 47 templates cover **59 of 240** reaction classes and 124 of 377 corpus steps, and closing the rest is a ~150-template grind with no bottleneck in it (Chapter 29).

That is a content problem, not a design one, and it is the correct kind of problem to have after this much work.

31.6 And the sentence to keep

Everything in this chapter is *stated*. None of it is absorbed into a fudge factor, and several items exist as printed refusals rather than as silent approximations — a species held at $\gamma = 1$ is named, a capped rate constant is reported, a compilation-tier datum is stamped, an unbalanced catalog row is counted.

THE POINT

That is the project's actual thesis, more than emergence is. A simulator's numbers are only worth what its account of its own uncertainty is worth, and the only defence against a confidently wrong number is to write down, beside every choice, what was measured and what was refused.

PART VII

Appendices

APPENDIX A

Glossary

Chemistry terms first, then the project's own vocabulary.

A.1 Chemistry

Activity (a_i) — the effective concentration that appears in an equilibrium expression: $a_i = \gamma_i x_i$. Reduces to mole fraction in an ideal mixture.

Activity coefficient (γ_i) — the correction from ideal. Above 1 means the species is less comfortable in the mixture than in its own pure liquid. See Chapter 8.

Acid / base — a proton donor / acceptor.

Activation energy (E_a) — the barrier a reaction must clear.

Antoine equation — the three-parameter fit $\log_{10} P^{\text{sat}} = A - B/(C + T)$ used for every volatile species here.

Aromatic — a ring with delocalised electrons (benzene), unusually stable.

Arrhenius equation — $k = Ae^{-E_a/RT}$.

Azeotrope — a liquid mixture that boils without changing composition; distillation cannot pass it.

Bond — a shared pair of electrons; a bound state, 300–500 kJ/mol deep.

Born energy — the electrostatic cost of charging an ion inside a dielectric. Used here for ion *transfer between phases*.

Catalyst — a species that speeds a reaction and is not consumed. Appears on both sides, so stoichiometry 0 and rate-law exponent 1.

Clausius–Clapeyron — $\mathrm{d}\ln P^{\text{sat}}/\mathrm{d}T = \Delta H_{\text{vap}}/RT^2$.

Concentration — amount per volume; molarity is mol/L, written M.

Detailed balance — $k_f/k_r = K$ at every temperature. Chapter 6.

Elementary step — a reaction that really is one collision, so that stoichiometry and rate-law order coincide.

Enthalpy ($H = U + pV$) — heat absorbed at constant pressure.

Entropy (S) — $R \ln \Omega$ per mole.

Ester — the product of an acid and an alcohol losing water; ethyl acetate is the running example.

Equilibrium constant ($K = e^{-\Delta G^\circ/RT}$) — where a reversible reaction stops.

Exothermic / endothermic — releases / absorbs heat.

Fusion — melting. ΔH_{fus}, T_m.

Gibbs free energy ($G = H - TS$) — minimised at constant T and p.

Hammett relation — $\log_{10}(k/k_0) = \rho \sum \sigma$; how substituents on an aromatic ring change a rate.

Henry's law — $p_i = x_i H_i$ for a gas dissolved in a liquid. Same functional shape as Raoult, different constant.

Ion — a charged atom or molecule.

Ionic lattice — an extended crystal of alternating ions; salt. Not a molecule and has no useful graph.

Isomers — same formula, different substance: *structural* (different connectivity), *stereo* (different arrangement in space), *tautomers* (interconverting by moving an H and a bond).

Le Chatelier's principle — a system at equilibrium shifts to oppose a change. Quantitatively, the van 't Hoff equation.

Mole — 6.022×10^{23} entities.

Mole fraction (x_i) — $n_i / \sum n_j$.

Oxidation / reduction — losing / gaining electrons.

pH — $-\log_{10}[\mathrm{H_3O^+}]$.

pKa — $-\log_{10} K_a$; small means strong acid.

Raoult's law — $p_i = x_i P_i^{\text{sat}}$ for an ideal liquid.

Reaction quotient (Q) — the same product of activities as K, evaluated away from equilibrium.

Solubility product (K_{sp}) — the equilibrium constant for a lattice dissolving into ions.

Standard state — the reference condition against which formation data is quoted. Ideal gas at 1 bar, or the pure liquid, or infinite dilution in water. Getting this wrong is the most consequential silent error in the project's history.

Stoichiometry — the integer coefficients in a balanced equation.

Transition state theory — the framework that identifies Arrhenius's A with $(k_B T/h)e^{\Delta S^\ddagger/R}$.

van 't Hoff equation — $\mathrm{d}\ln K/\mathrm{d}T = \Delta H^\circ/RT^2$.

Vapour pressure (P^{sat}) — the pressure of vapour in equilibrium with its own liquid.

A.2 The project's vocabulary

Affinity form — the rate law used for a reversible reaction inside a crystal: $k[\,\text{units}_f - \text{units}_r e^{\ln Q - \ln K}]$.

BOTH column — the intersection of *species-ready* and *template-ready*; the number to quote for how many corpus routes can run. Currently 38 of 173.

Class — a reaction *mechanism* label on a corpus step. Never an outcome label; 32 rows were re-labelled to enforce that.

Concrete reaction — what a template produces once it has matched real species: a reactant multiset, a product multiset, and kinetics.

Discovery — the fixpoint that finds every reaction a set of templates can run on a set of species.

Dryout band — the range of tiny liquid holdings over which the liquid-phase terms switch off. Historically the source of a matter-creating bug.

Edge — a connection between vessels in a rig: `VAPOUR`, `DRAIN`, `THERMAL`, `METER`.

Emergent — a behaviour that follows from integrating existing terms rather than from a rule. Chapter 28 lists twenty-four.

Event — a timestamped, serialisable player action; the only thing that may mutate a vessel, and it fires only between integrations.

KineticArrays / PhaseArrays — the numpy handoff from the chemistry layers to the numeric core.

Lattice — an ionic solid, held as a single species that may react and (with one recorded exception) may not dissolve or boil.

Marker — a corpus entry with no molecular graph: `coal-marker`, `collagen-marker`. Cannot be simulated, deliberately present so the corpus is honest about what it contains.

Playable — reachable from natural materials by running routes. 21 of 173.

Provenance / tier — where a number came from: `measured`, `mineral`, `compilation`, `benson`, `joback`, `ion`, `nonvolatile`, `refused`.

Refusal — a named, explained "no" from a provider or a guard. Treated as content rather than as an error.

Rig — vessels plus typed connections, integrated as one system.

Scenario / script — what a save contains. A run is a pure function of the two.

Seam — an inversion boundary designed to be replaceable: `matter` hides RDKit; `numerics` sees only arrays.

Species-ready — every species in a route resolves to a full property set.

Template — a SMARTS graph-rewrite rule plus forward kinetics.

Template-ready — every reaction class in a route has a template.

`wait_until` — the verb that turns a bench instruction into a root of the state vector.

APPENDIX B

Symbols, units and constants

B.1 Unit conventions

Internal units are SI-ish and unit objects never enter the numeric core. Units live only at domain boundaries.

quantity	unit
amount	mol
concentration	mol/L, written M
mole fraction	dimensionless, $\sum_i x_i = 1$
temperature	K
energy	J/mol (tables often quote kJ/mol; the code stores J/mol)
entropy, heat capacity	J/(mol K)
pressure	bar (1 atm = 1.01325 bar)
volume	L
time	s
power	W
UA	W/K
potential	V
rate	mol/(L s) for solution, mol/s for a surface reaction
Arrhenius A	whatever makes rate come out in the above

A TRAP THIS PROJECT ACTUALLY FELL INTO

The units of A depend on the overall reaction order: s^{-1} for first order, L mol^{-1} s^{-1} for second, and $(L/mol)^8$ s^{-1} for a ninth-order declaration. A pre-exponential quoted without its order is meaningless, and this is why the rate-ceiling audit compares against a limit computed *per molecularity*.

B.2 Constants

symbol	value	
N_A	$6.02214076 \times 10^{23}$ mol^{-1}	Avogadro

symbol	value	
R	8.314462618 J/(mol K)	gas constant, $= N_A k_B$
k_B	1.380649×10^{-23} J/K	Boltzmann
F	96,485 C/mol	Faraday, $= N_A e$
h	$6.62607015 \times 10^{-34}$ J s	Planck
P°	1 bar	standard pressure
T°	298.15 K	standard temperature
RT at 298 K	2.479 kJ/mol	the number to keep in your head
$k_B T/h$ at 300 K	6.25×10^{12} s^{-1}	the attempt frequency

B.3 Symbols

Thermodynamic

H, ΔH	enthalpy; ΔH_f of formation, ΔH_{vap} of vaporisation, ΔH_{fus} of fusion
S, ΔS	entropy
G, ΔG	Gibbs free energy; ΔG_f of formation
μ_i	chemical potential of species i
C_p	heat capacity at constant pressure
K	equilibrium constant
Q	reaction quotient
K_a, pK_a	acid dissociation constant
K_{sp}	solubility product
$^\circ$	superscript: standard state
‡	superscript: transition state

Kinetic

$k(T)$	rate constant
A	Arrhenius pre-exponential
E_a	activation energy
n	modified-Arrhenius temperature exponent, $k = AT^n e^{-E_a/RT}$
α	Evans–Polanyi transfer coefficient
ρ, σ^+	Hammett reaction constant and substituent constant
r_j	rate of reaction j
ν_i, Δ	stoichiometric coefficient; the (m, n) matrix

α_{ji}	rate-law exponent of species i in reaction j
Δn	$\sum_i \nu_i$, the change in mole count

Phase and composition

n_i	moles of species i
C_i	concentration
x_i, y_i	liquid and vapour mole fraction
ϕ_i	volume fraction
γ_i	activity coefficient; γ^* unsymmetric, γ^∞ at infinite dilution
a_i	activity, $\gamma_i x_i$
P^{sat}, p_i	saturation vapour pressure; partial pressure
H_i	Henry constant
T_b, T_m	normal boiling point, melting point
T_c, P_c, V_c	critical constants
ω	acentric factor
ε	relative permittivity
R_k, Q_k, a_{mn}	UNIFAC group volume, surface area, interaction

Vessel and rig

$\mathbf{y}$	state vector, $[n_{L1} \mid n_{L2} \mid n_G \mid n_S \mid T]$, length $4n+1$
k_{la}	mass-transfer coefficient for evaporation
k_{diss}	dissolution rate constant
k_{lle}	liquid–liquid transfer rate constant
UA	heat-transfer coefficient to the environment
Q_{input}	hotplate power, W
q_{rxn}, q_{vap}, q_{fus}, q_{loss}, q_{vent}	the energy-balance terms
E	cell potential, V
η_a	activation overpotential, V

B.4 Numbers worth remembering

RT at 298 K	2.48 kJ/mol
6 kJ/mol in ΔG°	a factor of 10 in K

10 kJ/mol in E_a	a factor of 55 in rate at 298 K
a 10 K rise	roughly doubles a rate
bond energy	300–500 kJ/mol
ΔH_{vap}	20–50 kJ/mol for common solvents
ΔS_{vap}	$\approx +90$ J/(mol K) (Trouton)
diffusion limit in solution	$\sim 10^{10}$ L mol^{-1} s^{-1}
bond vibration	$\sim 10^{13}$ s^{-1}
water's permittivity	78
toluene's permittivity	2.4

APPENDIX C

A map from ideas to files

Sizes are lines, as a rough guide to where the weight is.

C.1 `src/chemsim` — the engine (about 46,000 lines)

C.1.1 Layer 0 — `matter/`

file	lines	what
`molecule.py`	263	molecular graphs, canonical identity, the RDKit boundary

C.1.2 Layer 1 — `properties/`

file	lines	what
`physical_data.py`	13,736	**generated.** Measured $T_b/T_c/P_c/V_c$, 1,239 species
`benson_data.py`	2,413	**generated.** Benson group values from RMG
`unifac_data.py`	1,520	UNIFAC groups and the interaction matrix
`mineral_data.py`	940	**generated.** Ionic lattices on the solid basis
`solid_state.py`	922	reactions *inside* a crystal (the affinity form)
`thermochemistry.py`	853	the provider: tiers, refusals, provenance
`benson.py`	818	Benson group additivity
`psrk_data.py`	782	PSRK's gas extension to UNIFAC
`ion_data.py`	684	**generated.** Aqueous ions on the conventional scale
`surface.py`	641	a gas reacting *at* a crystal's surface

file	lines	what
element_data.py	492	**generated.** The elements; the floor every chain stands on
electrolyte.py	488	pKa-anchored ions, and pH as ordinary chemistry
dielectric_data.py	477	permittivities and Born radii
critical.py	472	Wilson–Jasperson, Fedors, Lee–Kesler
dielectric.py	448	the Born transfer term
volatility.py	437	Antoine, and Henry through the same array
formation_data.py	367	curated $\Delta H_f / \Delta G_f$, in both standard states
condensed.py	343	Rackett molar volume, Rowlinson–Bondi liquid C_p
solubility_product.py	306	K_{sp} from two tables on one basis
unifac.py	280	the parameter block, not the evaluation
fragmentation.py	262	the shared greedy priority matcher + search fallback
standard_state.py	244	ideal gas → liquid
joback.py	121	Joback group contribution
joback_data.py	111	the Joback table

C.1.3 Layer 2 — reactions/

file	lines	what
synthesis.py	2,626	the named-route template library
library.py	845	the curated templates the engine was built on
template.py	563	ReactionTemplate: SMARTS + kinetics + every declaration
hammett.py	450	ring deactivation, on the σ^+ scale

file	lines	what
thermo.py	434	reaction $\Delta H/\Delta G/K$, and detailed balance
electrochemistry.py	301	four whole-cell templates
reaction.py	75	ConcreteReaction

C.1.4 Layer 3 — network/

file	lines	what
builder.py	711	discovery, conservation checks, KineticArrays

C.1.5 Layer 4 — numerics/

file	lines	what
vessel_integrator.py	3,103	**the flask.** The $4n+1$ RHS, every term
rig_integrator.py	708	several coupled vessels as one system
lle.py	369	the phase-split decision (Michelsen tangent plane)
activity.py	360	UNIFAC + Born evaluation, per RHS call
jacobian.py	227	one bound SciPy lacks
integrator.py	96	the isothermal mass-action kernel

C.1.6 Layers 4.5–7

file	lines	what
vessel/vessel.py	2,563	assembly, reporting, pour_into, filter_into
engine/world.py	1,040	the stepper, the script, save/load
ui/session.py	579	the worker thread; no widgets
ui/app.py	572	the Tkinter view; never calls the engine

file	lines	what
vessel/conditions.py	359	the wait_until vocabulary
vessel/rig.py	343	glassware as a graph
ui/examples.py	240	four worked starting points
discovery/refine.py	183	rate-based network refinement
recipes.py	181	curated preparations, as data
engine/scenario.py	168	what a save contains
engine/events.py	108	the only things that may mutate a vessel

C.2 data/catalog/ — the coverage corpus

file	what
compounds/*.psv	1,583 compounds: id, name, SMILES, class, role, domains
routes.psv	173 route headers
route_steps.psv	377 steps, each with a mechanism class
COVERAGE_REPORT.md	**generated.** Can the engine do this chemistry?
ROUTE_INDEX.md	**generated.** Feedstocks → intermediates → products
PLAYABLE.md	**generated.** Can a player get there from a rock? Runs things.
README.md	the class-relabelling argument, and the two rules that follow

C.3 validation/ — the audits

Each file answers one question and most were written because a specific number turned out to be wrong. A representative selection:

file	asks
catalog_coverage.py	how much of the corpus resolves and runs
rate_ceiling.py	is any rate constant above the collision limit
tolerance_audit.py	which numbers are solver resolution rather than chemistry

file	asks
jacobian_bound.py	does the num_jac bound bind, and where
wall_clock.py	what does each operation actually cost
boiling_points.py	how far off were the estimates the measured table replaced
corpus_balance.py	do the catalog's own rows balance
granularity.py	is the playability scorer right
wait_conditions.py	what does each proposed wait_until look like on a real trajectory
process_losses.py	where does the yield actually go
unifac_gap.py	what is not decomposable, and what does that cost
physical_estimation.py	how good are Wilson–Jasperson and Fedors here

C.4 examples/ — runnable narratives

esterification, thermochemistry, vessel, workshop, activity, wait_until, multistep_prep, extraction, competing_pathways, fractional_distillation, plate_column, named_routes, lime_cycle, mercury_retort, roasting_and_the_catalyst_gate, electrolysis_cell, oil_of_vitriol, dropping_funnel.

C.5 tools/ — the generators

build_benson_data.py	build_ion_data.py	build_playable.py
build_dielectric_data.py	build_mineral_data.py	build_route_index.py
build_element_data.py	build_physical_data.py	catalog.py

Each generated table says **GENERATED — do not hand-edit** at the top and names its build script, because the derivation is where the reasoning lives and a generated file cannot carry the collision report its build prints.

C.6 The project's own documents

file	lines	what
README.md	625	the thesis and the current state
MILESTONES.md	6,545	**the primary record.** Every milestone, with its measurements and its refusals

file	lines	what
`HANDOFF.md`	7,751	session-to-session context
`NEXT_SESSION.md`, `NEXT_PROMPT.md`	2,900	the work queue
`GAME_DESIGN.md`	581	what this is ultimately for
`ASSESSMENT.md`	327	an outside-view review
`EQUIPMENT_PLAN.md`, `EQUIPMENT_CATALOG.md`	788	glassware, planned and built

APPENDIX D

Running it

D.1 Install

```
python -m pip install -e ".[dev,viz]"
```

Runtime dependencies are numpy, scipy and RDKit, and nothing else. `dev` adds pytest and `thermo` — which is a **test-only oracle**: the suite cross-checks this project's Joback table, its fragmentation, and every UNIFAC and PSRK parameter against it. `viz` adds matplotlib. Python 3.11+.

Tkinter is standard library, which is most of why the interface is Tkinter.

D.2 The window

```
python -m chemsim.ui          # or the chemsim-ui script
```

Glassware, the selected vessel's temperature / pressure / pH / phase volumes / per-phase composition, the engine's own reports, and the recipe as it accumulates. Four worked starting points including the benzoic-acid preparation.

Watch the **cost meter** — it is the single most useful thing on screen for understanding the engine's behaviour.

D.3 Examples, roughly in order of what they teach

```
python examples/esterification.py     # graph -> template -> network -> equilibrium
python examples/thermochemistry.py    # properties and equilibrium from structure alone
python examples/vessel.py             # boiling, boiling dry, self-heating, distillation
python examples/workshop.py           # crystallisation, melting, pH/titration, the engine
python examples/activity.py           # azeotropes and real solubilities, from UNIFAC
python examples/wait_until.py         # a recipe with no durations in it
python examples/competing_pathways.py # the same flask at three temperatures
python examples/extraction.py         # two liquid layers and an acid/base workup
python examples/fractional_distillation.py
python examples/plate_column.py       # eight plates, reflux ratio 5
python examples/multistep_prep.py     # the benzoic-acid preparation, end to end
python examples/lime_cycle.py         # a reaction inside a crystal
python examples/roasting_and_the_catalyst_gate.py
python examples/mercury_retort.py     # a route that EMERGES from two declarations
python examples/electrolysis_cell.py
```

```
python examples/oil_of_vitriol.py
python examples/dropping_funnel.py    # drip it too fast and the pot runs away
python examples/named_routes.py       # the named historical routes, integrated
```

D.4 Tests

```
python -m pytest -q
```

THIS TAKES ABOUT TWELVE MINUTES

It is a real integration suite — it builds networks and integrates stiff systems — not a unit-test suite. Run it deliberately rather than in a loop, and prefer a targeted file while iterating:

```
python -m pytest tests/test_vessel.py -q
python -m pytest tests/test_conservation.py -q
```

Some tests worth knowing about by name:

file	what it pins
`test_conservation.py`	element and charge conservation across all four phase blocks
`test_detailed_balance.py`	$k_f/k_r = K$, and equilibrium reached from both directions
`test_joback.py`, `test_benson.py`	the group tables against the `thermo` oracle
`test_activity.py`	UNIFAC parameters, and the azeotrope
`test_standard_state.py`	the gas → liquid shift, and the pH invariants that must survive it
`test_competing_templates.py`	that ether beats ethene at 140 °C and loses at 180
`test_playable.py`	the tech-tree headline numbers and all four scoring rules
`test_ui.py`	the session layer, without opening a window

D.5 Regenerating the catalog artefacts

```
python tools/catalog.py                # structural validation only
python tools/build_route_index.py      # writes ROUTE_INDEX.md
python validation/catalog_coverage.py  # writes COVERAGE_REPORT.md + derived/
python tools/build_playable.py         # writes PLAYABLE.md -- ~1 min, RUNS the deep chain
```

A TRAP THIS PROJECT ACTUALLY FELL INTO

> Run **all** of them. `ROUTE_INDEX.md` went stale by three milestones because it is the one generated file no audit reads, so a stale index changes no measured number and produces no failure.

D.6 Regenerating the data tables

Each requires its upstream source and each prints an argument as it goes:

```
python tools/build_physical_data.py     # from `chemicals`, keyed off data/catalog
python tools/build_element_data.py      # from `chemicals` + `thermo`
python tools/build_mineral_data.py
python tools/build_ion_data.py
python tools/build_dielectric_data.py
python tools/build_benson_data.py       # needs an RMG-database clone
```

D.7 The audits

These are the files that found most of what is in Part IV of this manual. They are cheap to run and each answers one question:

```
python validation/rate_ceiling.py       # any rate constant above the collision limit?
python validation/tolerance_audit.py    # which numbers are resolution, not chemistry?
python validation/wall_clock.py         # what does each operation actually cost?
python validation/boiling_points.py     # 2 s; how far off were the old estimates?
python validation/corpus_balance.py     # do the catalog's own rows balance?
python validation/game_gates.py         # the standing verdicts, re-measured
```

D.8 Regenerating this manual

```
python docs/manual/make_figures.py      # ~2 min; several figures RUN the engine
bash   docs/manual/build.sh             # pandoc + xelatex -> chemsim-manual.pdf
bash   docs/manual/build.sh --figures   # both
```

Requires pandoc and a LaTeX distribution with xelatex. The sources are the markdown files under `chapters/`, with `preamble.tex` for the styling, `callouts.lua` for the coloured boxes, and `metadata.yaml` for the fonts and page setup.

D.9 A short orientation route, if you have an hour

1. `python examples/vessel.py` — watch a flask boil, boil dry and superheat, and note that no boiling point exists in the code.
2. `python examples/competing_pathways.py` — the same charge at three temperatures giving three different products.
3. Read `src/chemsim/numerics/vessel_integrator.py`'s docstring. It is the clearest single statement of what the project is.
4. `python -m chemsim.ui`, load the benzoic-acid preparation, and watch the recipe panel fill in as you work.

5. Read `MILESTONES.md` §M6 (a reaction inside a crystal) — it is the best single example of the project's habit of deciding a modelling question *by arithmetic* rather than by argument.